A Six-Platform Diagram Atlas

Mermaid, Excalidraw, PlantUML, D2, Diagrams (Python), and Graphviz Perspectives on a Phase 1 Combined IND/IDE Protocol of On-Premises LLM-Directed Robotic Pancreaticoduodenectomy with Perioperative Daraxonrasib (RMC-6236) in KRAS-Mutated PDAC

Kevin Kawchak, CEO ChemicalQDevice | kevink@chemicalqdevice.com | 10.5281/zenodo.20780121

Atlas version: v1.0.0 **Figures:** 120 (6 sections × 20) **Source protocol:** trial-protocol/final-protocol/publication
Palette: Corporate Blue `#00417A`, Professional Gray `#6C757D`, Classic White (inherited from `protostyle.sty`)

Independent research draft. Not an active IND or IDE, not an approved protocol, and not medical or regulatory advice; not endorsed by the FDA, NIH, HHS, an IRB, ICH, or any sponsor. Every figure in this atlas is an adaptation of data, structure, or argument already present in the parent protocol, rendered from a new perspective; clinical figures derive from the author's simulation sources and are illustrative unless tied to a cited reference. All 120 figures are vector TikZ and contain no raster images.

Keywords Diagram atlas; Mermaid; Excalidraw; PlantUML; D2; Diagrams (Python); Graphviz; TikZ; Pancreatic Ductal Adenocarcinoma; Robotic Whipple; Daraxonrasib; Physical AI; On-premises LLM; IND/IDE; Phase 1 Trial Protocol; VVUQ

How to read this atlas

The parent publication carries twenty TikZ figures. This atlas adds one hundred and twenty more, organised as six sections of twenty. Each section adopts the idiom of one open-source diagram platform and applies that idiom to the facets of the protocol which that platform is strongest at expressing, so that the same underlying trial is seen six different ways rather than six times the same way. No figure repeats a figure of the parent publication.

Because the figures are drawn natively in LaTeX with TikZ rather than rendered by the platforms themselves, they must not be called Mermaid diagrams, PlantUML diagrams, and so on. They are referred to instead as **mermaid-type**, **excalidraw-type**, **PlantUML-type**, **D2-type**, **Diagrams (Python)-type**, and **Graphviz-type** figures, and every caption states the type explicitly. The monospace tag in the top left of each frame names the native construct being reproduced, for example `flowchart TD`, `@startuml`, or `digraph { }`, so a reader can carry any figure back to its platform of origin.

Figure numbering restarts at Figure 1 in each of the six sections and ends at Figure 20, so a citation must name both the section and the figure, for example "Section 4, Figure 11 (D2-type)". The palette, the frame, the caption style, and the typographic settings are inherited from `protostyle.sty` of the parent publication and are collected in `dxstyle.sty`; the two publication hues are extended only by tints and shades of themselves, so the atlas and the protocol remain colour-consistent.

Every figure is drawn with PGF/TikZ [1]; no figure in this atlas is a raster image, and none of the six platforms is invoked at compile time. The quantitative content is adapted from the parent protocol and from the sources that protocol cites: the epidemiology and survival baseline [2], the contemporaneous Dutch nationwide robotic pancreatoduodenectomy comparators [3], the ISGPS fistula grading [4], the pan-KRAS dose anchor of the RASolute 302 programme [5], the early-feasibility convention for significant-risk devices [6], the electronic-records standard underlying the audit chain [7], the Physical AI adaptation of 21 CFR part 312 [8], the verification-before-generation framing [9], the reporting guidance applied to the

advisory component [10], and the author's robotic Whipple and daraxonrasib simulation from which the illustrative operative values derive [11].

Section	Referred to as	Platform strength the twenty figures exploit
1. Mermaid	mermaid-type	Decisions in time: flowchart, sequence, state, ER, Gantt, journey, and the quantitative primitives (pie, xychart, Sankey, packet, quadrant).
2. Excalidraw	excalidraw-type	Deliberately unfinished thinking: whiteboard sketches, sticky-note clustering, storyboards, matrices, and annotated napkin arithmetic.
3. PlantUML	PlantUML-type	Formal software notation: the full UML set plus timing, salt wireframes, work breakdown, ArchiMate layering, and C4 containers.
4. D2	D2-type	Nesting and tabulation: containers within containers, true grid layouts, SQL-table shapes, and stepwise layer storytelling.
5. Diagrams (Python)	Diagrams (Python)-type	Infrastructure reality: clustered deployment, data pipelines, network segmentation, failover, and lifecycle architecture.
6. Graphviz	Graphviz-type	Pure graph structure: directed acyclic graphs, record nodes, fault and event trees, bipartite maps, and radial or circular layouts.

Table of Contents

How to read this atlas **1**

1 Mermaid-type Diagrams **3**

2 Excalidraw-type Diagrams **18**

3 PlantUML-type Diagrams **35**

4 D2-type Diagrams **53**

5 Diagrams (Python)-type Diagrams **68**

6 Graphviz-type Diagrams **81**

References **99**

1 Mermaid-type Diagrams

Mermaid renders text into diagrams and is strongest where a protocol has to be read as a *sequence of decisions in time*: flowcharts, sequence diagrams, state machines, entity relationships, Gantt schedules, journeys, and the newer quantitative primitives (`pie`, `xychart-beta`, `sankey-beta`, `packet-beta`, `quadrantChart`). The twenty figures below take one Mermaid diagram type each and apply it to a facet of the protocol that the parent publication states in prose or in a table but never draws [12]. Because these are reproduced natively in LaTeX rather than rendered by the Mermaid engine, they are referred to throughout as **mermaid-type** figures; the monospace tag at the top left of every frame names the Mermaid construct being reproduced.

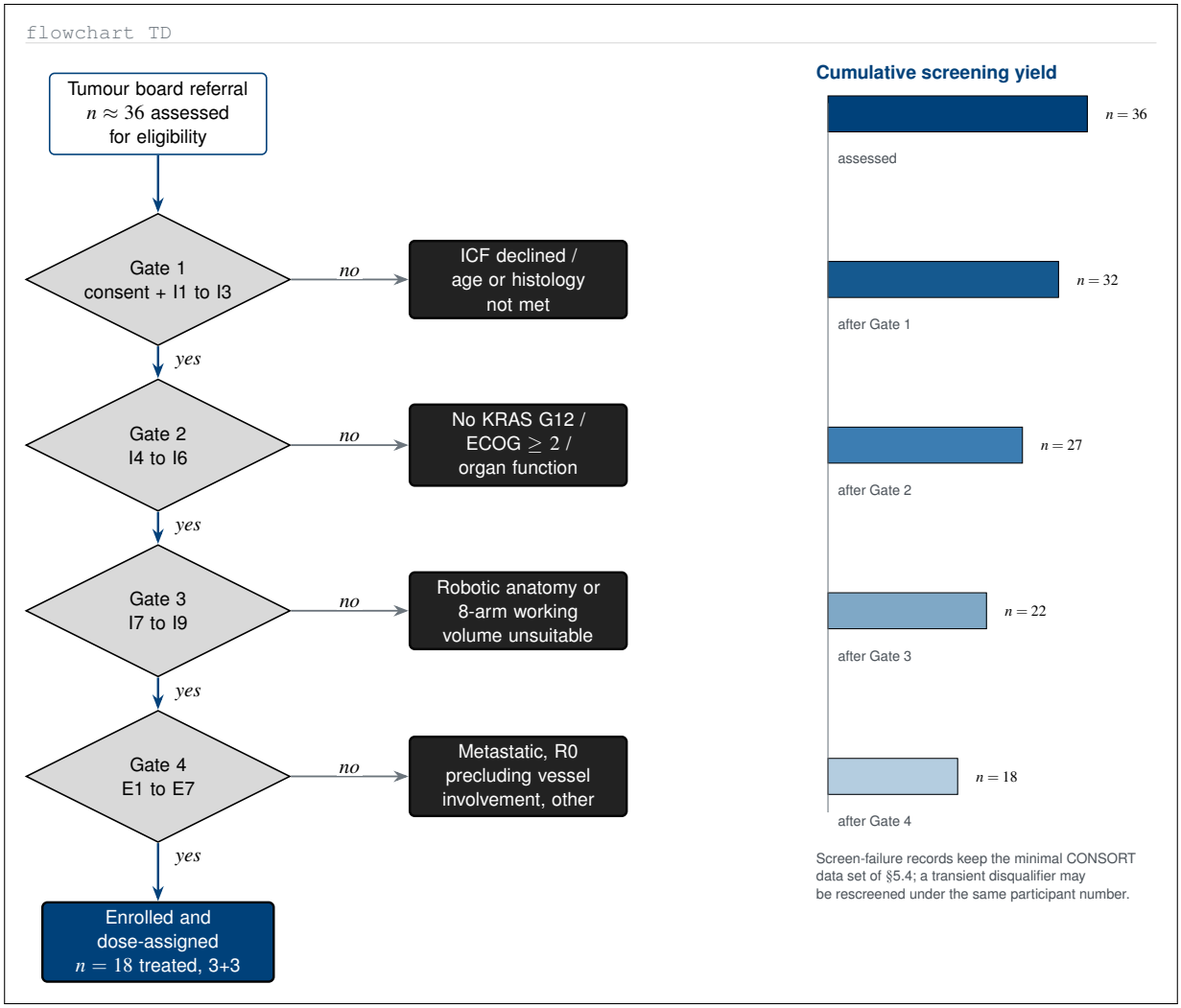


Figure 1. Mermaid-type flowchart of the four-gate eligibility funnel that turns approximately 36 assessed individuals into the 18 treated participants of the 3+3 design. The nine inclusion criteria of §5.1 and the seven exclusion criteria of §5.2 are grouped into four sequential gates, each with its own screen-failure branch, and the bar column at right shows the cumulative screening yield after every gate.

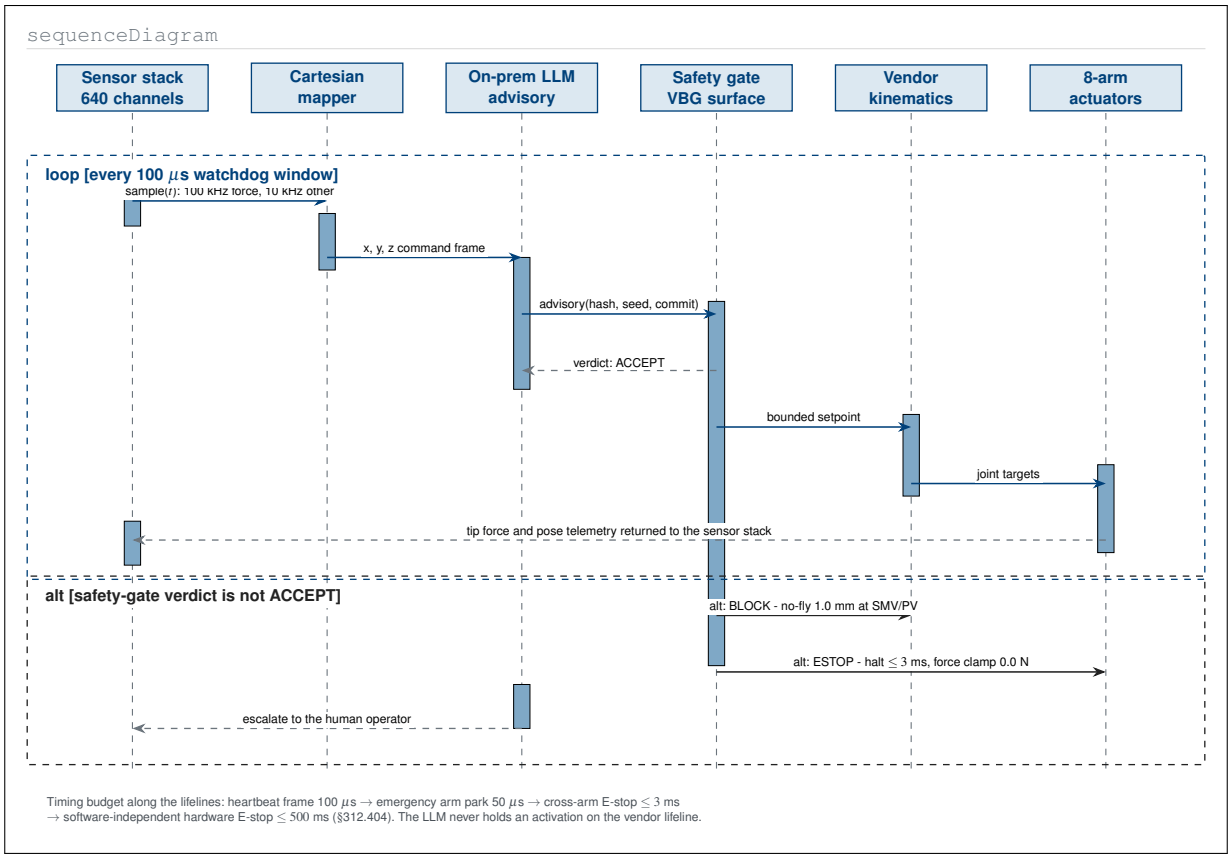


Figure 2. Mermaid-type sequence diagram of one intra-operative control cycle, with lifelines for the six participants in the loop and activation bars marking the window in which each holds control. The loop fragment repeats inside every 100 microsecond watchdog window; the alt fragment shows the two non-accept verdicts, a no-fly block at 1.0 mm and a cross-arm emergency stop, that interrupt it.

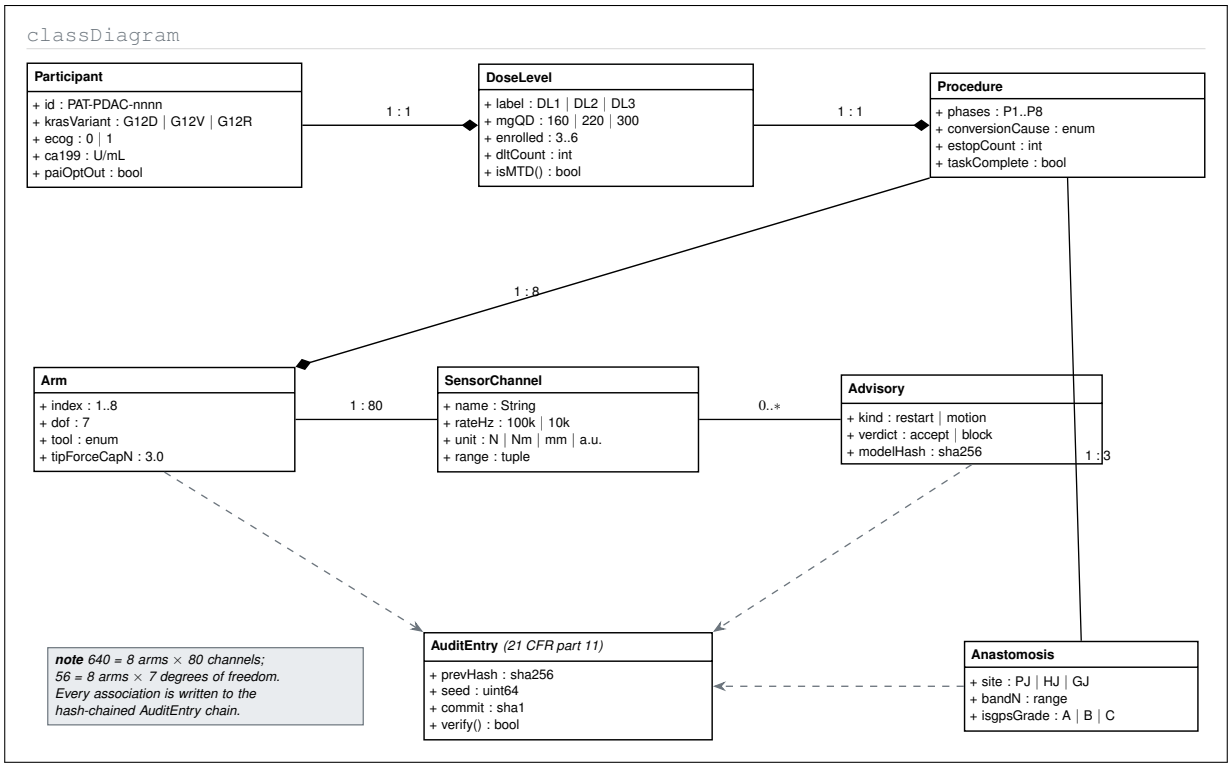


Figure 3. Mermaid-type class diagram of the protocol domain model, giving the trial the object schema its case report form implies. Composition edges carry the fixed cardinalities of the platform (one procedure to eight arms, one arm to eighty channels, one procedure to three anastomoses) and every association is bound by a dependency edge to the hash-chained part 11 audit entry.

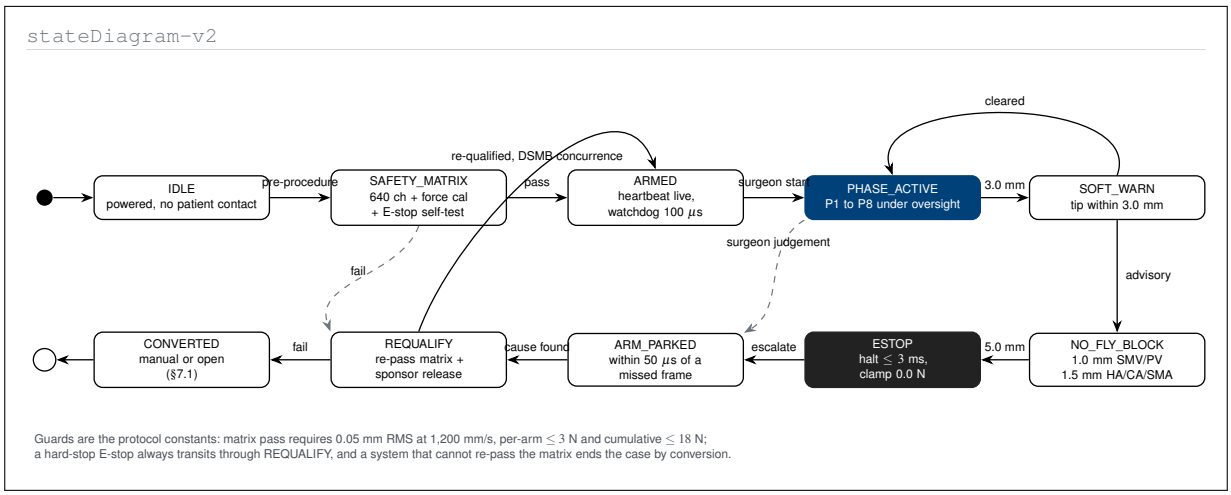


Figure 4. Mermaid-type state diagram of the investigational device across one case. Guard conditions are the protocol's own constants, and the machine makes explicit what the prose only implies: no path returns from a hard-stop emergency stop to an active phase without passing through the parked, re-qualification, and data-safety-monitoring-board concurrence states.

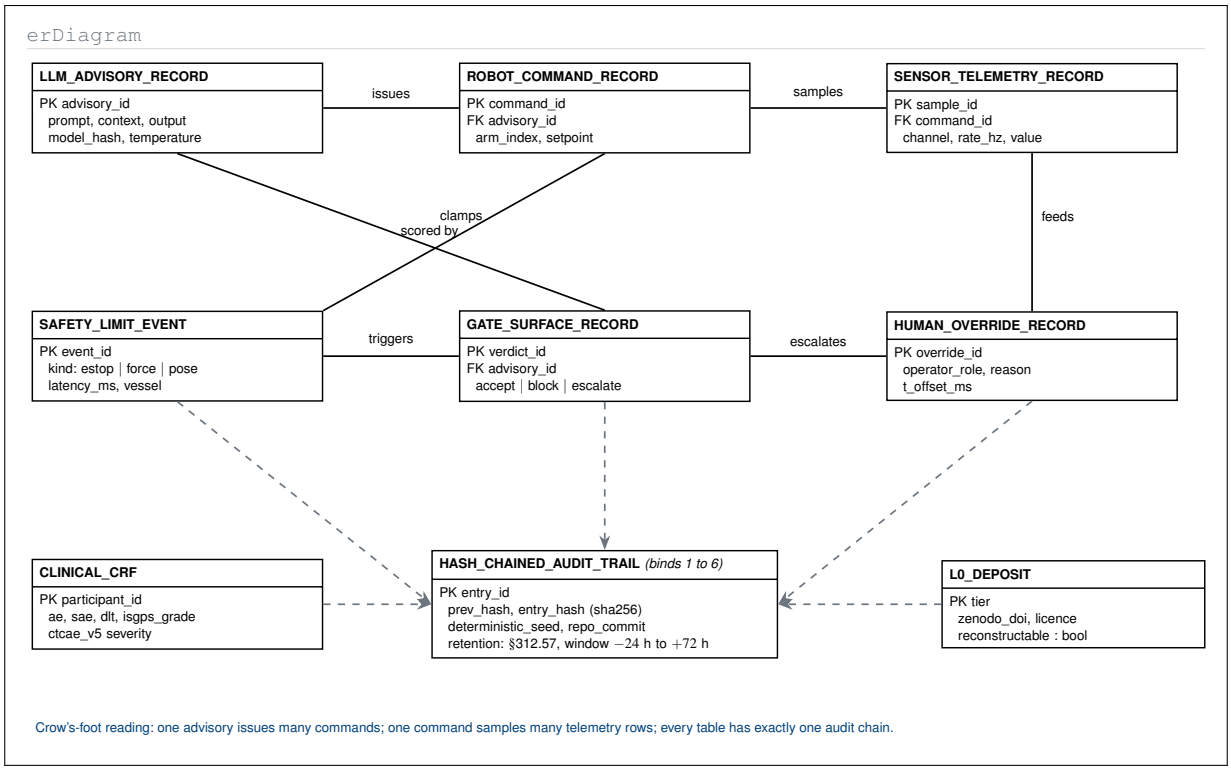


Figure 5. Mermaid-type entity-relationship diagram of the seven Physical AI record types of §10.9 plus the clinical case report form and the level-zero Zenodo deposit. The parent protocol lists these seven records as a numbered sentence; here they are given keys, foreign keys, and the single hash-chained audit trail that binds all of them to one deterministic seed and repository commit.

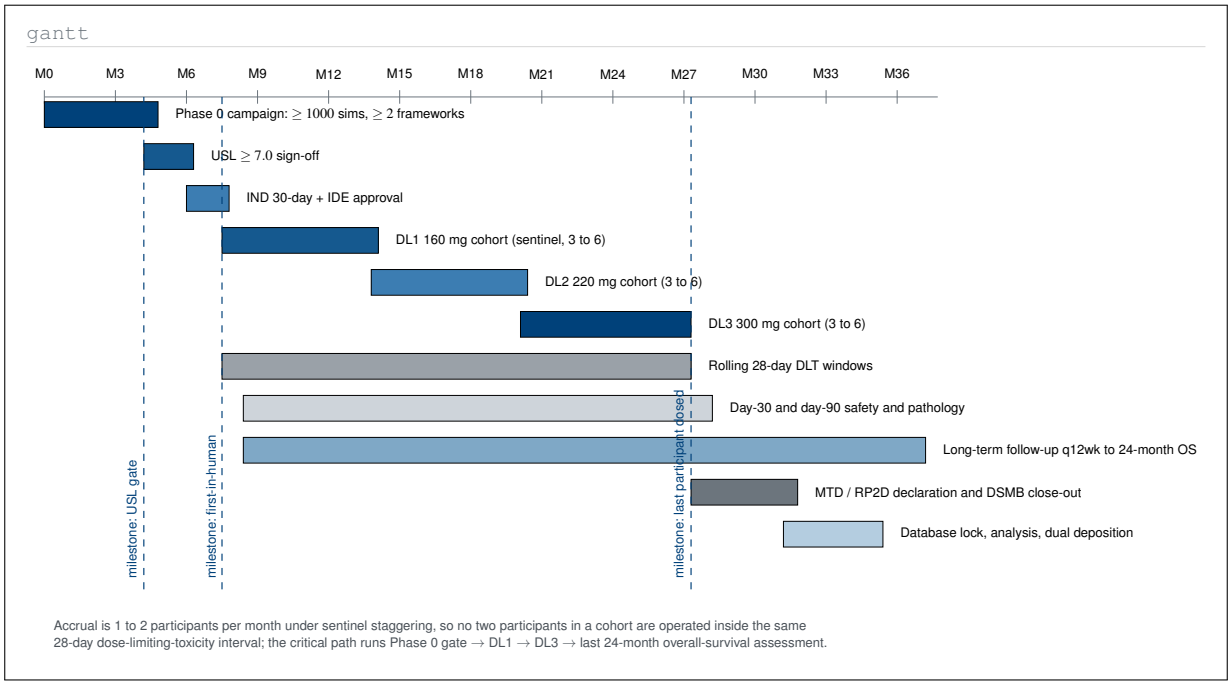


Figure 6. Mermaid-type Gantt chart of the study calendar the protocol specifies only as prose constraints: a Phase 0 validation campaign, three sequentially gated dose cohorts under sentinel staggering at one to two participants per month, rolling 28-day dose-limiting-toxicity windows, and a long-term follow-up tail that carries the trial to a 36-month operational horizon.

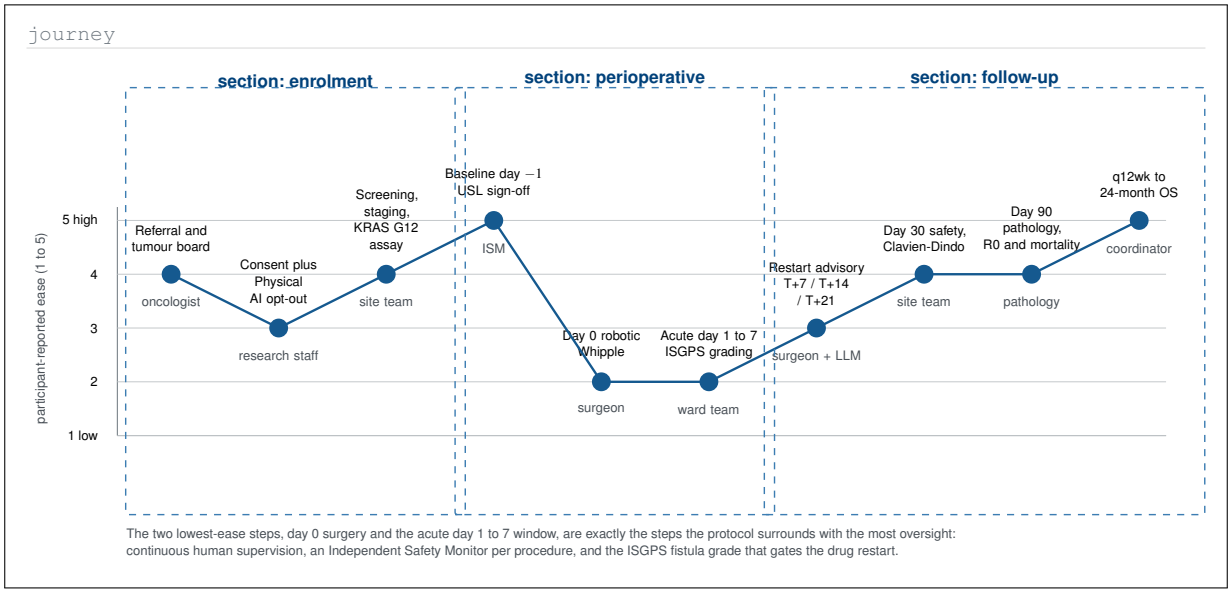


Figure 7. Mermaid-type user-journey chart of the enrolled participant across the three protocol sections, scoring each step for participant-reported ease and naming the actor who owns it. The trough at day 0 and the acute window aligns with where the protocol concentrates its oversight, which is the design argument the prose makes but never plots.

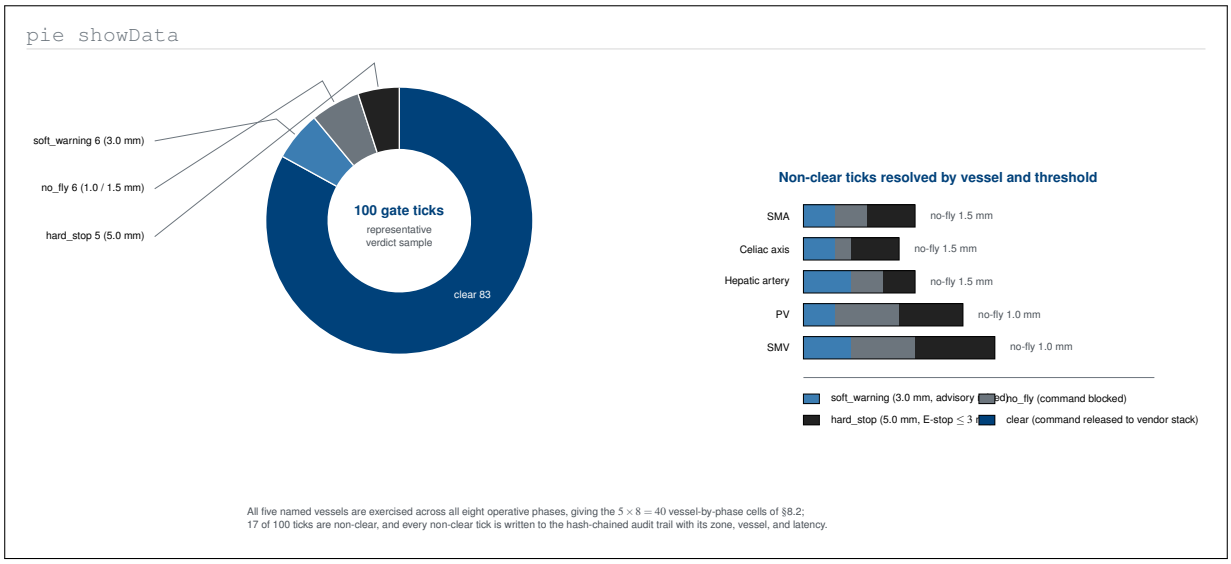


Figure 8. Mermaid-type pie chart of the representative 100-tick vascular-gate verdict sample, drawn as a donut so the 83 clear ticks do not visually swamp the 17 that are not. The companion stacked bars decompose those 17 non-clear ticks by named vessel and by which of the three concentric thresholds produced them, a split the parent protocol reports only as four totals.

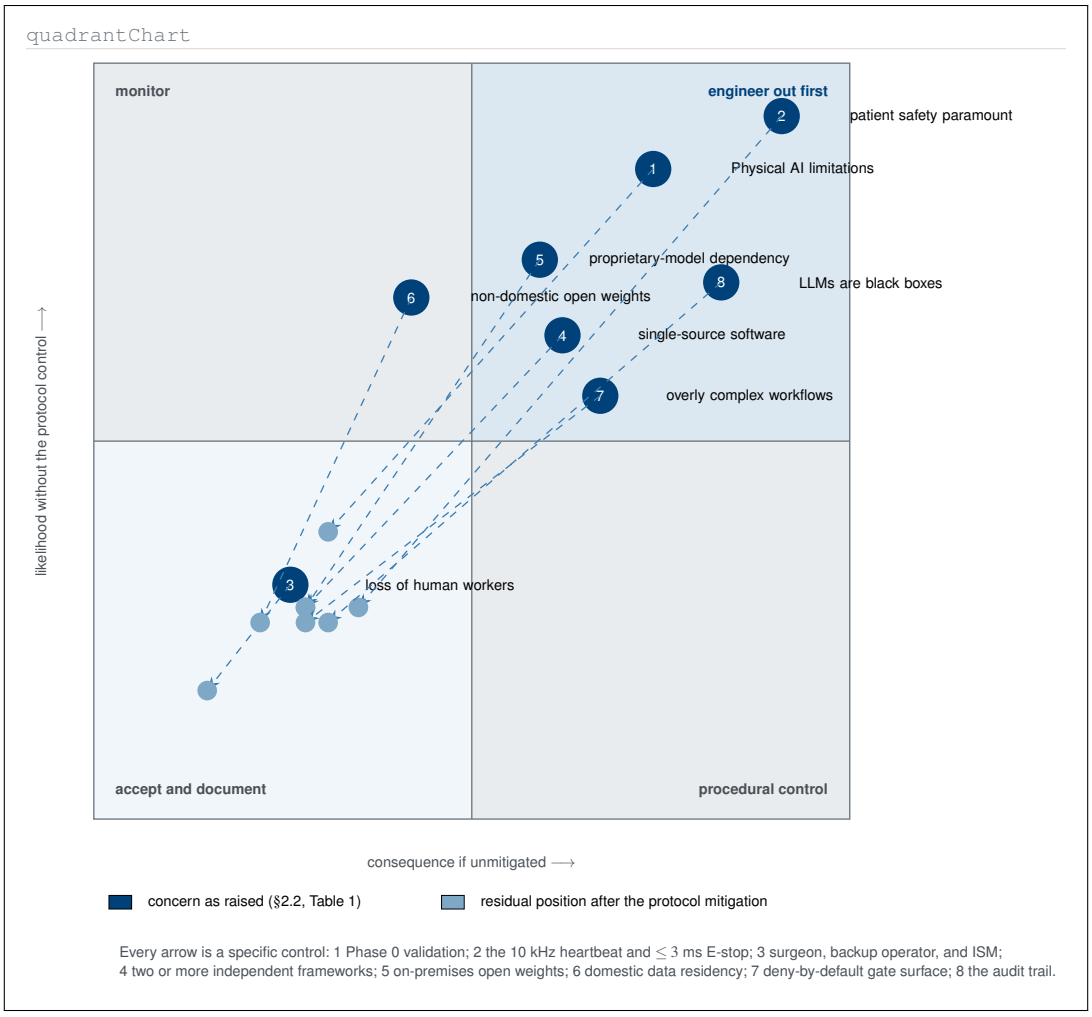


Figure 9. Mermaid-type quadrant chart placing the eight recurring Physical AI concerns of Table 1 on likelihood against consequence, then drawing each concern's displacement once its named protocol mitigation is applied. The chart converts a two-column mapping table into a residual-risk statement, which is the form a review board actually needs.

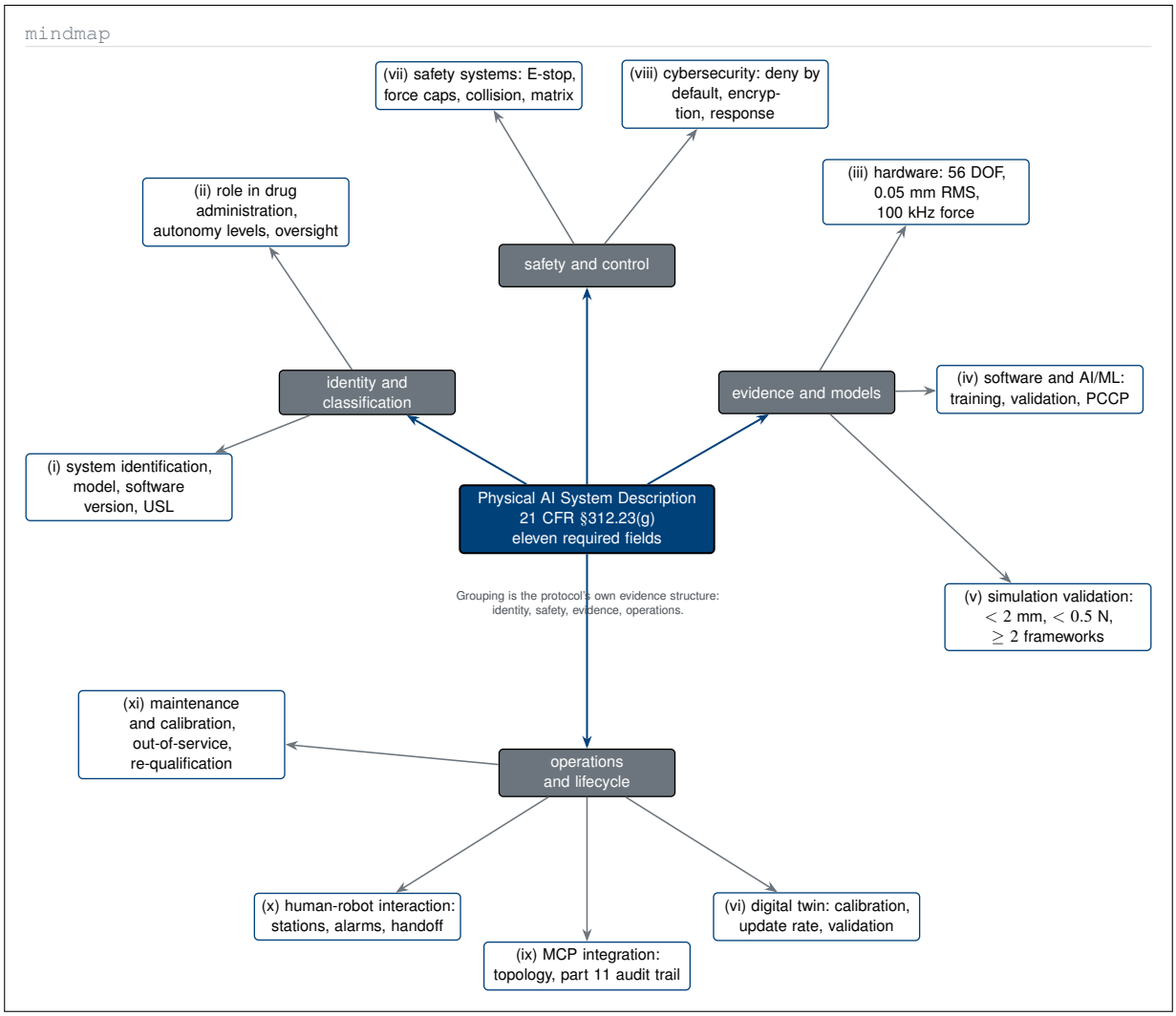


Figure 10. Mermaid-type mindmap of the eleven Physical AI System Description fields required by 21 CFR §312.23(g), organised into the four evidence families the submission actually assembles rather than the flat roman-numeral list of the parent text. The radial form makes visible that safety and evidence fields together account for five of the eleven.

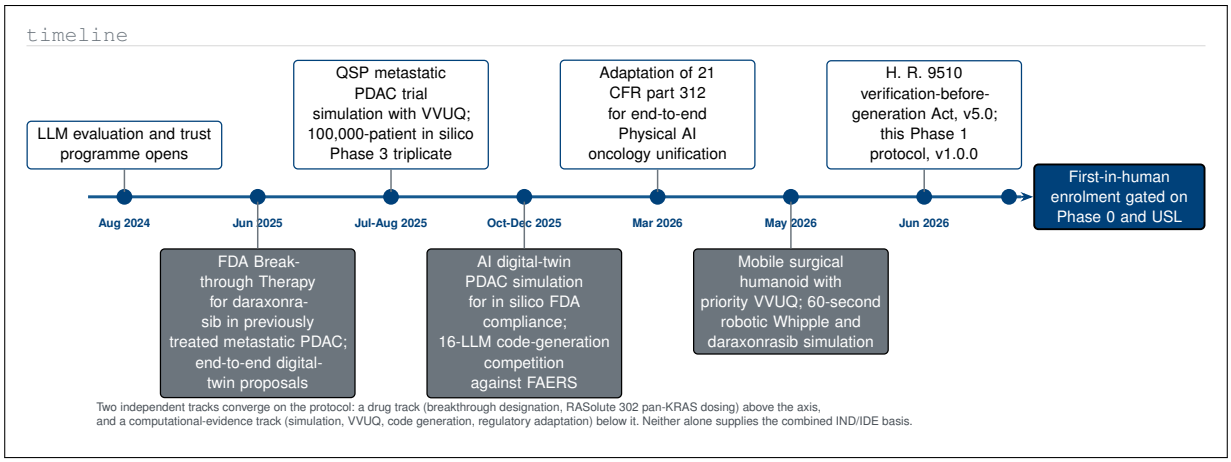


Figure 11. Mermaid-type timeline of the evidence chronology that precedes first-in-human enrolment, separating the drug track above the axis from the computational and regulatory evidence track below it. The parent introduction cites these works as a list; drawn on a shared axis they show two independent lines of evidence converging on a single 2026 protocol.

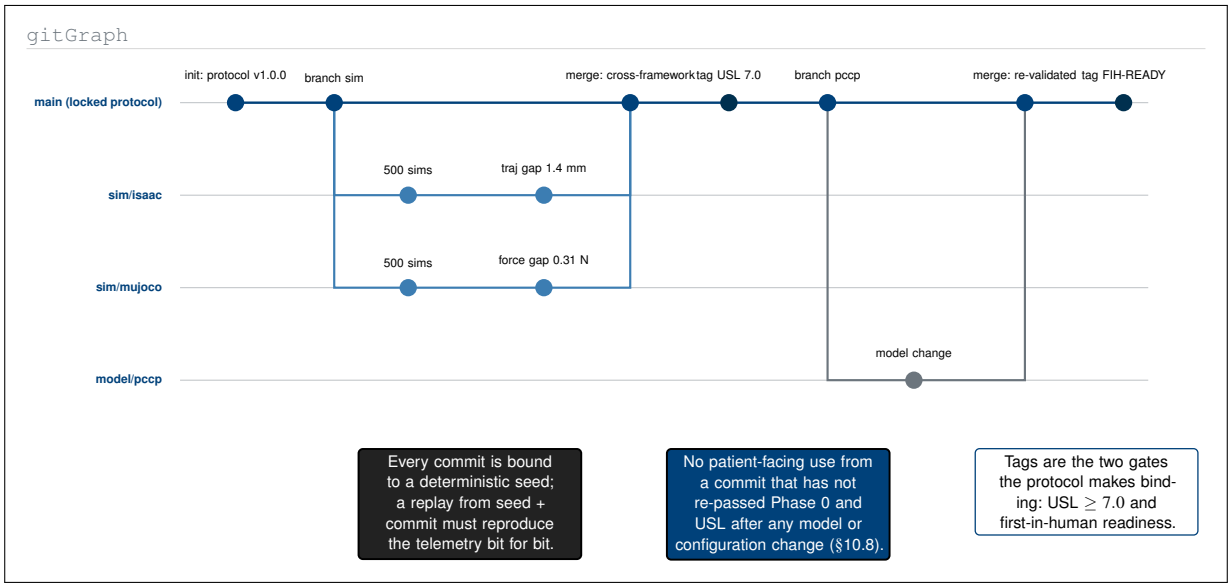


Figure 12. Mermaid-type gitGraph of the deterministic build lineage that makes the device re-traceable. The two simulation branches are the two independent frameworks the Phase 0 gate requires, their merge is the cross-framework consistency finding, and the predetermined-change-control branch shows why no model change can reach a participant without a re-validation merge and a fresh tag.

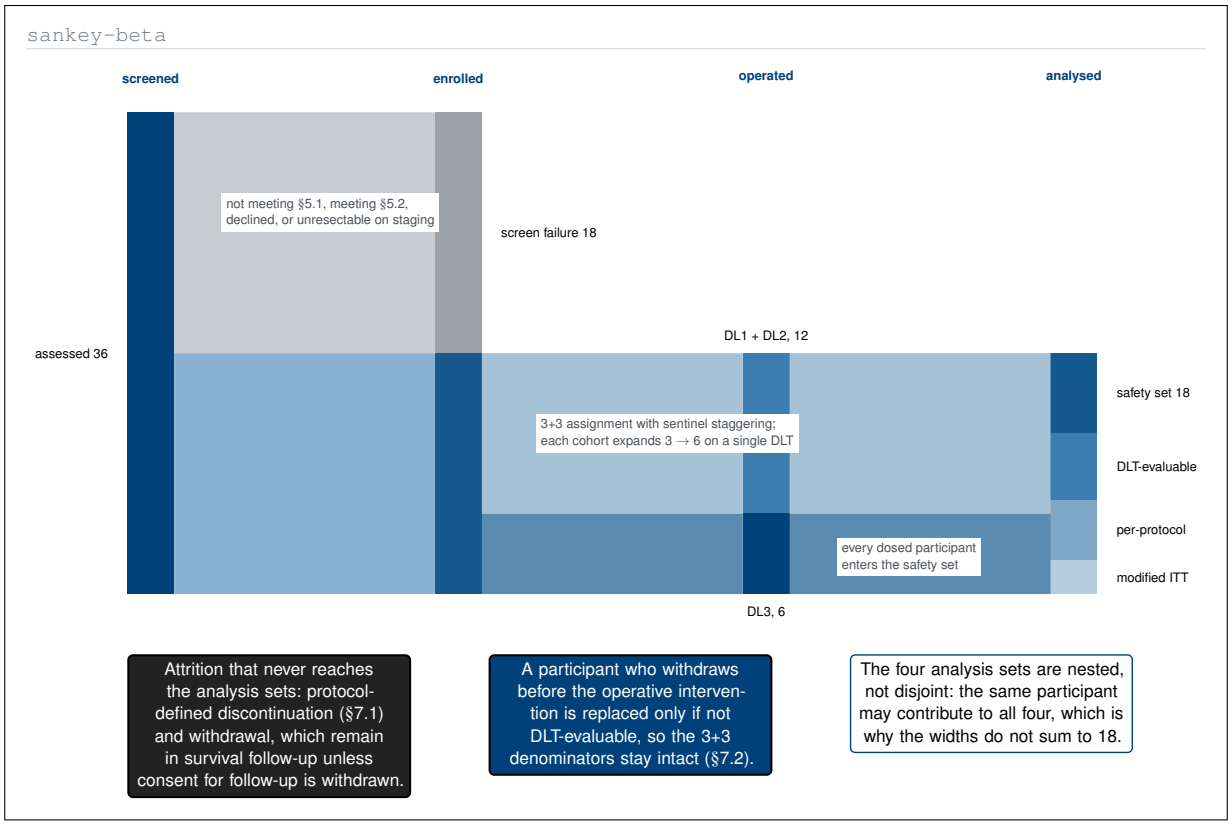


Figure 13. Mermaid-type Sankey diagram of participant flow by volume rather than by box count, from 36 assessed through the 3+3 dose assignment to the four analysis populations. Ribbon width is proportional to participants, which makes the one-in-two screen-failure fraction and the nesting of the four analysis sets immediately legible.

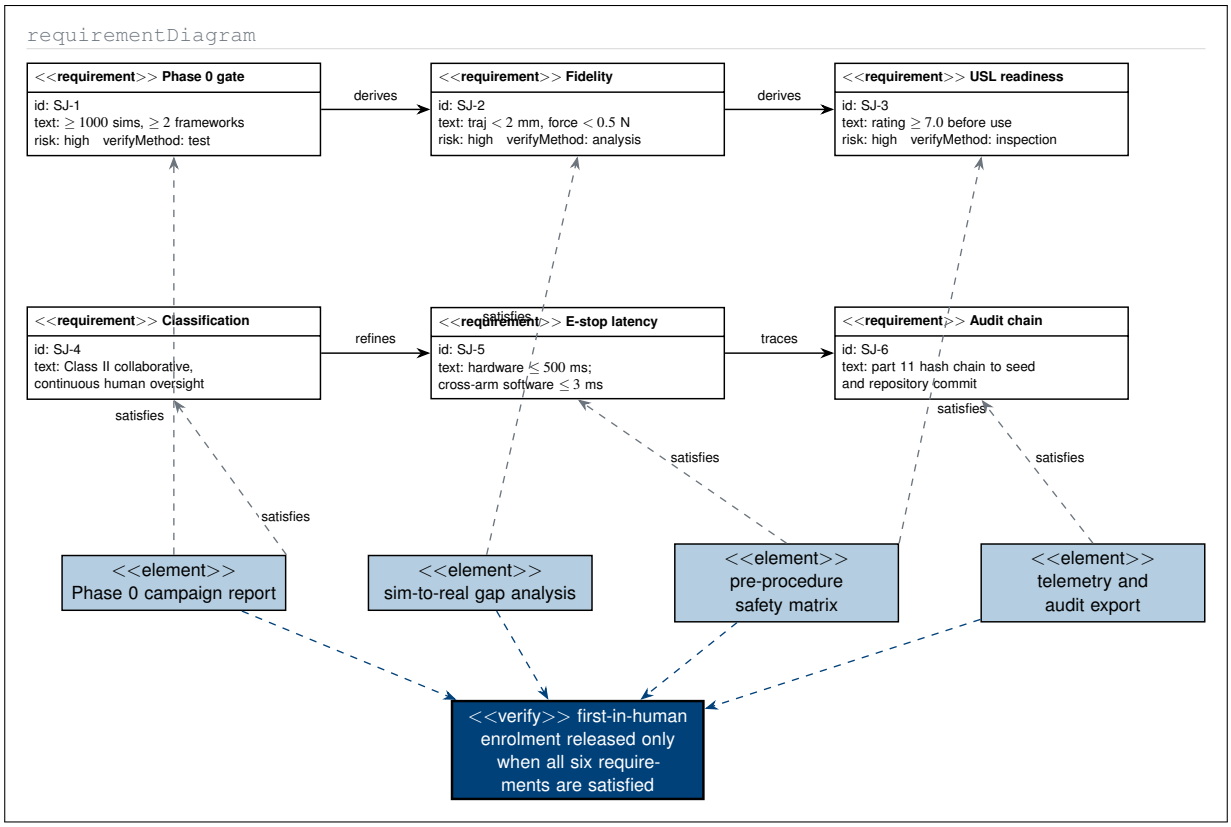


Figure 14. Mermaid-type requirement diagram tracing the six Subpart J preconditions of §0.3 to the submission elements that satisfy them and to the verification method each carries. The parent text states these obligations as a paragraph; expressed as a requirement graph they become an auditable trace in which no enrolment release is possible with an unsatisfied node.

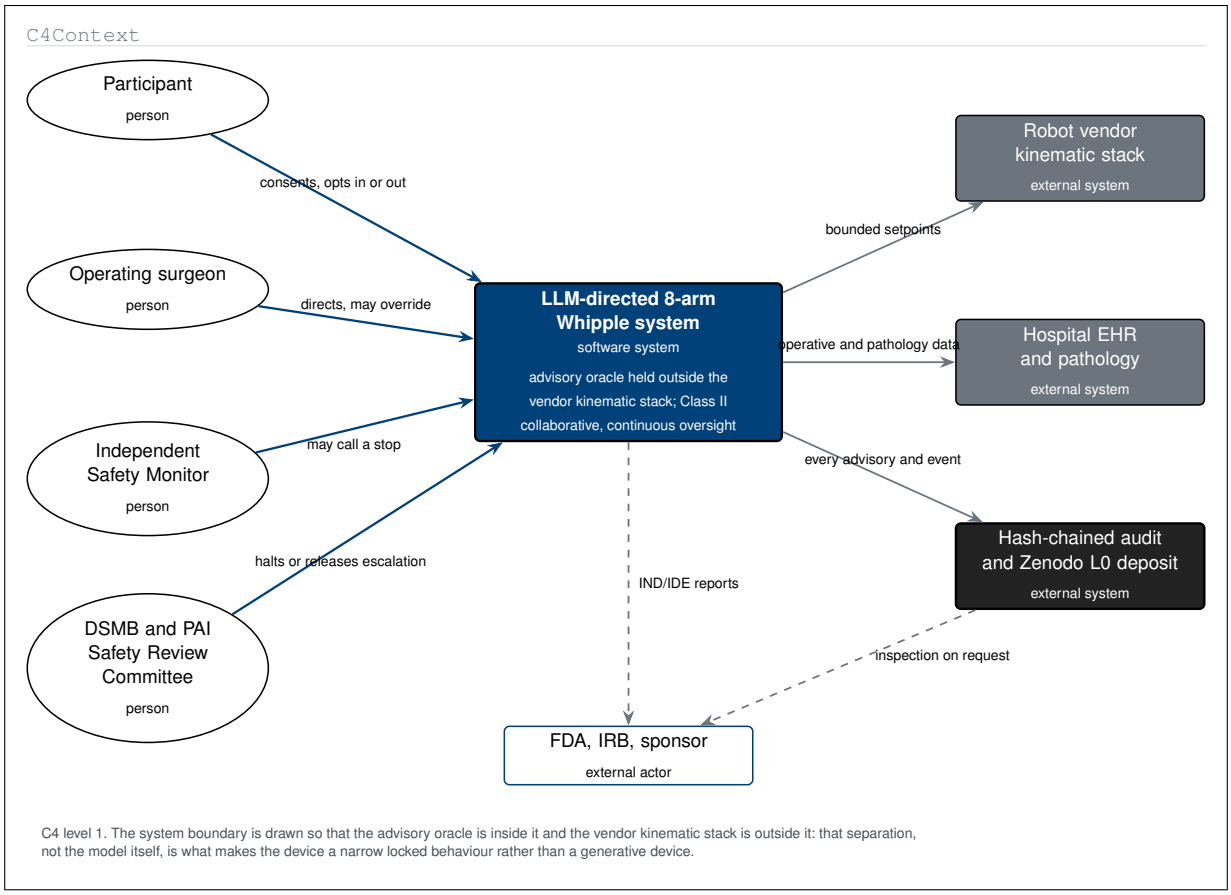


Figure 15. Mermaid-type C4 context diagram placing the investigational system inside a single boundary with its four human actors and its three external systems. The boundary is drawn deliberately so that the advisory oracle sits inside and the robot vendor kinematic stack sits outside, which is the architectural claim on which the narrow, locked device characterisation rests.

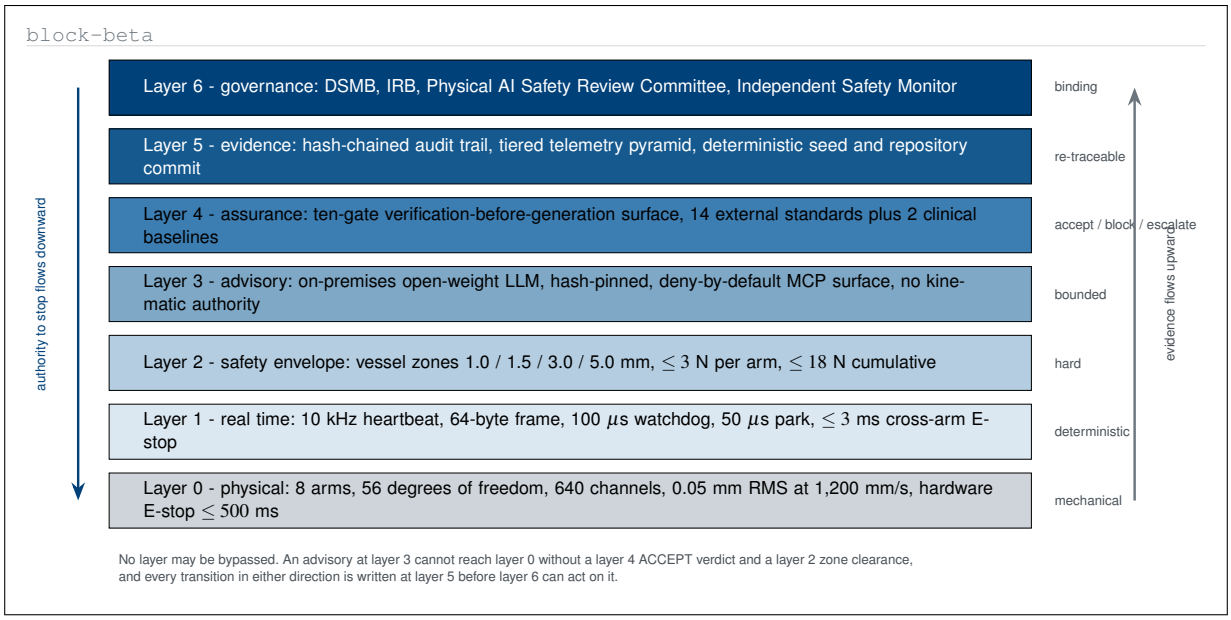


Figure 16. Mermaid-type block diagram of the on-premises stack as seven strictly ordered layers, from the mechanical platform to the governance bodies. Two opposing arrows carry the invariant the protocol relies on but never draws: stop authority propagates downward while evidence propagates upward, and no layer may be bypassed in either direction.

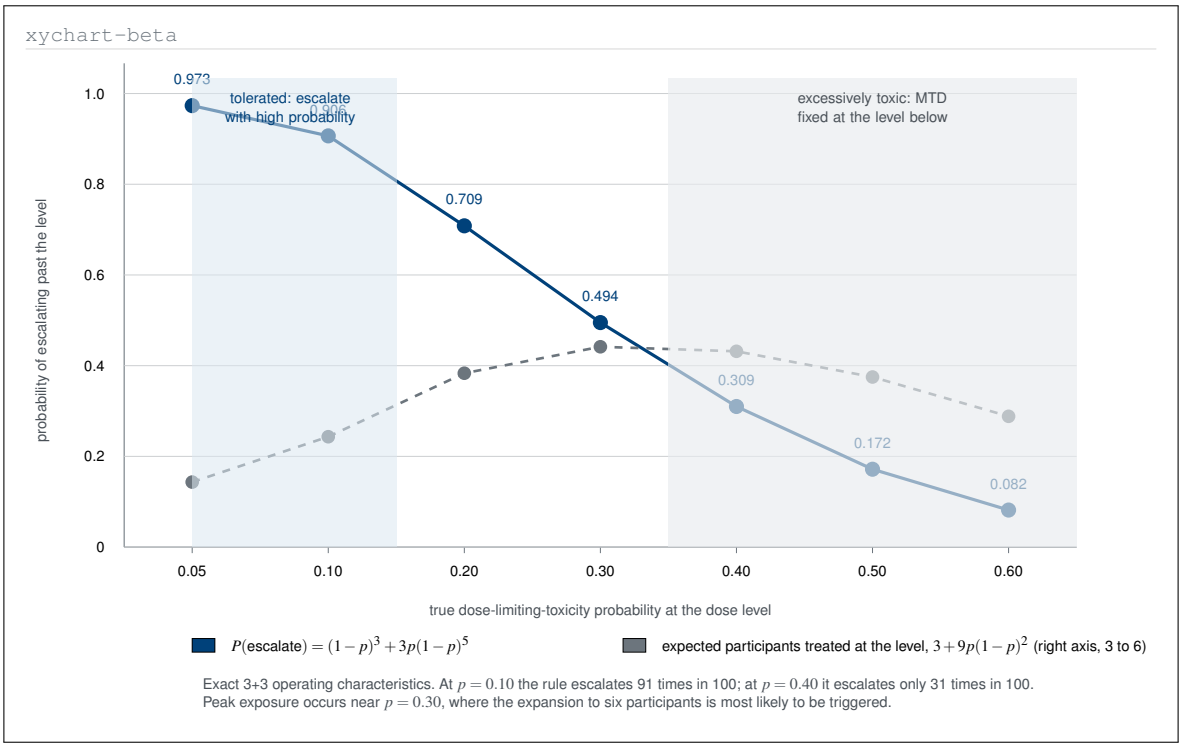


Figure 17. Mermaid-type xy chart of the exact operating characteristics of the 3+3 rule, computed from the escalation probability and plotted against the true dose-limiting-toxicity rate. The parent statistics section asserts that a ten-percent dose is very likely escalated and a forty-percent dose very likely identified as toxic; this figure supplies the numbers behind that claim.

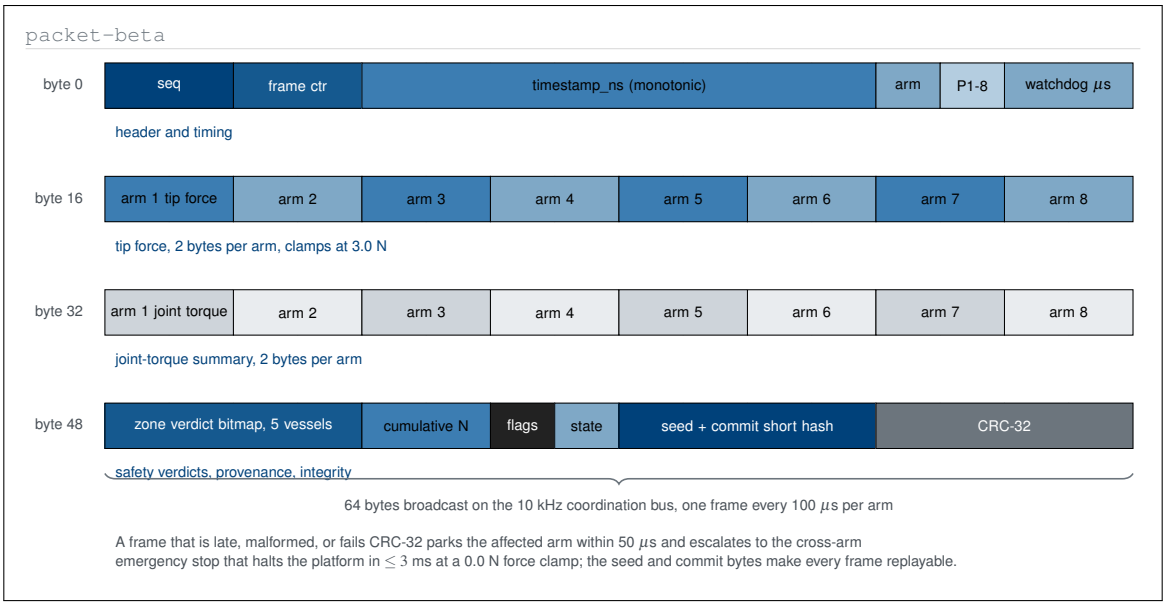


Figure 18. Mermaid-type packet diagram giving the 10 kHz heartbeat its byte-level layout: sixty-four bytes carrying header and timing, per-arm tip force and joint torque, the five-vessel zone verdict bitmap, the deterministic seed and repository commit, and a CRC-32. The parent protocol names the frame size and deadline; this figure shows what has to fit inside them.

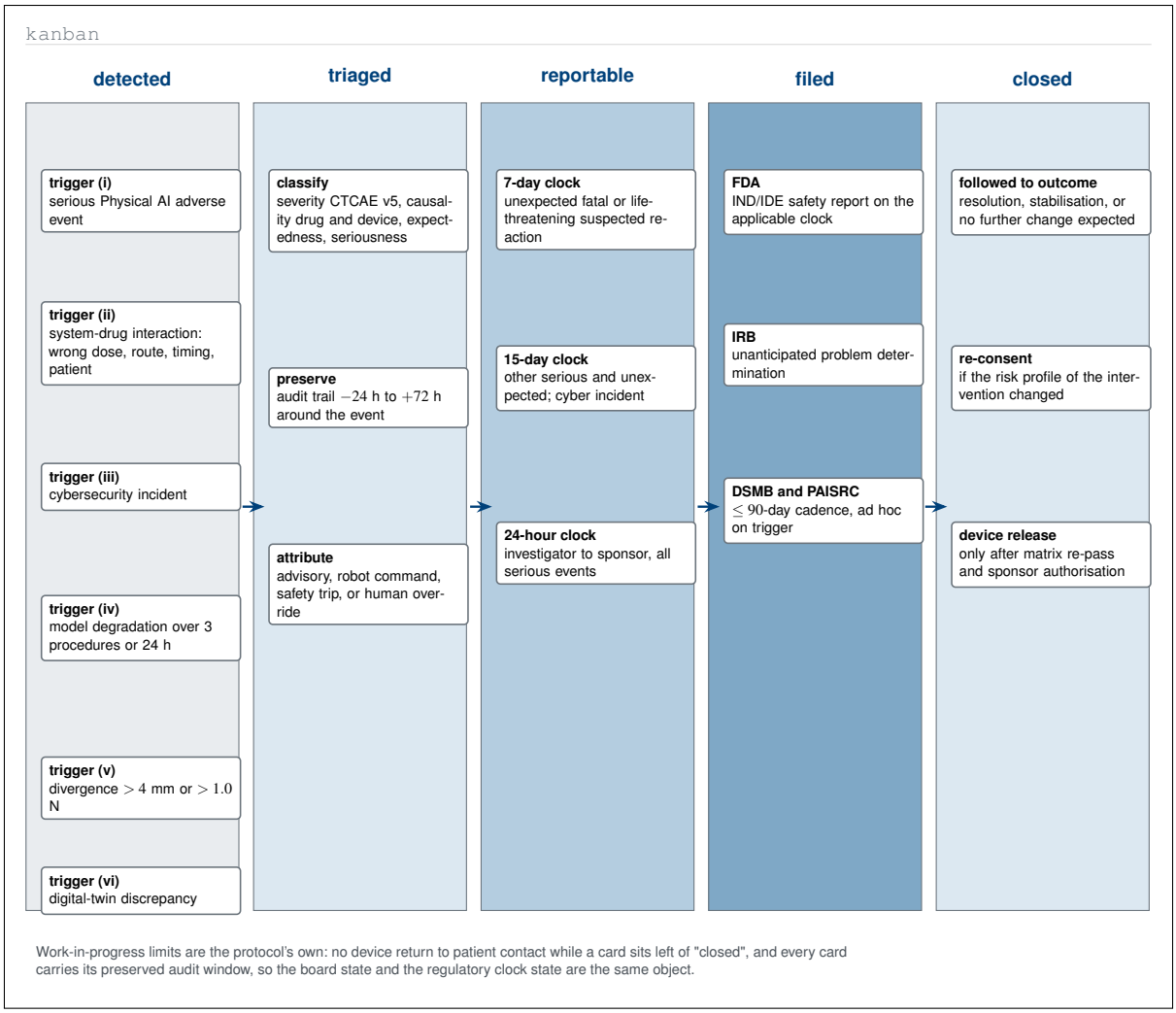


Figure 19. Mermaid-type kanban board of Physical AI safety-event triage, with the six reporting triggers of §8.3.6 as the backlog column and the regulatory clocks as work-in-progress states. Rendering the reporting obligation as a board makes the binding constraint explicit: the device does not return to patient contact while any card is left of the closed column.

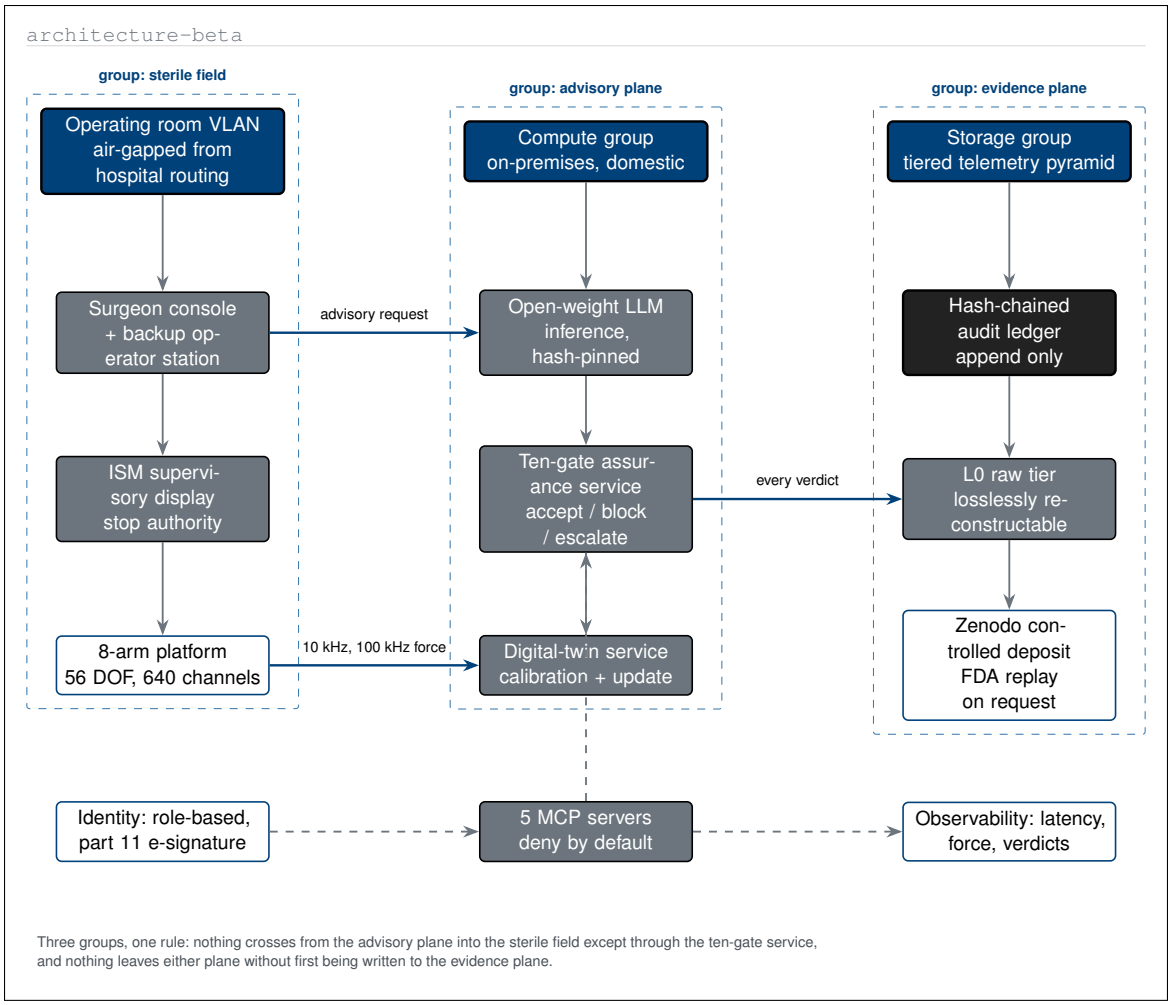


Figure 20. Mermaid-type architecture diagram of the on-premises deployment as three grouped planes: sterile field, advisory plane, and evidence plane. Grouping the components this way states the deployment invariant directly, namely that advisory traffic reaches the sterile field only through the ten-gate service and nothing leaves either plane unwritten to the evidence plane.

2 Excalidraw-type Diagrams

Excalidraw is a hand-drawn-style virtual whiteboard, and its strength is the opposite of Mermaid's: it is at its best where a diagram should look provisional [13]. A deliberately imperfect stroke signals that the artefact is an argument being worked out rather than a specification being issued, which is exactly the register for spatial layouts, sticky-note clustering, storyboards, decision matrices, wireframes, and napkin arithmetic. The twenty figures below use that register on the parts of the protocol that are usually settled in a room rather than in a document: the operating-room floor, the concern-to-mitigation workshop, the failure storyboard, the console alarm surface, and the interval arithmetic behind the sample size. They are referred to as **excalidraw-type** figures.

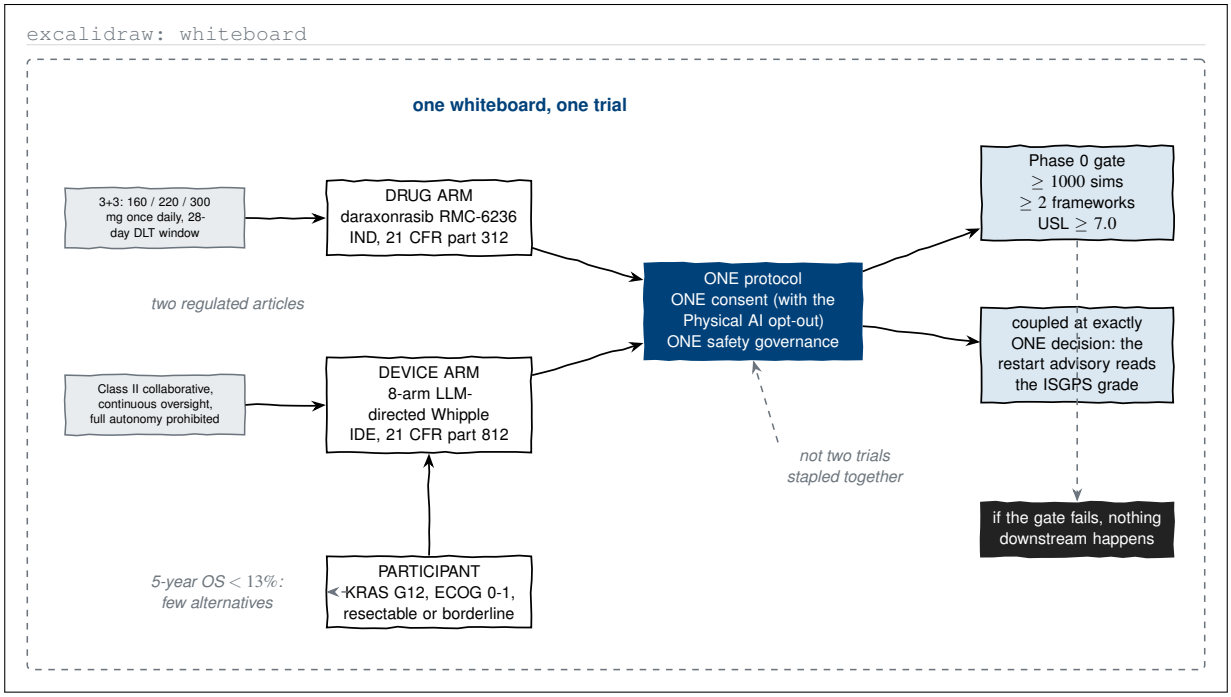


Figure 1. Excalidraw-type whiteboard sketch of the whole study on a single surface: two regulated articles, one protocol, one consent, one governance structure, and a single coupling point between the arms. Sketching it this way exposes the claim the prose spends a section establishing, that the combined investigation is not two trials stapled together.

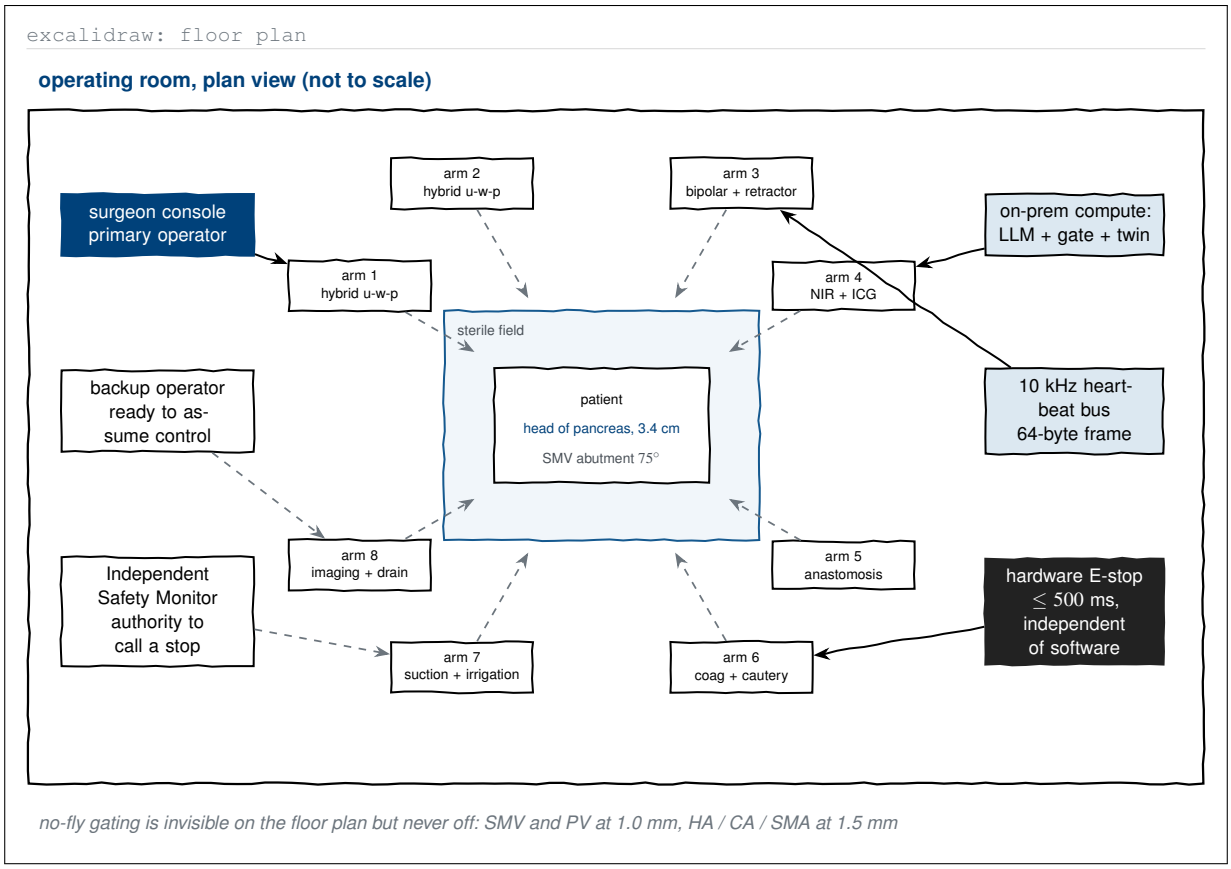


Figure 2. Excalidraw-type operating-room floor plan placing the eight arms, their tool assignments, the three human stations, and the on-premises compute rack in physical relation to the patient. The parent protocol describes arm roles in a table; drawn as a room it becomes clear that the stop authorities sit outside the sterile field and reach the platform by three independent routes.

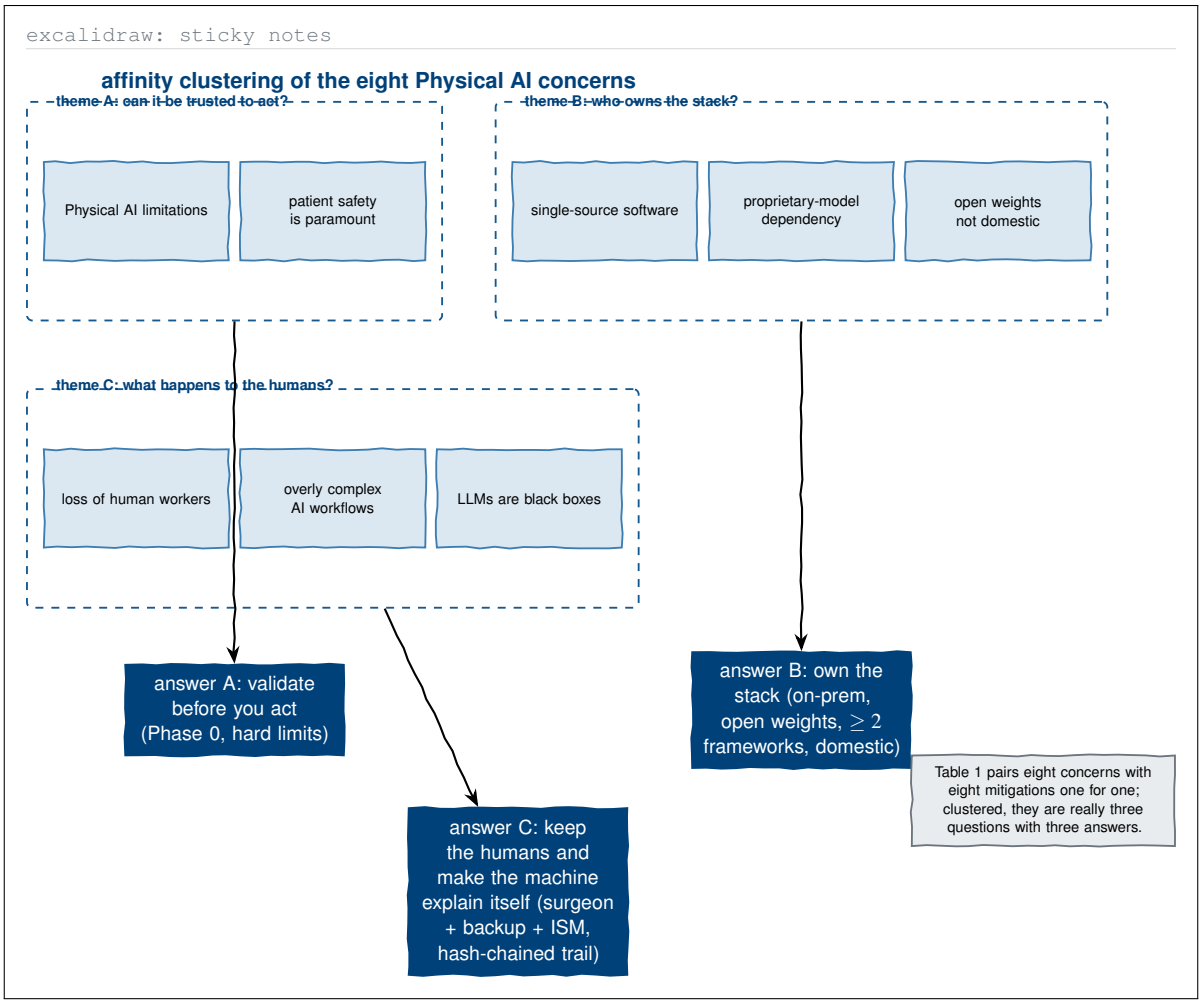


Figure 3. Excalidraw-type sticky-note affinity board that re-groups the eight Physical AI concerns of Table 1 into the three questions they actually represent. The one-to-one concern-to-mitigation mapping of the parent protocol is correct but flat; clustering shows that the eight mitigations collapse into three design commitments.

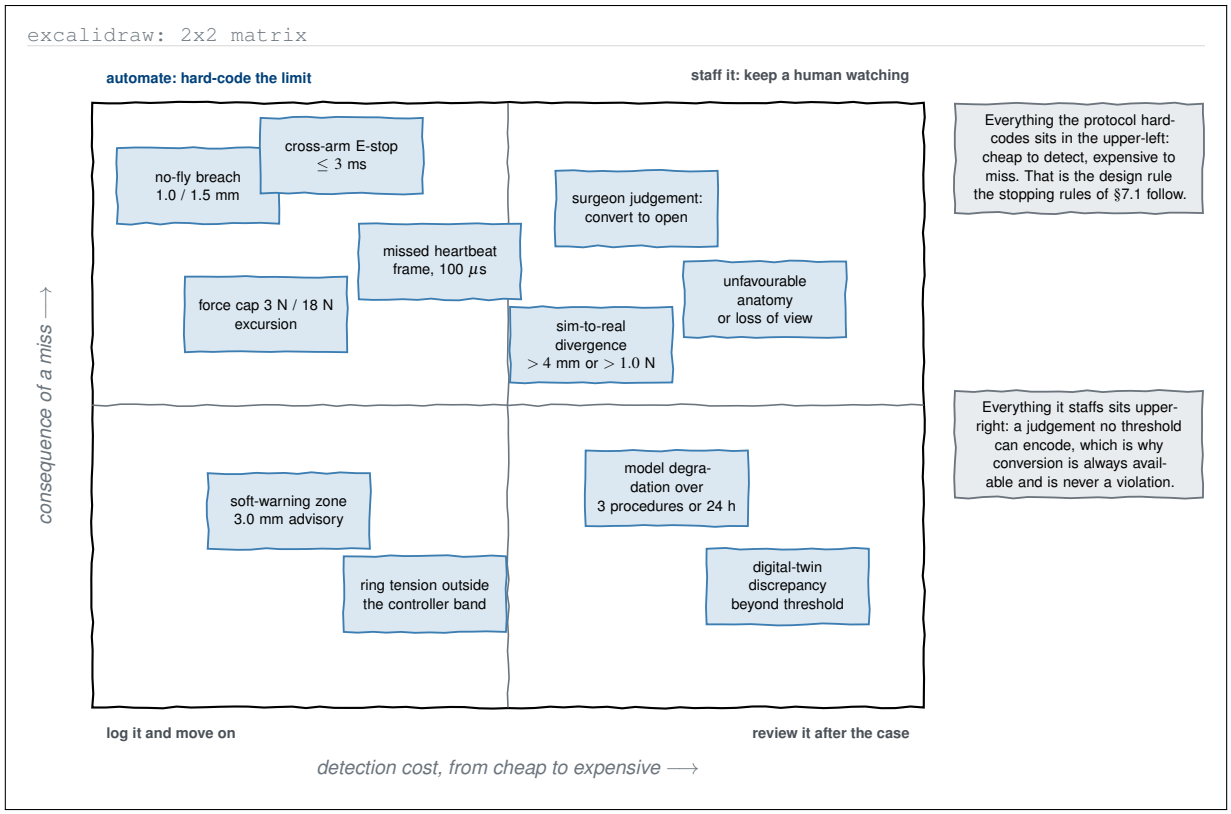


Figure 4. Excalidraw-type two-by-two matrix sorting every stopping and pause trigger in the protocol by detection cost against the consequence of missing it. The sketch recovers the unstated design rule: cheap-to-detect, expensive-to-miss events are hard-coded limits, while expensive-to-detect judgements are staffed with a human who may always convert.

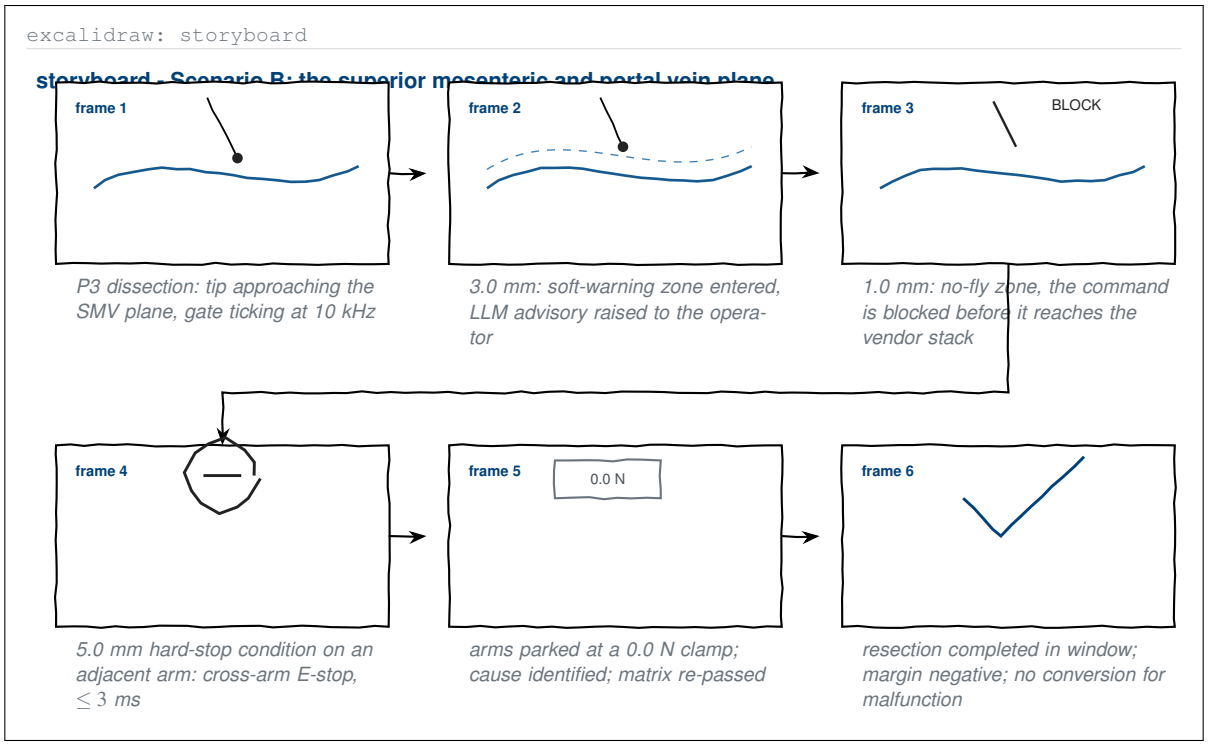


Figure 5. Excalidraw-type six-frame storyboard of Scenario B, the intra-operative vessel injury the parent introduction argues counterfactually. Each frame carries the threshold that fires in it, so the abstract claim that a no-fly gate with a three millisecond emergency stop averts the fistula-sepsis cascade becomes a sequence with named distances and latencies.

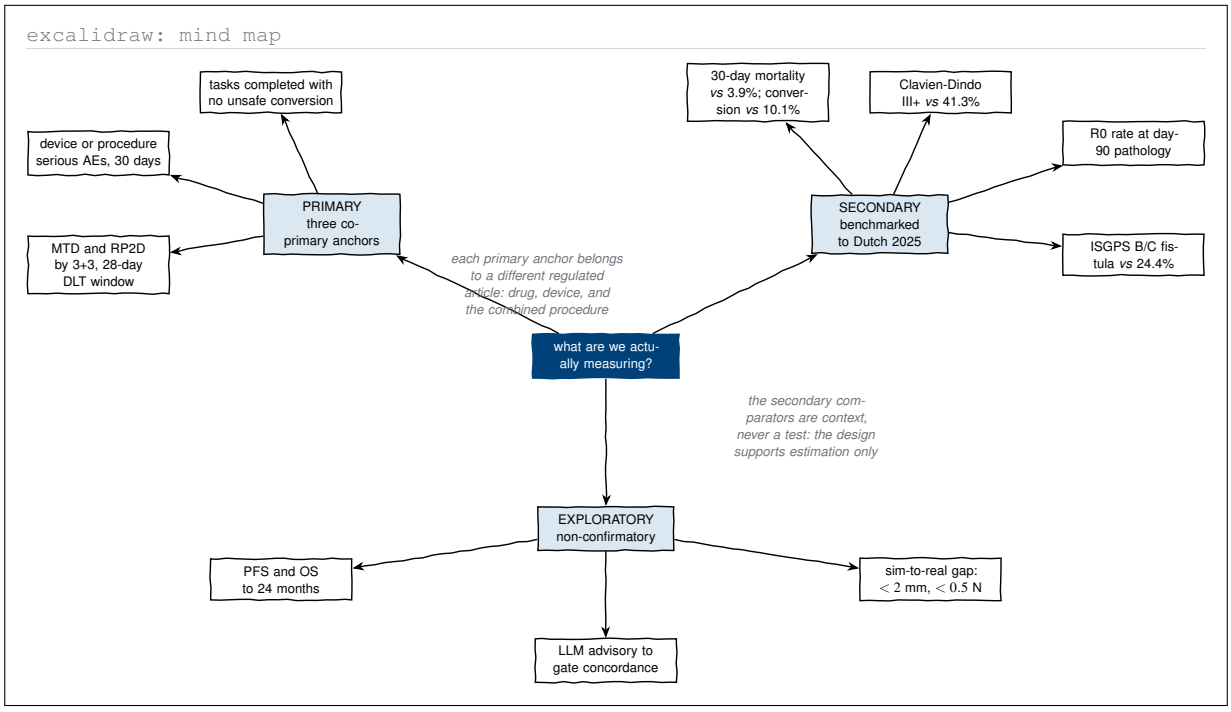


Figure 6. Excalidraw-type mind map of what the study measures, branching from a single question rather than from a table row. Sketching it makes the structural point of §3 visible at a glance: each of the three co-primary anchors belongs to a different regulated article, and every secondary comparator is context rather than a hypothesis test.

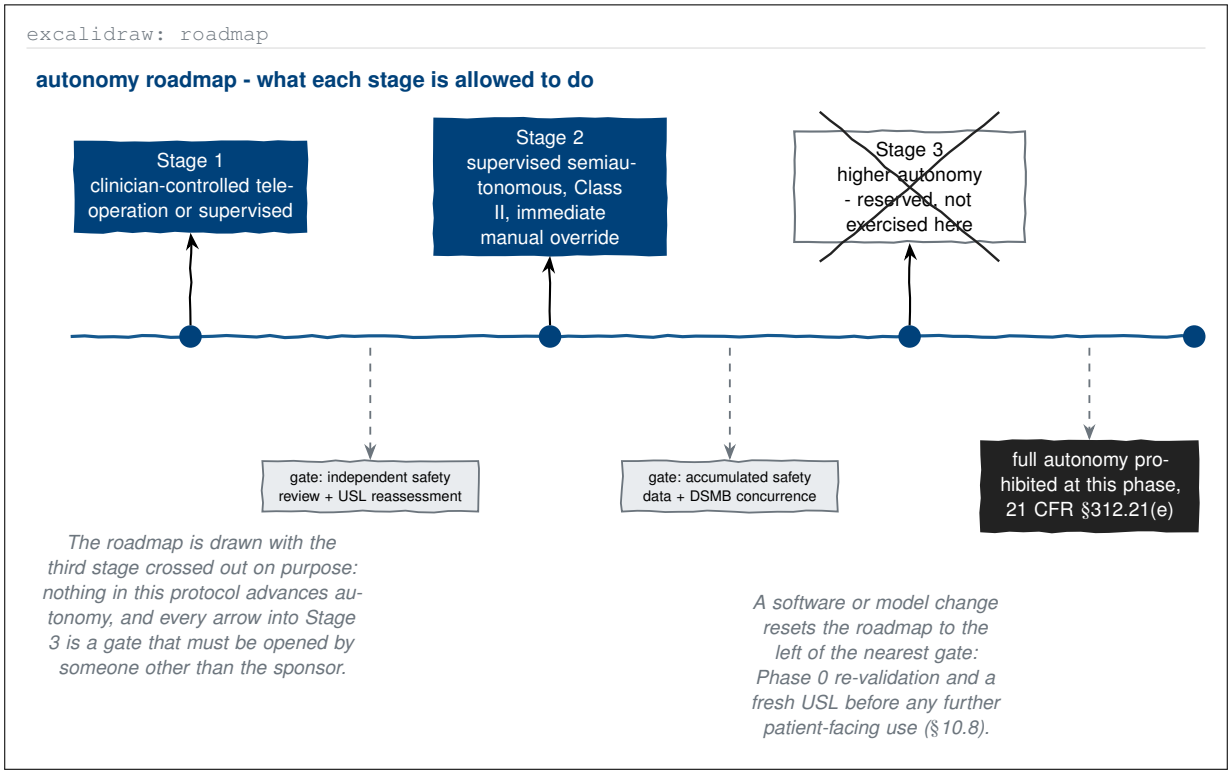


Figure 7. Excalidraw-type roadmap of the phase-graduated staged-autonomy model, drawn with the third stage deliberately crossed out. The sketch register carries the protocol's own caution better than a clean diagram would: Stage 3 exists on the roadmap but is not exercised, and every gate into it is opened by a body other than the sponsor.

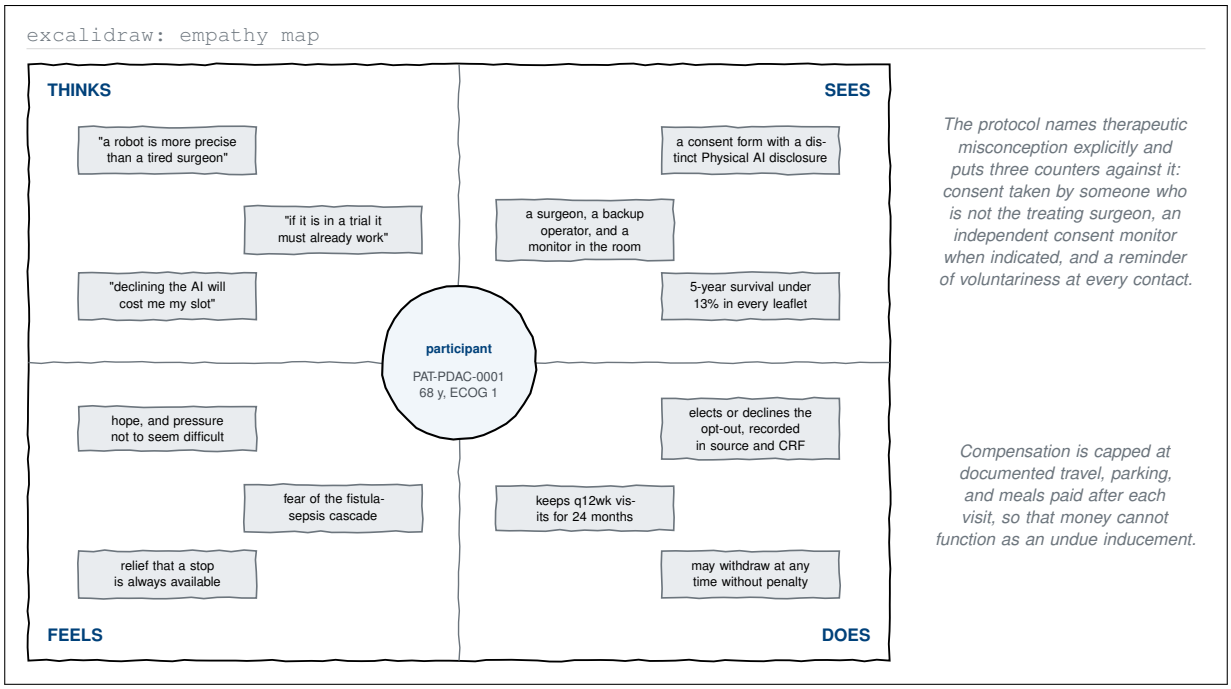


Figure 8. Excalidraw-type empathy map of the enrolled participant, built around the synthetic reference exemplar. The three beliefs in the upper-left quadrant are the therapeutic misconception the protocol names in §10.1, and the right-hand quadrants show the specific consent-process counters the protocol places against each of them.

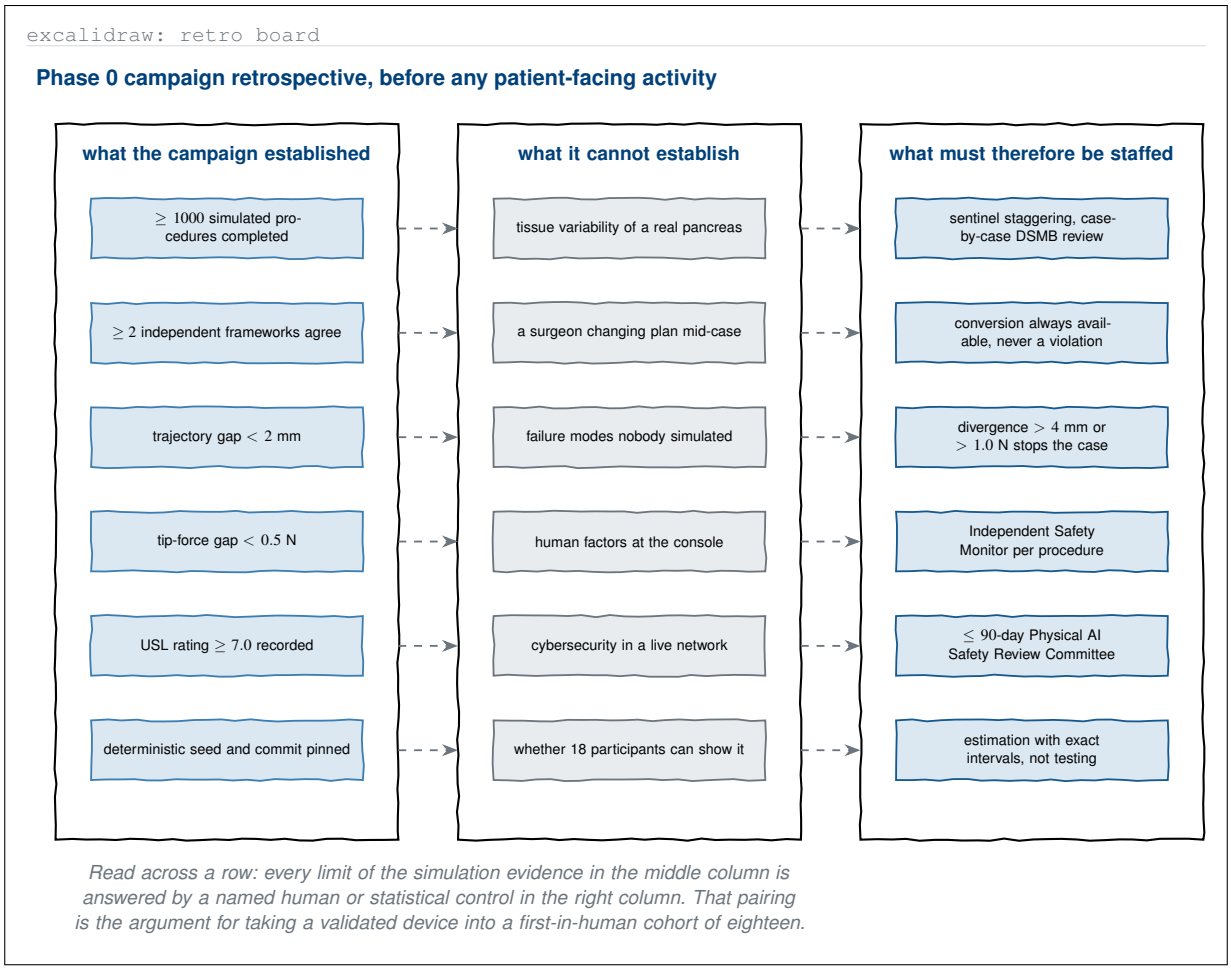


Figure 9. Excalidraw-type retrospective board on the Phase 0 validation campaign, with three columns that read across rather than down. Each limit of the simulation evidence is placed on the same row as the human or statistical control that answers it, which is the argument the protocol makes for entering a first-in-human cohort on simulation evidence.

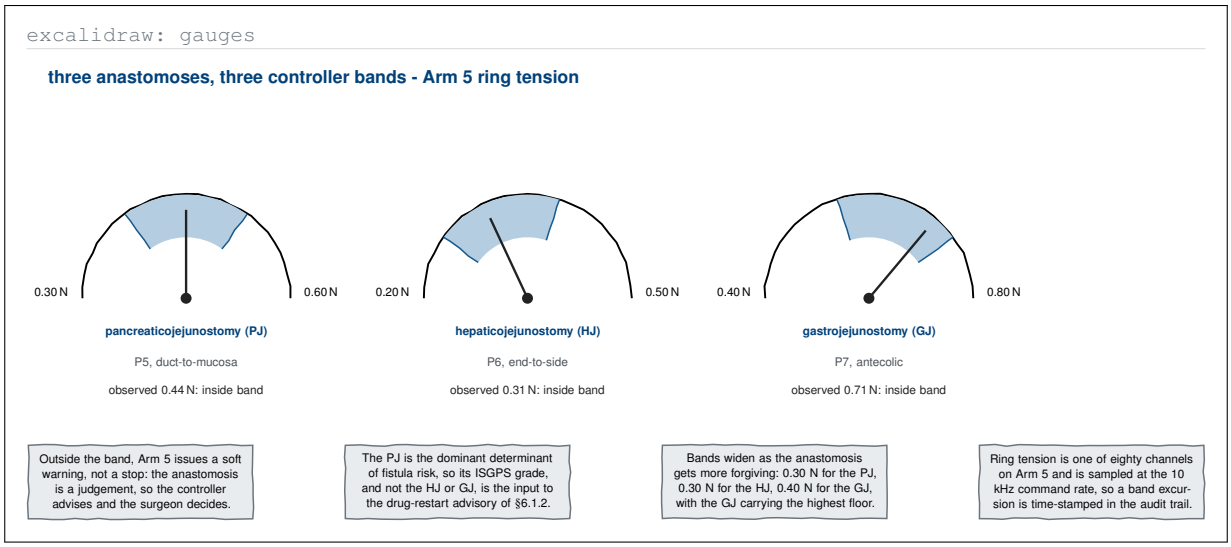


Figure 10. Excalidraw-type gauge sketch of the three reconstructive anastomoses and their Arm 5 ring-tension bands, drawn as dials with a needle inside each band. The parent protocol gives the three bands as numbers in a sentence; as dials they show that the bands widen with the forgiveness of the anastomosis and that only the pancreaticojejunostomy feeds the drug advisory.

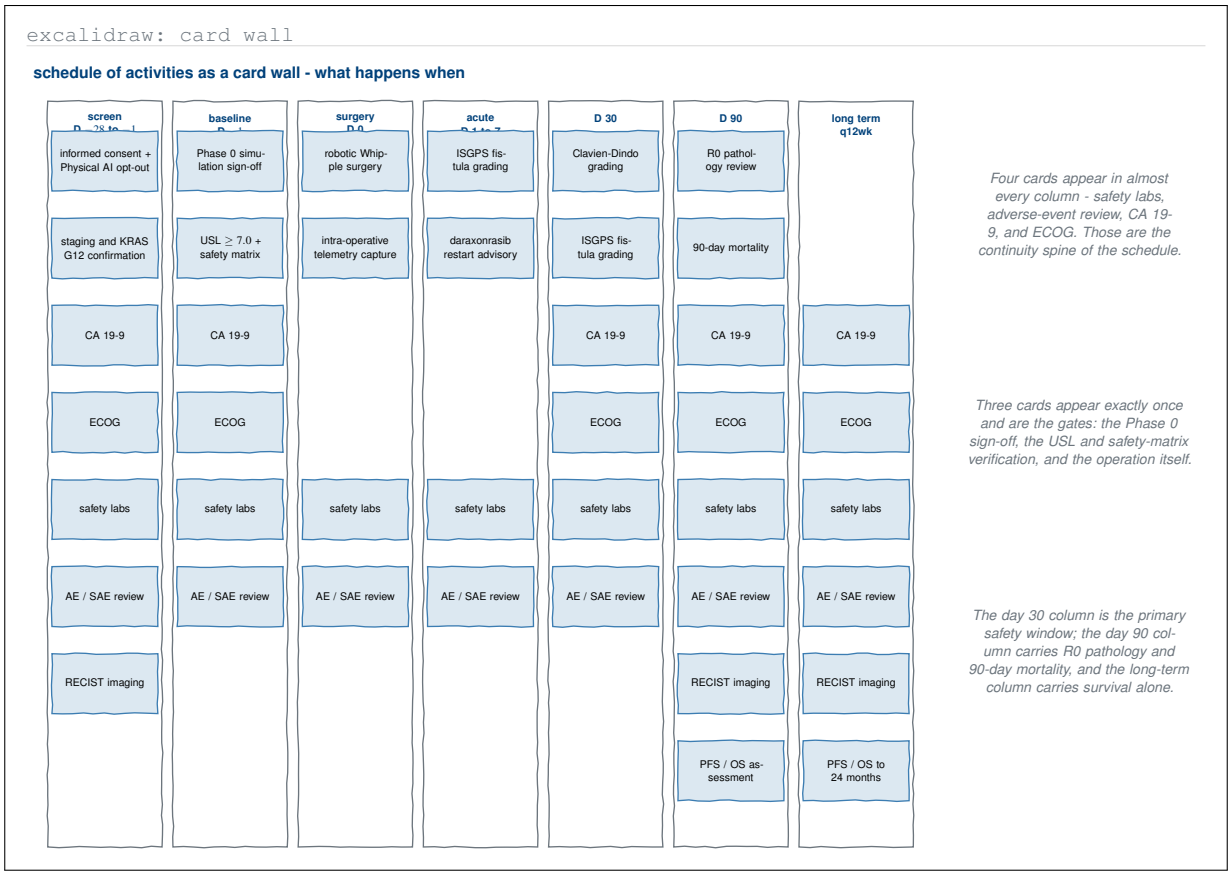


Figure 11. Excalidraw-type card wall of the Schedule of Activities, one column per visit and one card per assessment. Laid out as cards rather than as a cross-tabulated grid, the schedule separates into a continuity spine that recurs at almost every visit and three single-occurrence gate cards that release the study from one stage to the next.

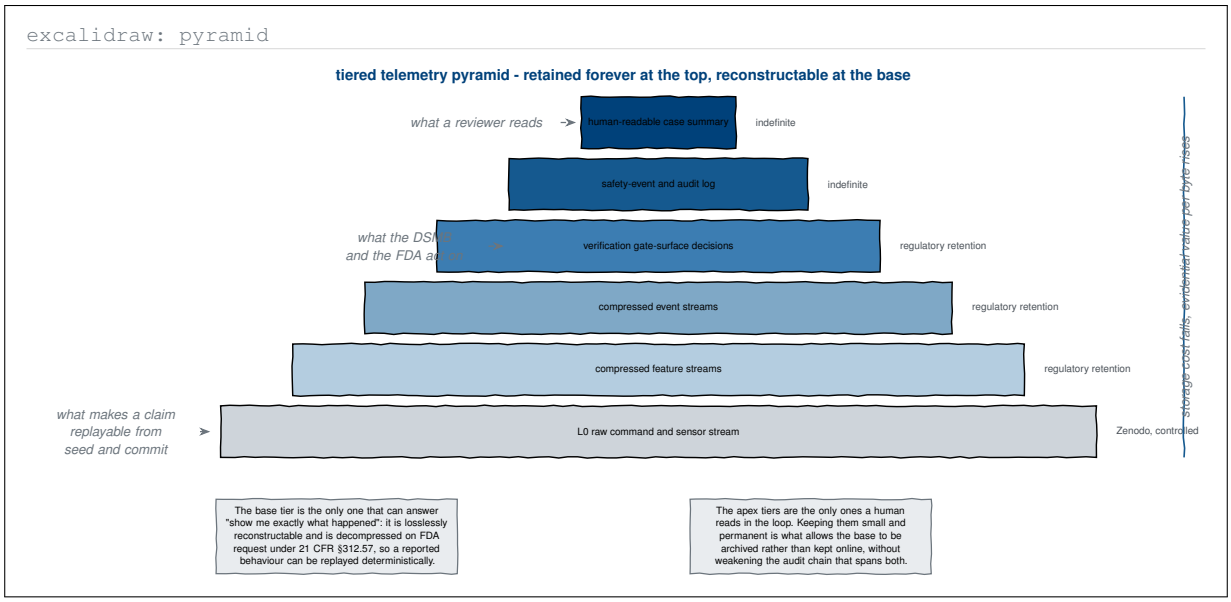


Figure 12. Excalidraw-type pyramid of the tiered telemetry record, from the permanent human-readable apex to the losslessly reconstructable level-zero base. The sketch adds the retention terms and the two readerships of the record, showing why the protocol can archive the largest tier without weakening the audit chain that spans every tier.

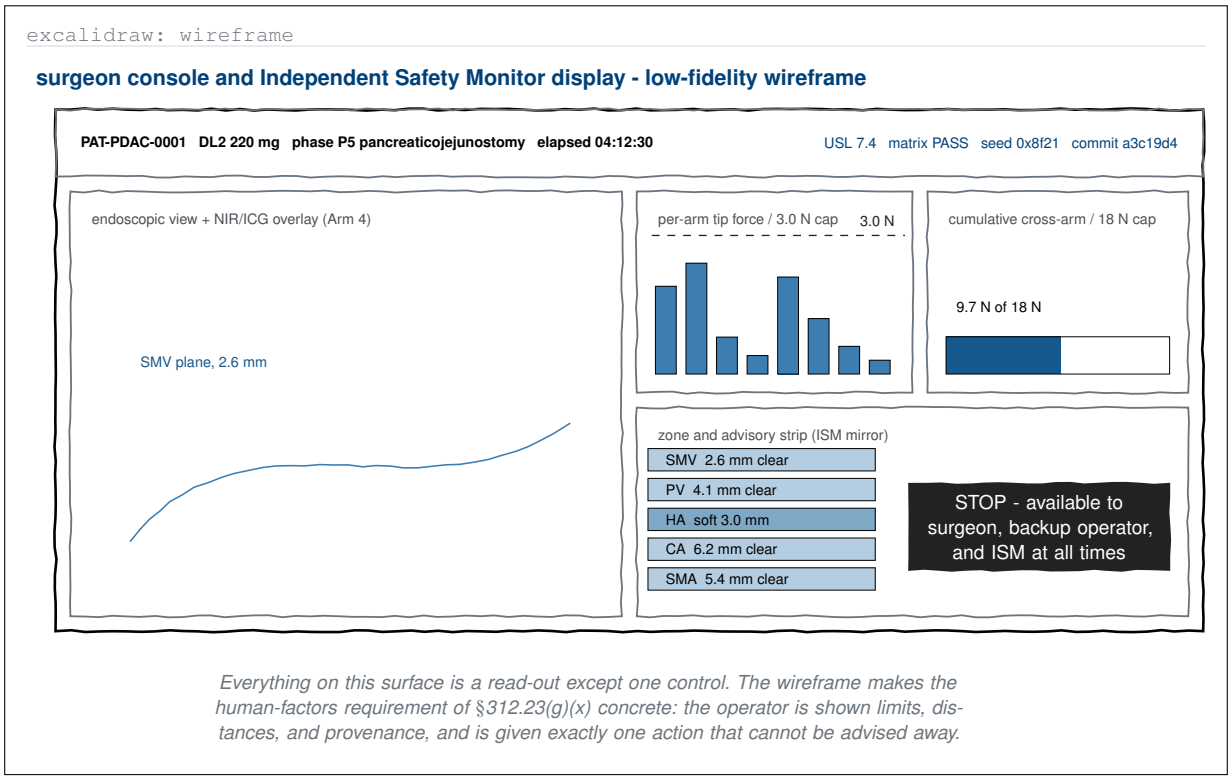


Figure 13. Excalidraw-type low-fidelity wireframe of the operator console and the mirrored Independent Safety Monitor display, populated with values from a mid-case moment. The wireframe makes the human-robot interaction field of §312.23(g)(x) concrete: the surface is entirely read-out except for a single stop control available to three people at once.

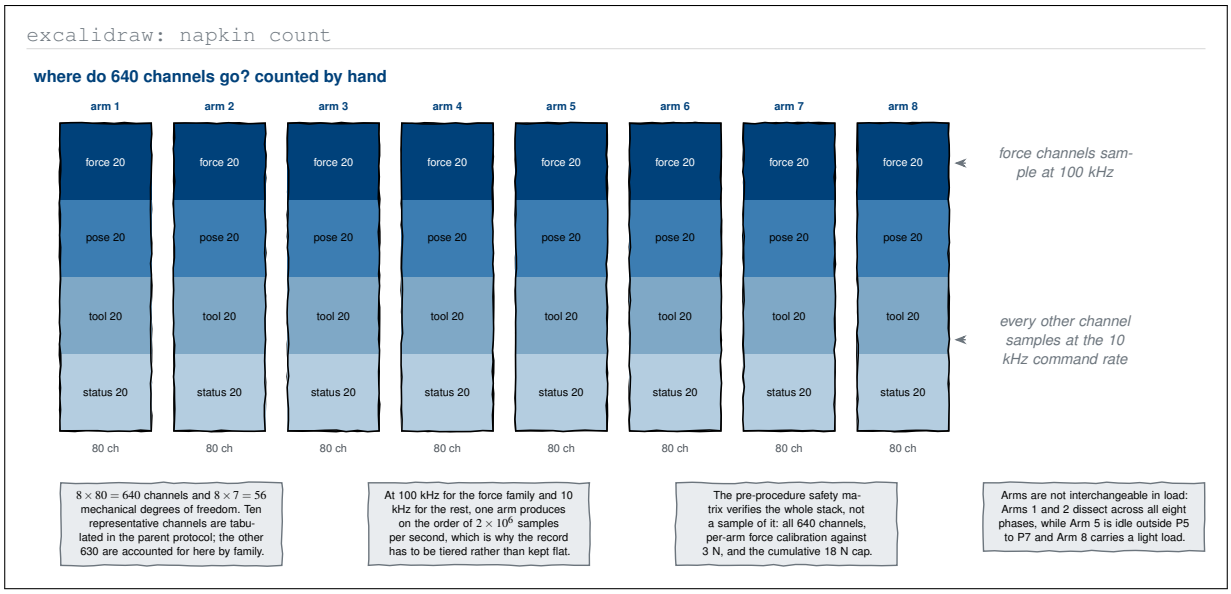


Figure 14. Excalidraw-type hand count of the 640-channel sensor budget, drawn as eight stacks of eighty and split into the four channel families each arm carries. The sketch turns two constants of the parent protocol into a sampling-rate argument, showing why a record on this scale has to be tiered rather than kept flat.

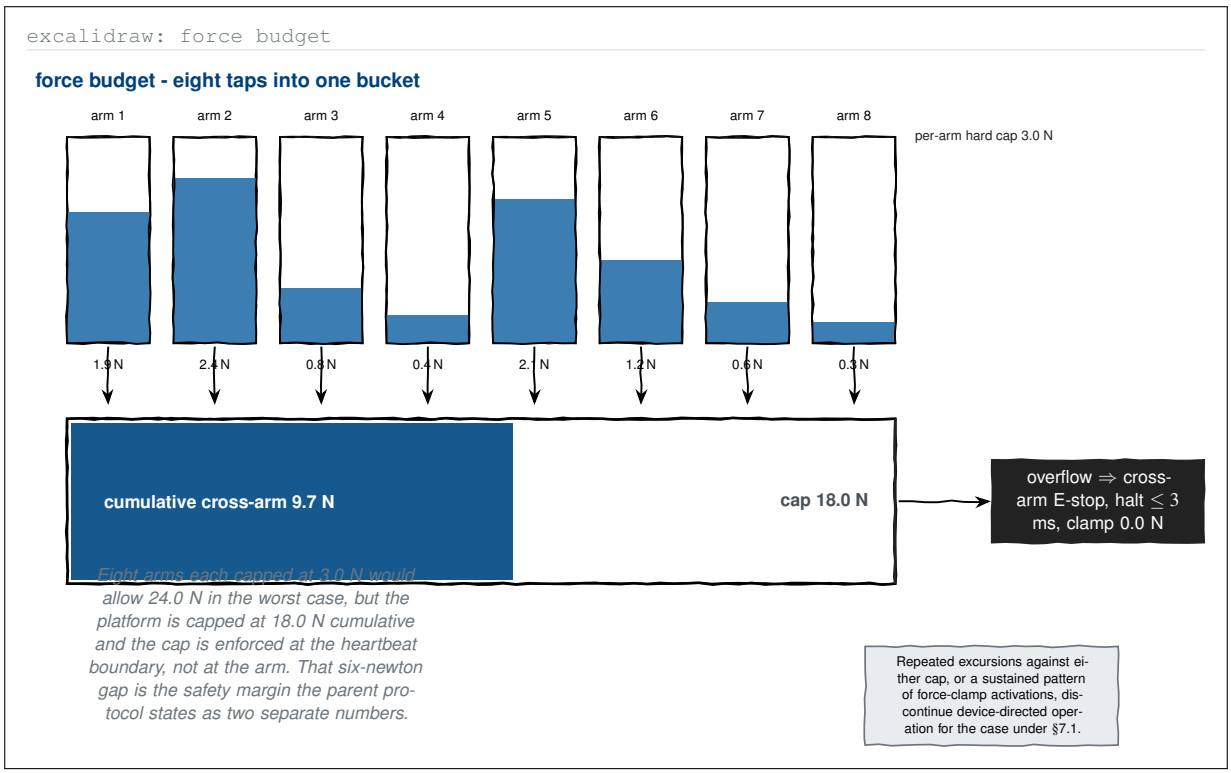


Figure 15. Excalidraw-type sketch of the force budget as eight taps feeding one bucket. Drawing the per-arm and cumulative caps together exposes a margin the parent protocol never states: eight arms at three newtons would permit twenty-four newtons, but the platform is clamped at eighteen and the clamp is enforced at the heartbeat boundary rather than at the arm.

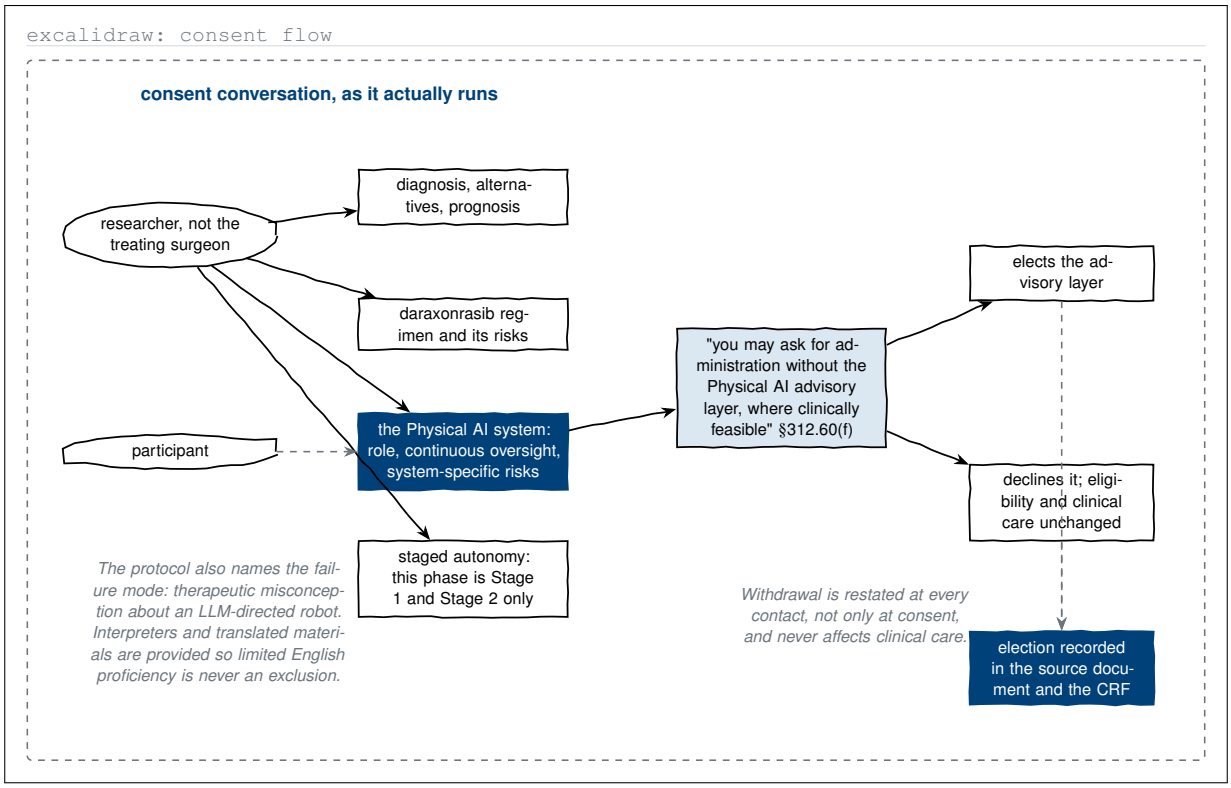


Figure 16. Excalidraw-type sketch of the consent conversation, drawn as dialogue rather than as a flow of boxes. The distinct Physical AI disclosure and the documented opt-out under 21 CFR §312.60(f) sit on the main line of the conversation, and both branches terminate at the same obligation: the participant's election is recorded in the source document and the case report form.

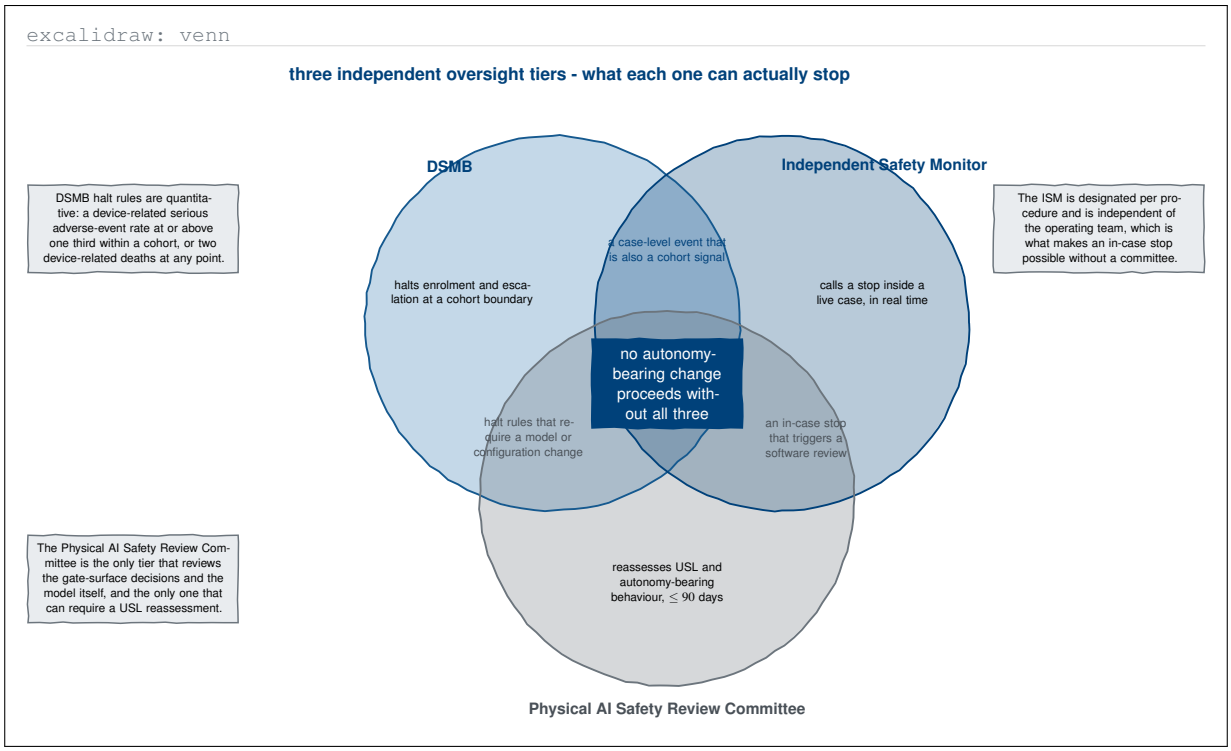


Figure 17. Excalidraw-type Venn sketch of the three independent oversight tiers, drawn so that the overlaps rather than the circles carry the argument. Each tier stops something the others cannot, and the central intersection is the protocol's actual rule: no autonomy-bearing change proceeds without all three.

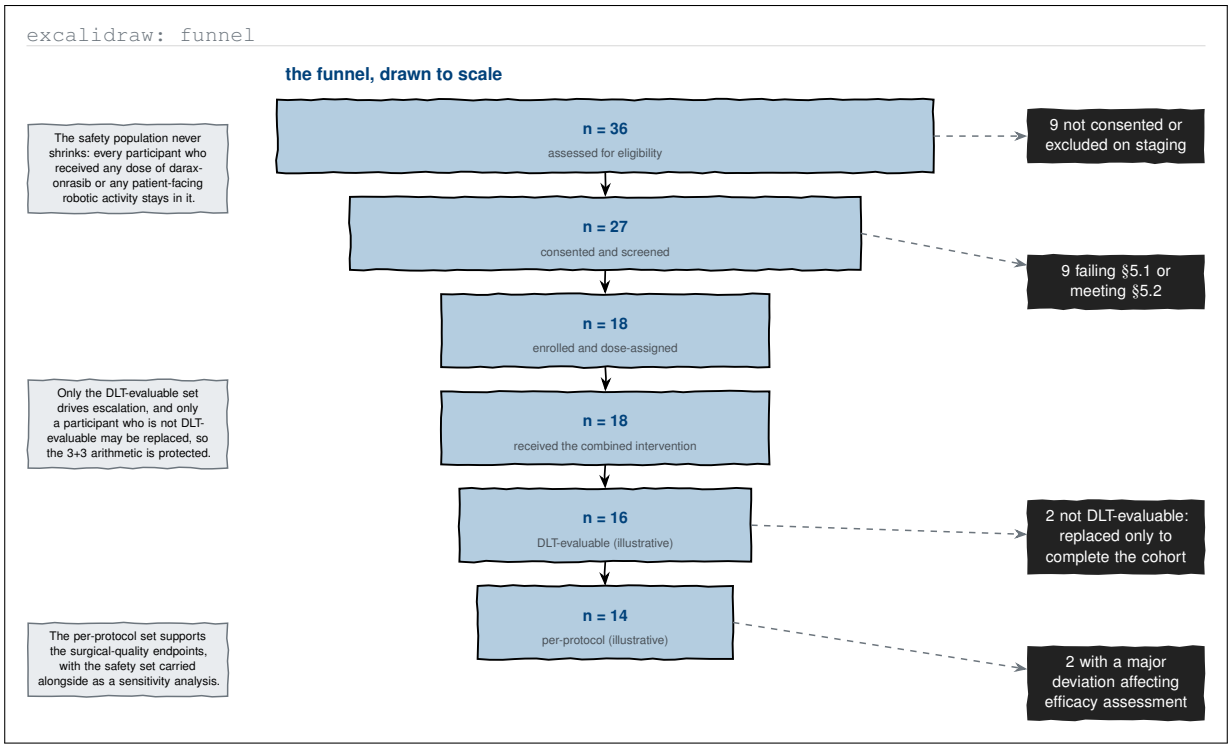


Figure 18. Excalidraw-type funnel drawn to scale from assessment through the four analysis populations, with illustrative attrition at each narrowing. The sketch fixes the asymmetry the parent statistics section states in words: the safety population never shrinks, while the evaluable and per-protocol sets do, and only non-evaluable participants may be replaced.

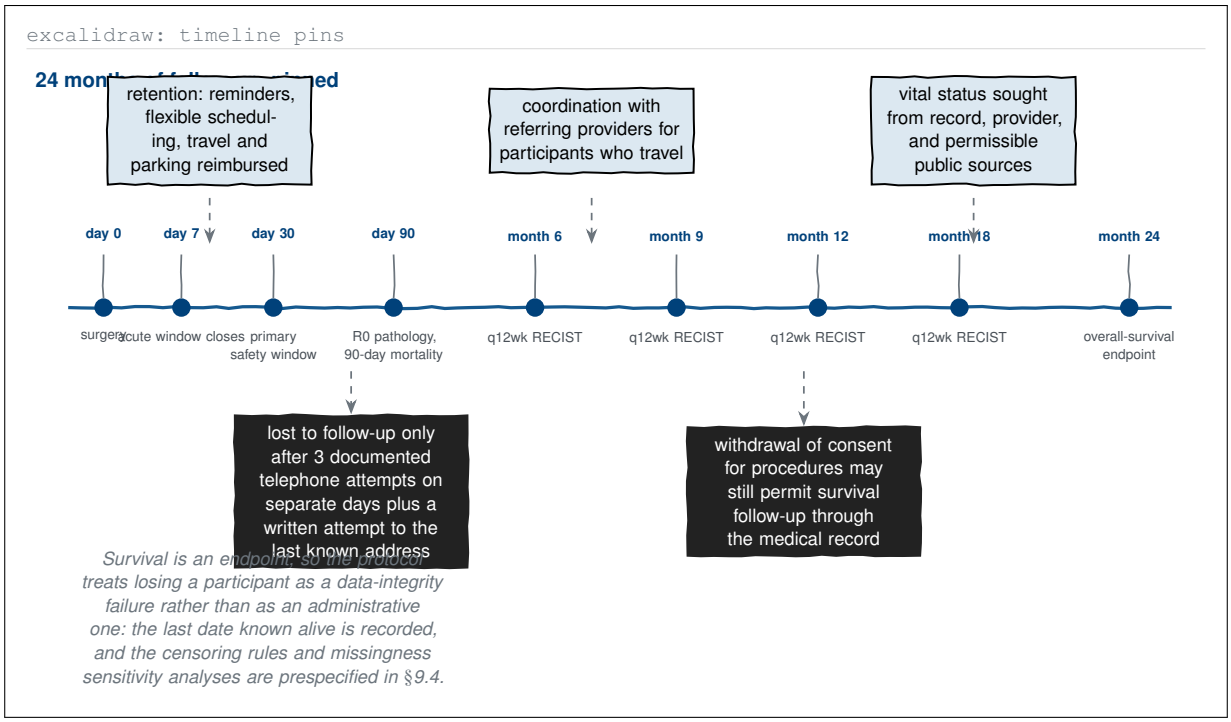


Figure 19. Excalidraw-type pinned timeline of the 24-month follow-up, carrying the retention measures above the axis and the two loss mechanisms below it. Drawn this way, the protocol's unusual treatment of loss to follow-up as a data-integrity failure rather than an administrative one becomes the visible feature of the tail.

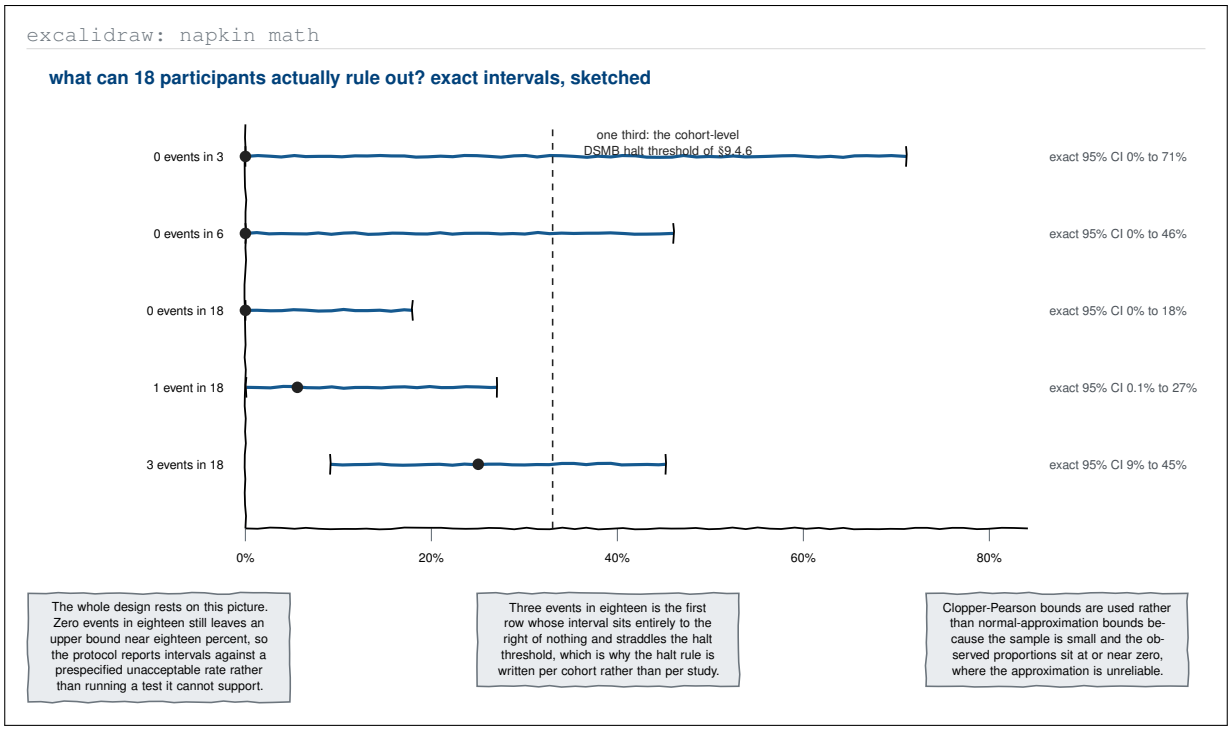


Figure 20. Excalidraw-type napkin arithmetic of what a cohort of eighteen can exclude, drawing the exact Clopper-Pearson intervals against the one-third cohort halt threshold. The sketch supplies the picture behind the parent protocol's decision to report estimation with exact intervals rather than run a significance test the sample cannot support.

3 PlantUML-type Diagrams

PlantUML's strength is formal notation: it carries the whole Unified Modeling Language, and it adds timing diagrams, salt wireframes, work-breakdown structures, ArchiMate layering, and C4 containers on the same textual grammar [14]. Where Mermaid is best at narrating a decision and Excalidraw is best at holding one open, PlantUML is best at pinning a design down so that two readers cannot interpret it differently. That is what a combined IND/IDE needs for its actors, its interfaces, its deployment, its stereotypes, and its timing budgets. The twenty figures below are referred to as **PlantUML-type** figures, and each is tagged with the PlantUML directive that would produce it.

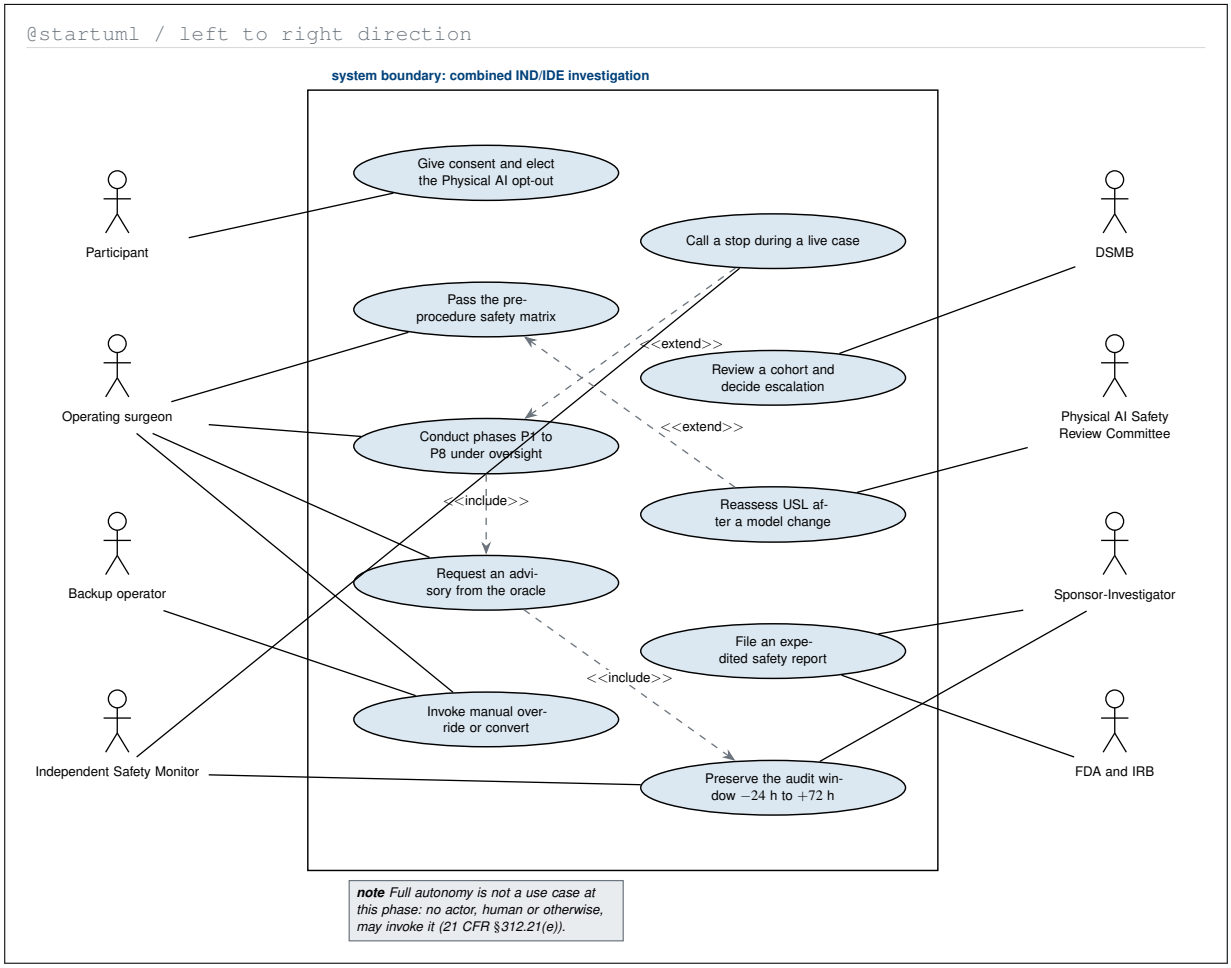


Figure 1. PlantUML-type use-case diagram of the combined investigation, with the four operational actors on the left, the four oversight actors on the right, and the system boundary drawn around the ten use cases the protocol authorises. Full autonomy appears nowhere inside the boundary, which states the §312.21(e) prohibition as a structural fact rather than a sentence.

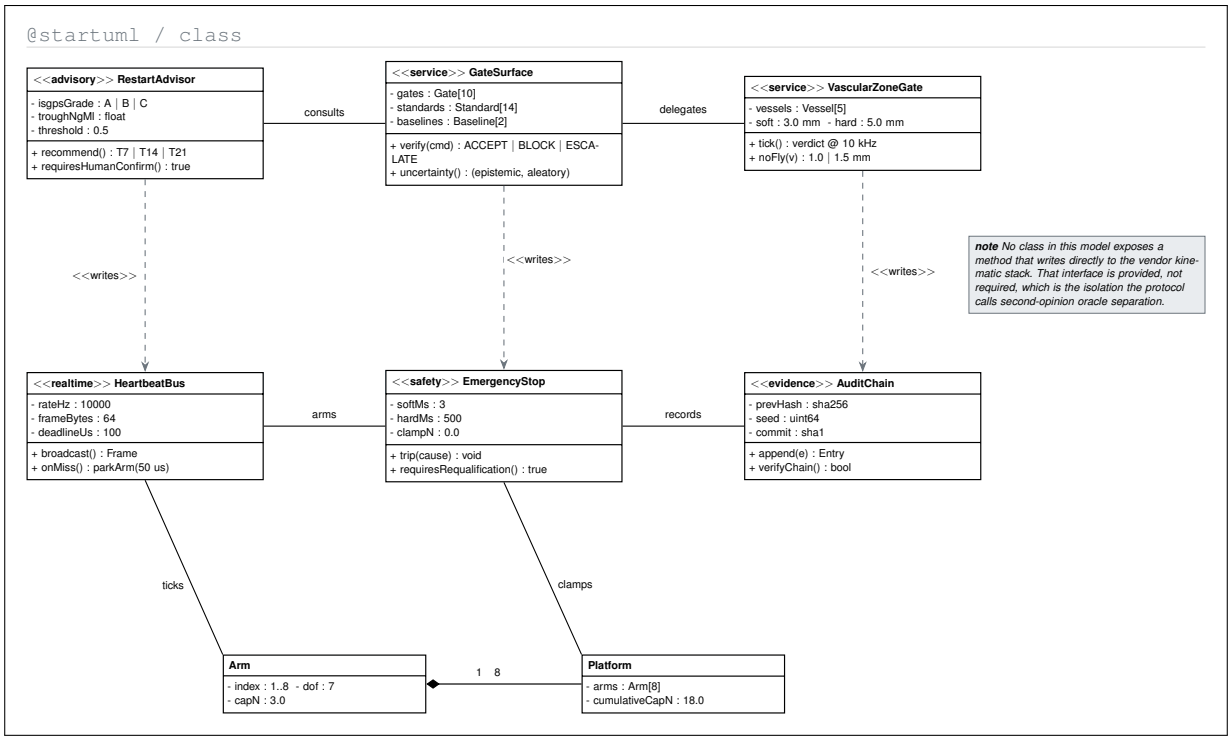


Figure 2. PlantUML-type class diagram of the control-side software model, with stereotypes separating advisory, service, real-time, safety, and evidence responsibilities. The model encodes the protocol's isolation claim as a structural property: no class offers an operation that writes to the vendor kinematic stack directly.

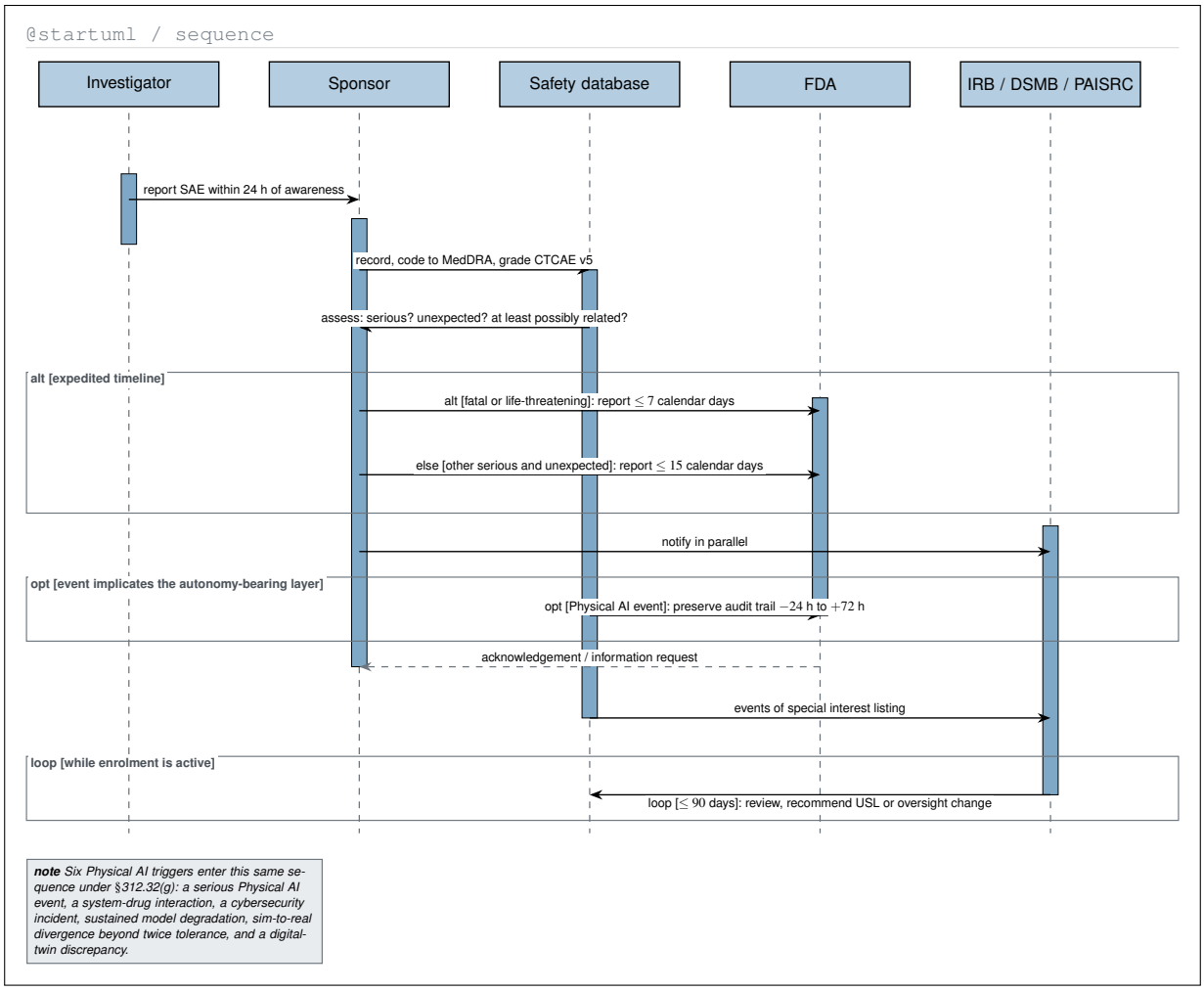


Figure 3. PlantUML-type sequence diagram of the safety-reporting workflow, with the alt, opt, and loop fragments PlantUML draws natively. The two expedited clocks, the parallel notification of the oversight bodies, and the audit-window preservation obligation are placed on one timeline so their interaction rather than their individual definitions is what is read.

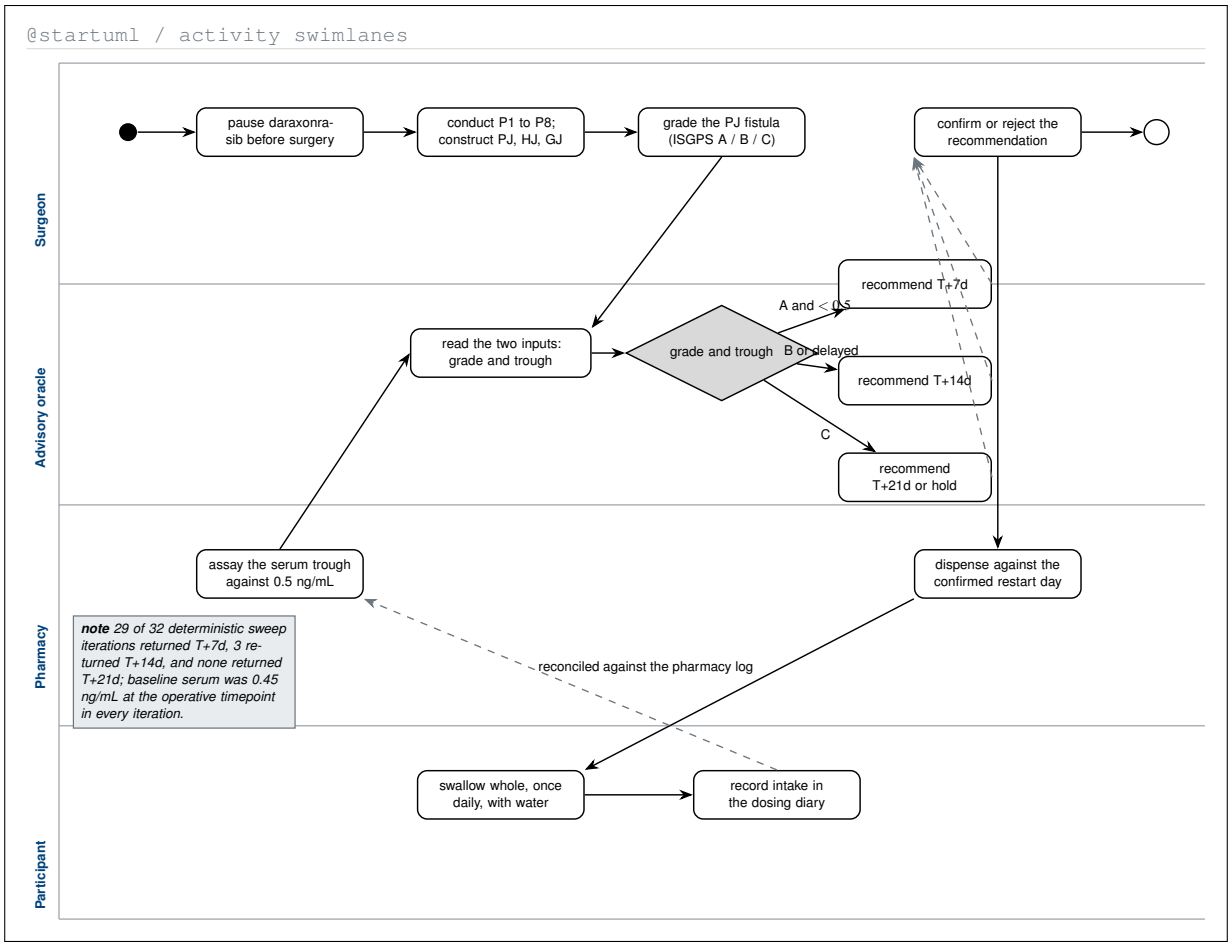


Figure 4. PlantUML-type activity diagram with four swimlanes, assigning every step of the perioperative pause-and-restart advisory to the party that performs it. Placing the oracle in its own lane makes the protocol's constraint visible: the advisory lane produces a recommendation and never a dispensing action, and every path leaves it through the surgeon's confirmation.

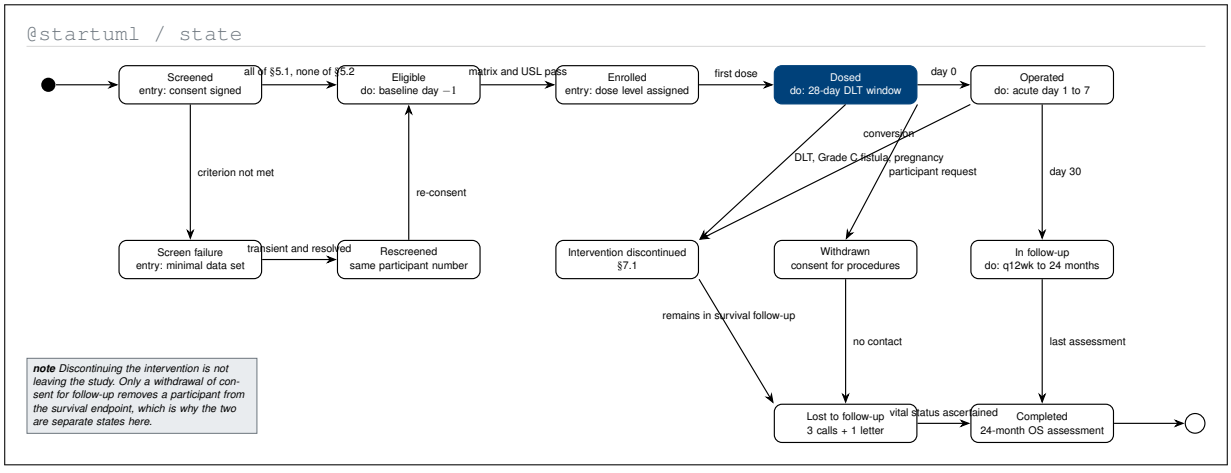


Figure 5. PlantUML-type state machine of the participant lifecycle, from screening to the 24-month endpoint. Separating discontinuation from withdrawal into distinct states with distinct exits renders the distinction §7 makes in prose, namely that stopping the intervention leaves a participant in survival follow-up while withdrawing consent for follow-up does not.

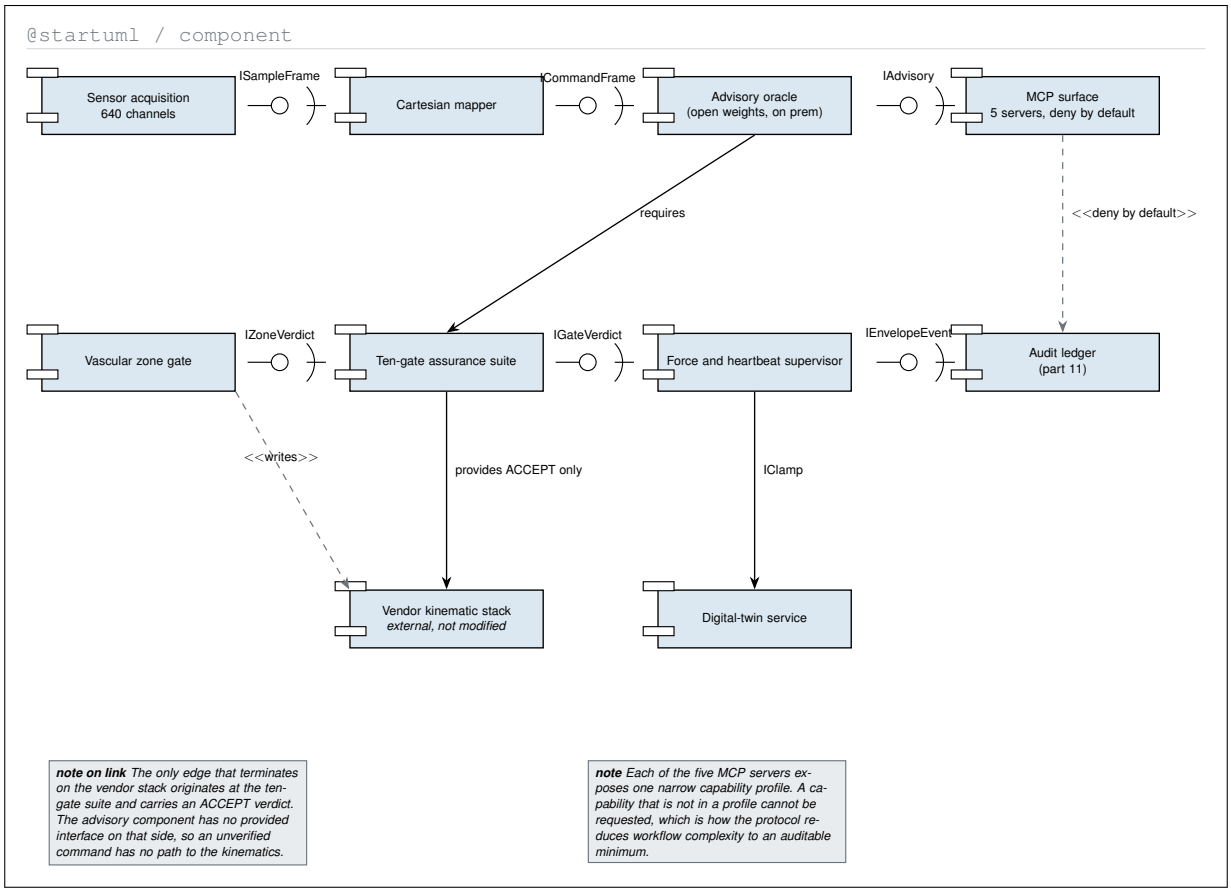


Figure 6. PlantUML-type component diagram with explicit provided and required interfaces in ball-and-socket notation. Drawn as components rather than as boxes, the isolation argument becomes checkable by inspection: exactly one edge terminates on the vendor kinematic stack, and it originates at the ten-gate assurance suite carrying an ACCEPT verdict.

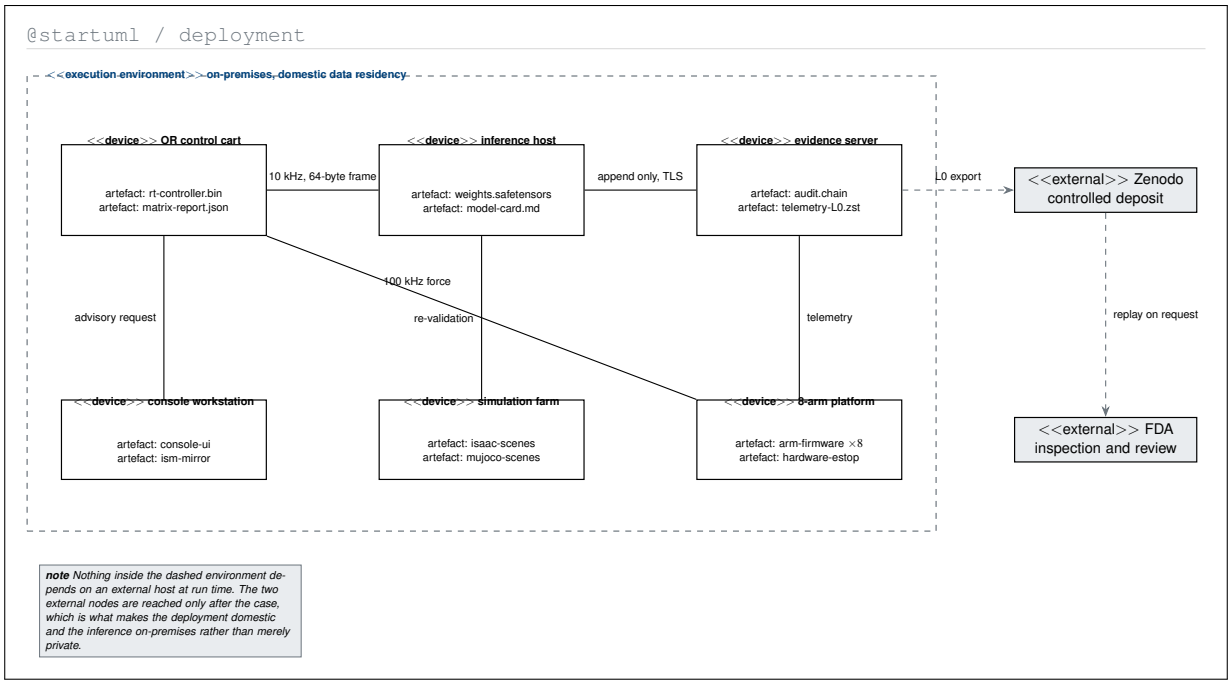


Figure 7. PlantUML-type deployment diagram of the physical nodes and the artefacts each one carries, wrapped in the on-premises execution environment. Drawing the two external nodes outside that boundary and reaching them only after the case makes the data-residency claim of Table 1 checkable rather than asserted.

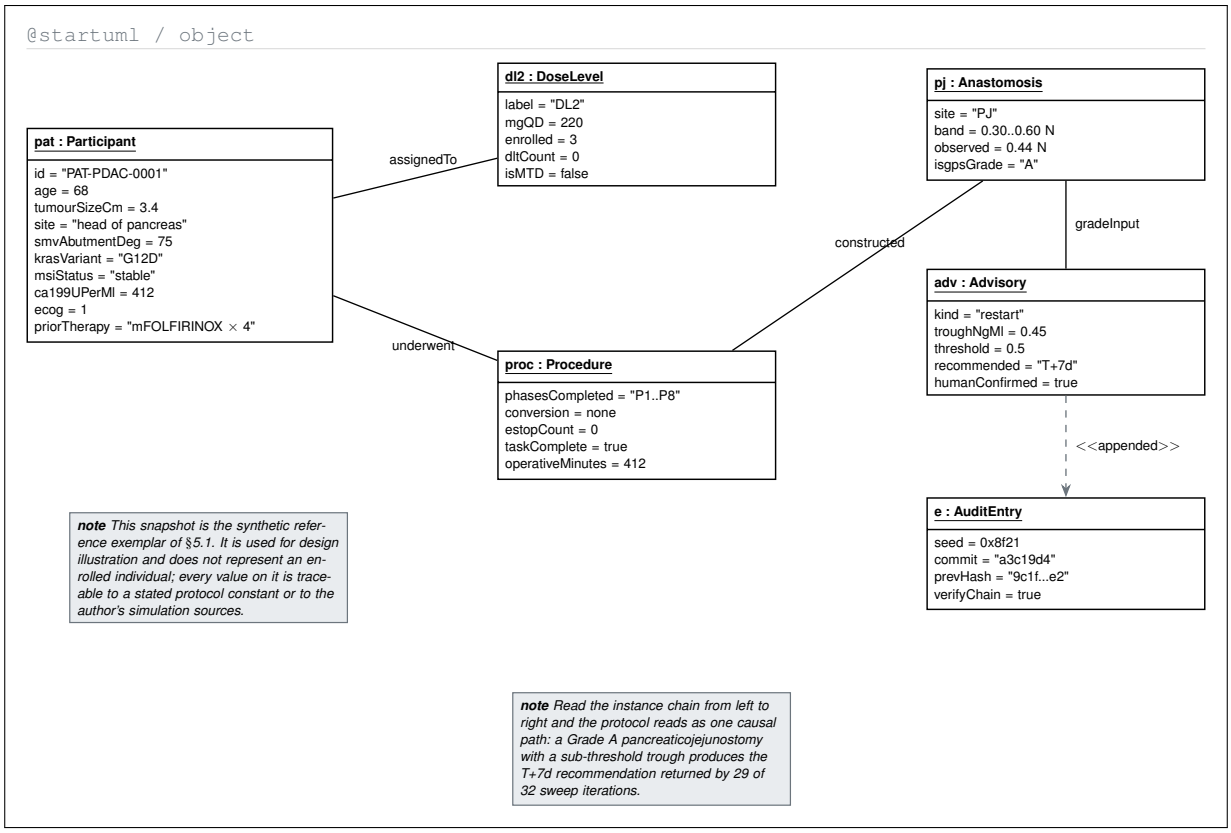


Figure 8. PlantUML-type object diagram instantiating the class model on the synthetic reference exemplar. Following the instance chain from participant to audit entry traces the protocol's single coupling point end to end: a Grade A pancreaticojejunostomy with a sub-threshold trough yields the T+7d restart that dominated the simulation sweep.

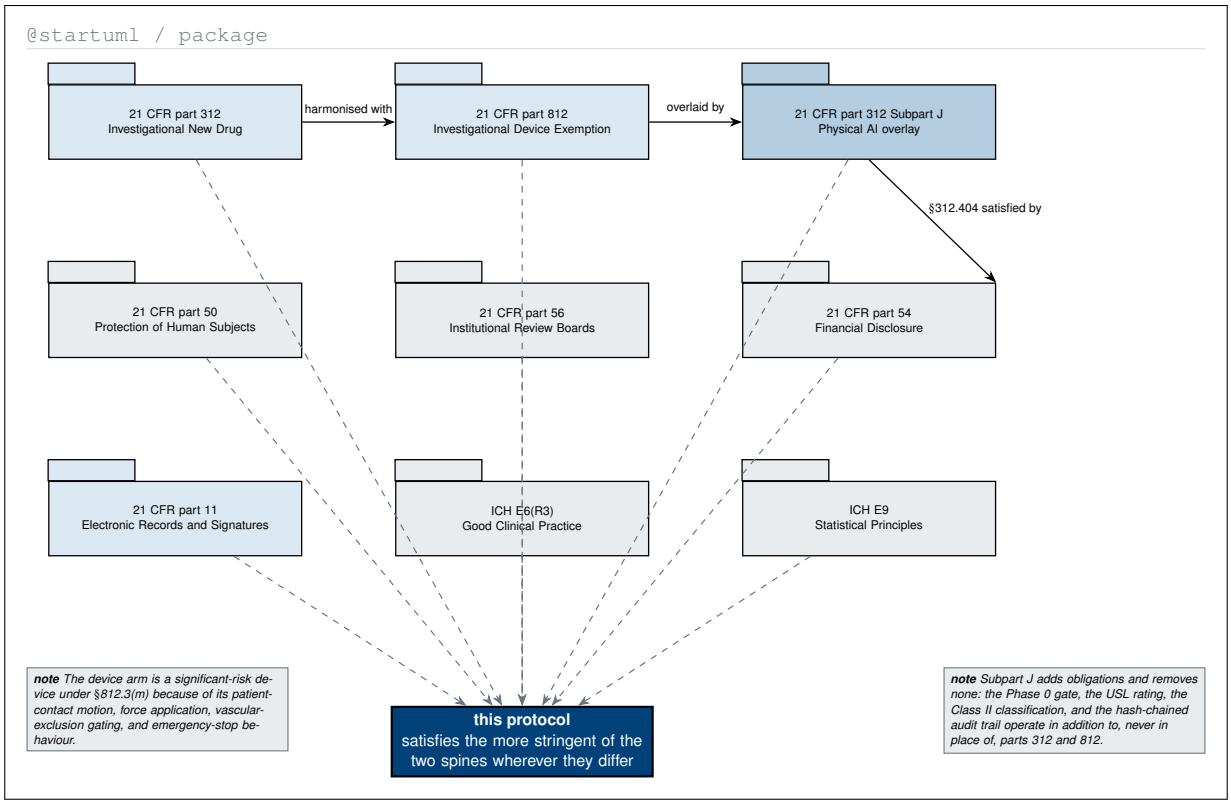


Figure 9. PlantUML-type package diagram of the regulatory spine, drawn with the tabbed package notation. The dependency edges make the compliance statement of §0 structural: two harmonised spines, an overlay that only adds, and one protocol that must satisfy whichever of the two is more stringent at any point of difference.

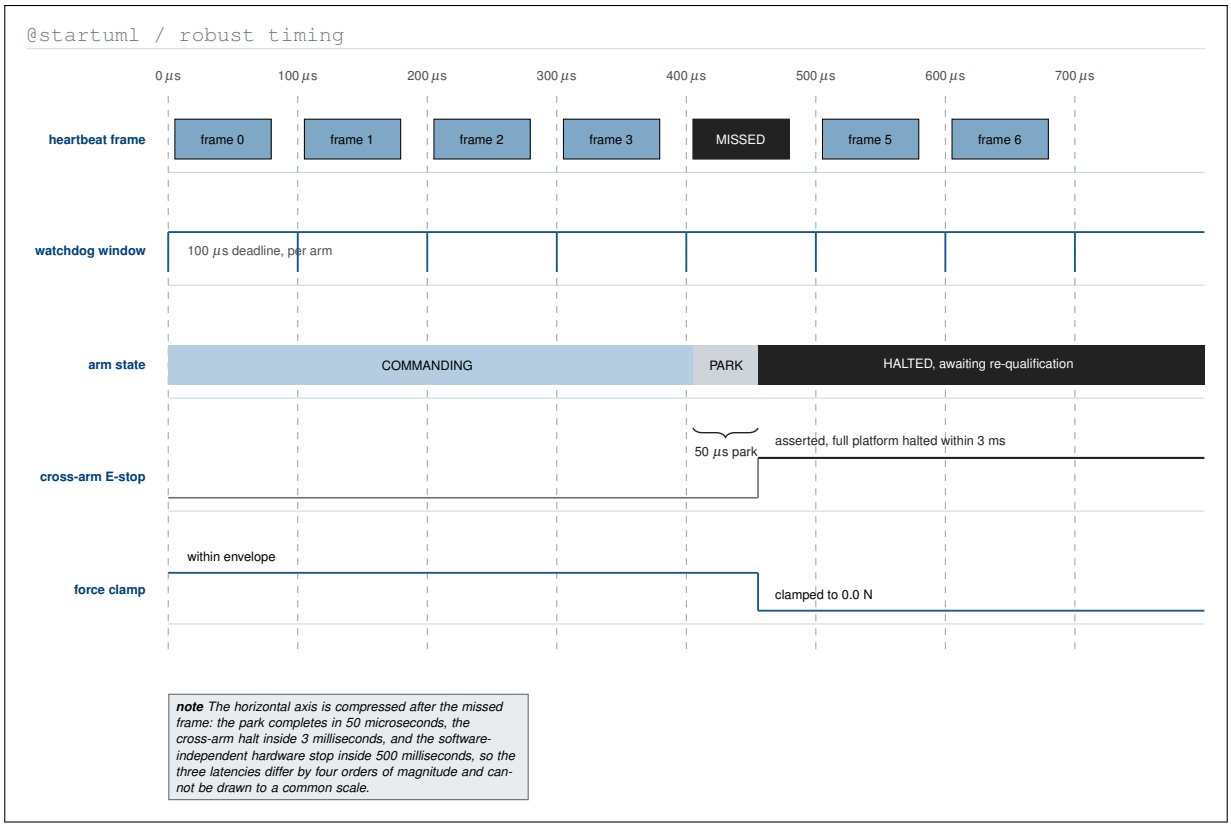


Figure 10. PlantUML-type robust timing diagram of five signals across seven heartbeat periods, ending in a missed frame. It is the only figure in the atlas that puts the frame deadline, the fifty microsecond park, and the three millisecond halt on one axis, and it records why they cannot share a scale: the three latencies span four orders of magnitude.

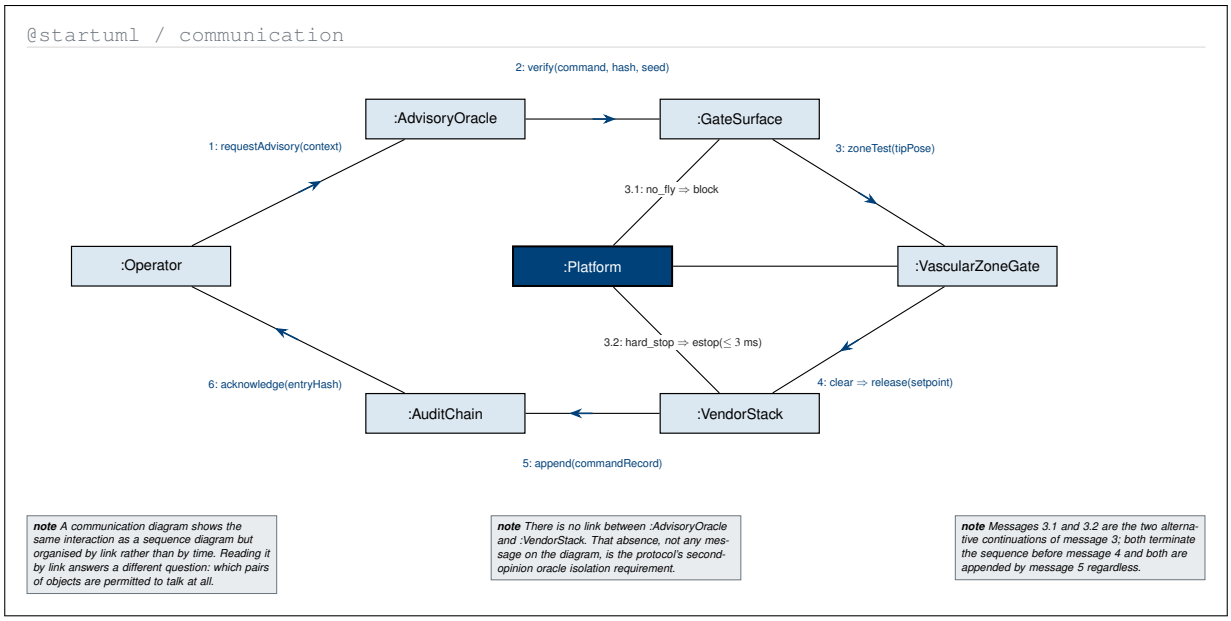


Figure 11. PlantUML-type communication diagram of the same interaction as the control sequence, but organised by link rather than by time. Read this way it answers a different question, namely which pairs of objects may communicate at all, and the missing link between the advisory oracle and the vendor stack is the isolation requirement itself.

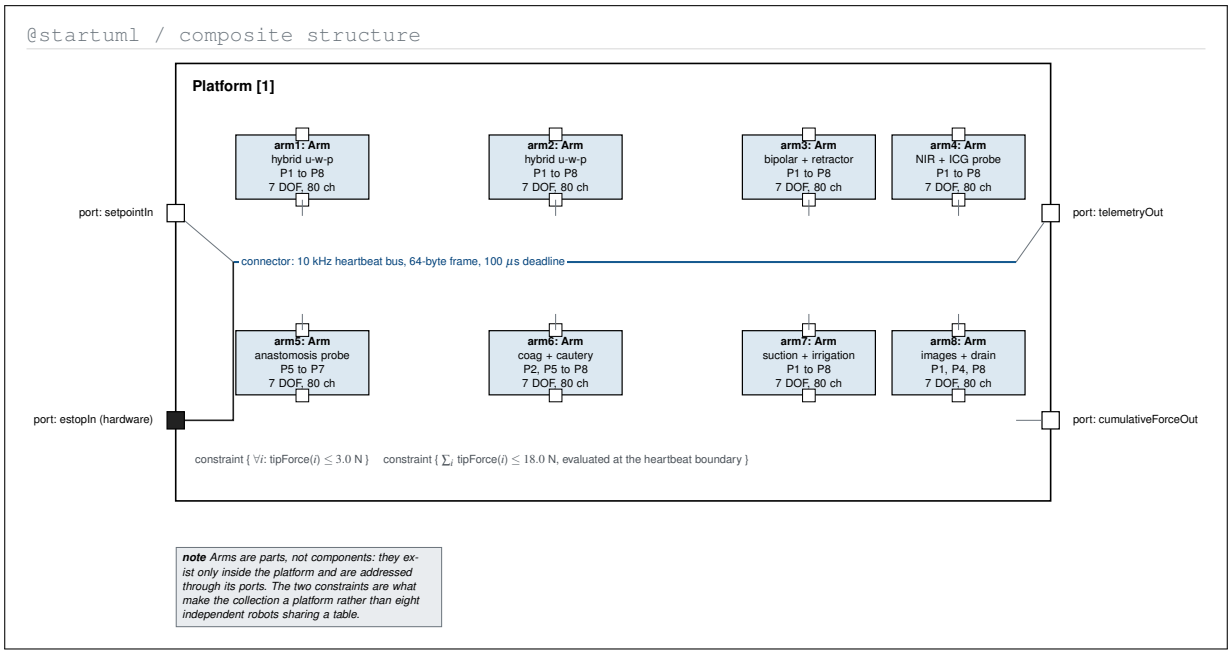


Figure 12. PlantUML-type composite structure diagram of the eight-arm platform, showing the arms as internal parts joined by the heartbeat connector and reached only through the platform's four ports. The two force constraints written on the frame are what make the collection a platform rather than eight independent robots sharing a table.

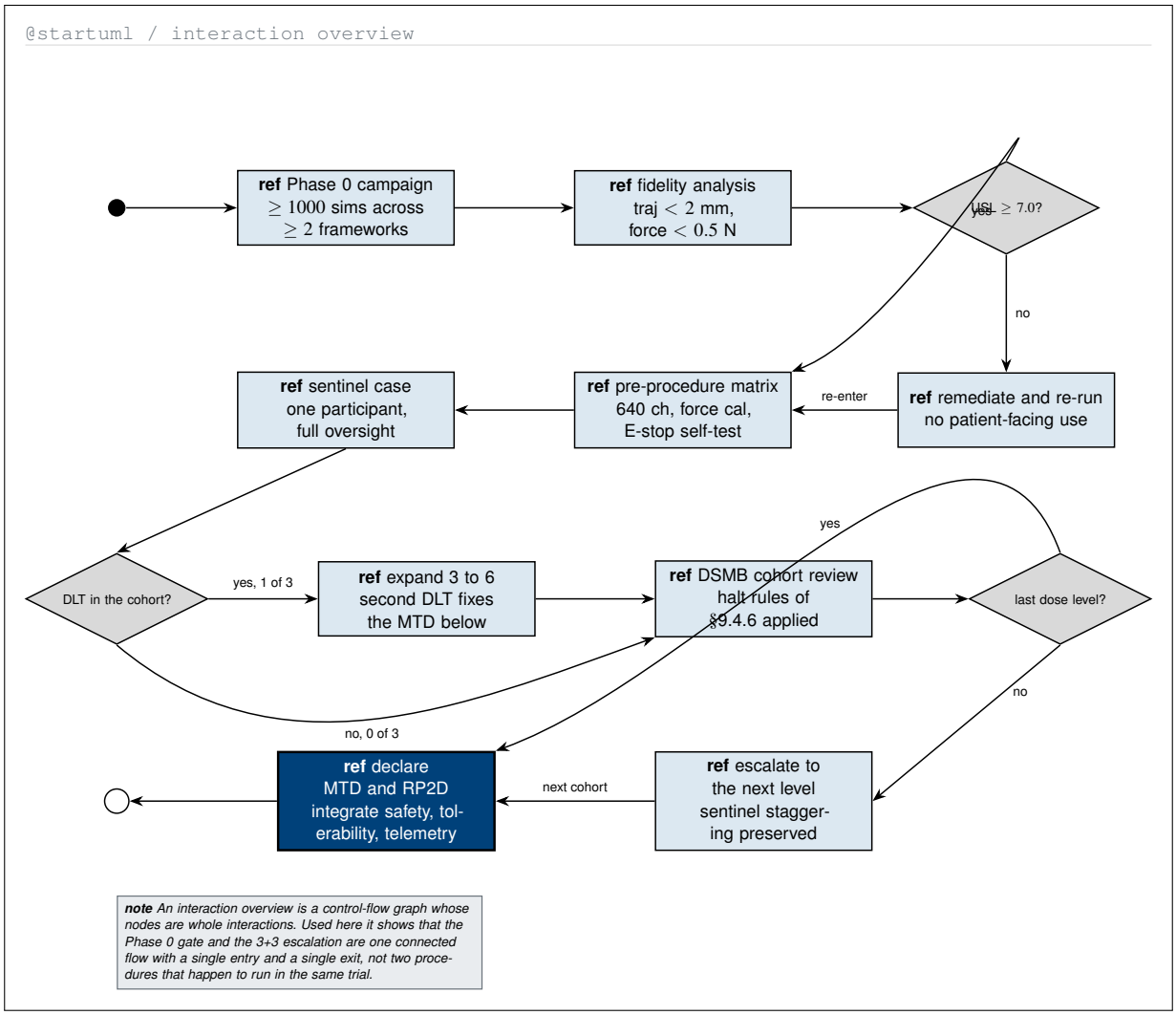


Figure 13. PlantUML-type interaction overview diagram whose nodes are whole interactions rather than single steps. Composed this way, the Phase 0 validation gate and the 3+3 escalation appear as one connected control flow with a single entry and a single exit, rather than as two procedures that happen to share a trial.

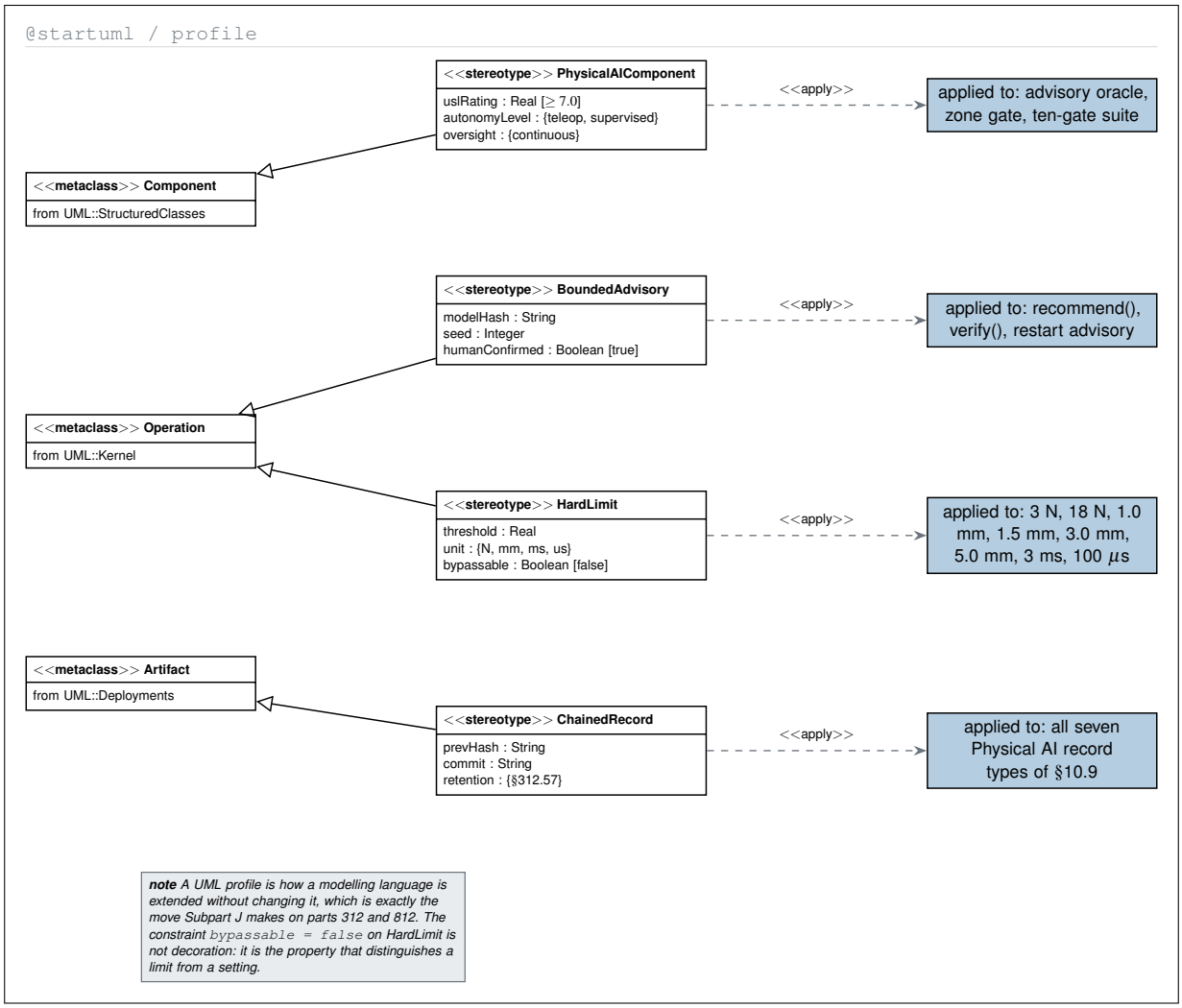


Figure 14. PlantUML-type profile diagram defining four Physical AI stereotypes as extensions of standard UML metaclasses, with the protocol constants each one is applied to. The profile mechanism mirrors what Subpart J does to parts 312 and 812, and the false bypassable constraint is what separates a hard limit from a configurable setting.

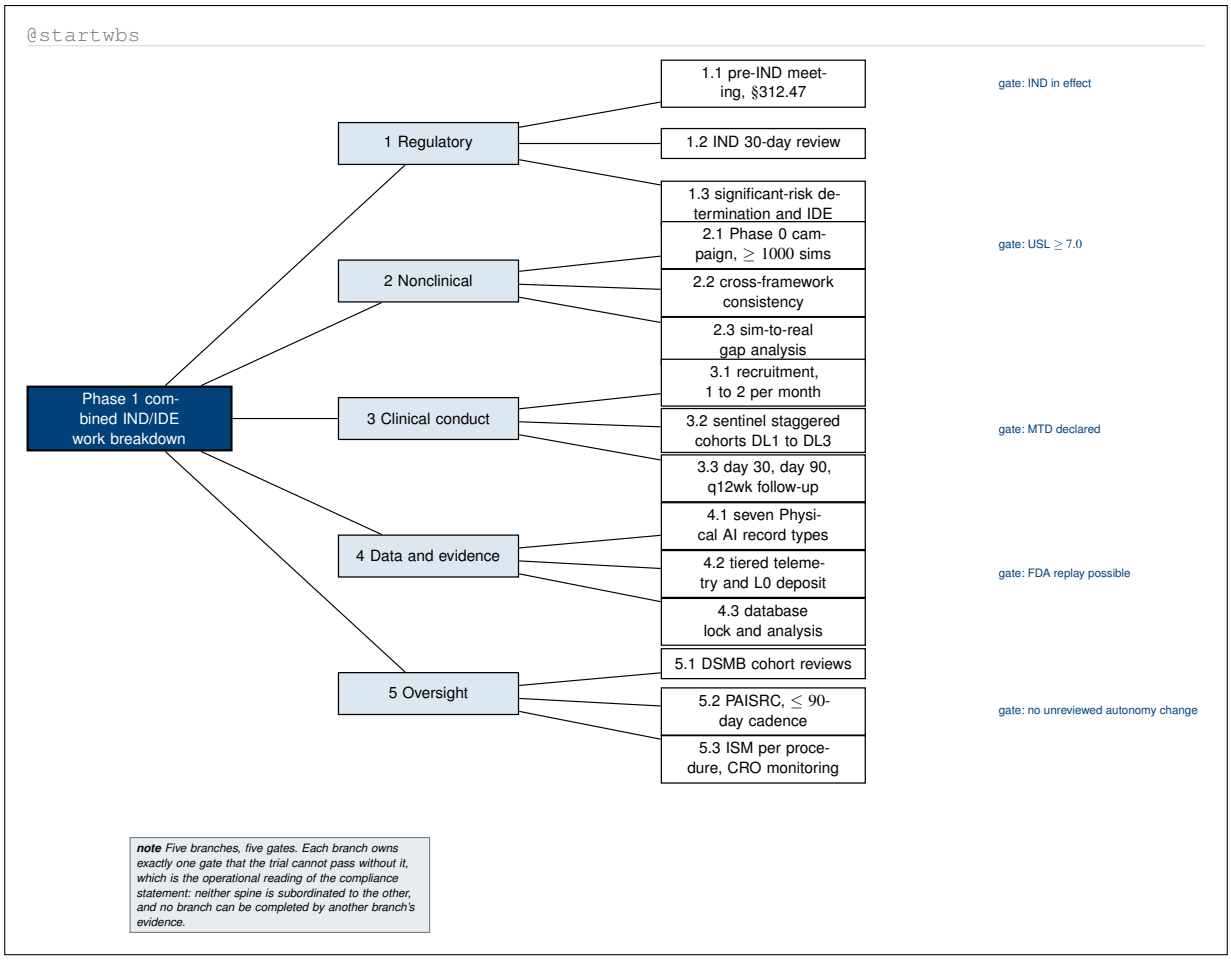


Figure 15. PlantUML-type work breakdown structure decomposing the study into five branches, each carrying exactly one gate the trial cannot pass without. The symmetry is the point: no branch can be completed with another branch's evidence, which is the operational reading of the compliance statement that neither regulatory spine is subordinated to the other.

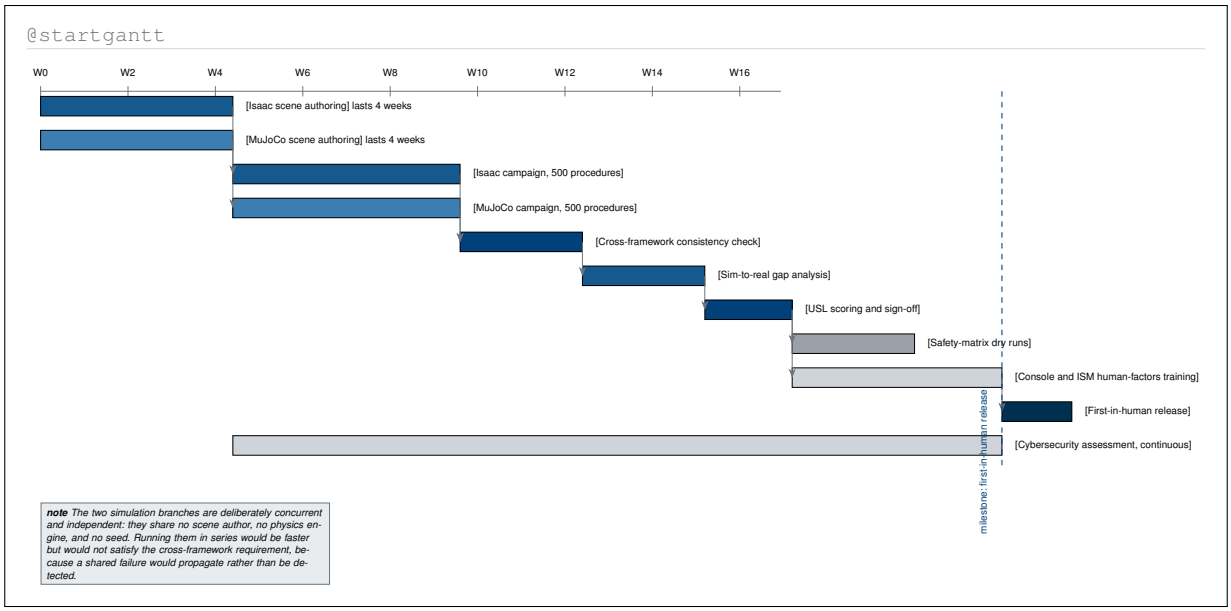


Figure 16. PlantUML-type project Gantt chart of the Phase 0 validation campaign, with the dependency arrows PlantUML draws between tasks. The two simulation branches are scheduled concurrently and share no author, engine, or seed, which is the scheduling consequence of the cross-framework requirement rather than a convenience.

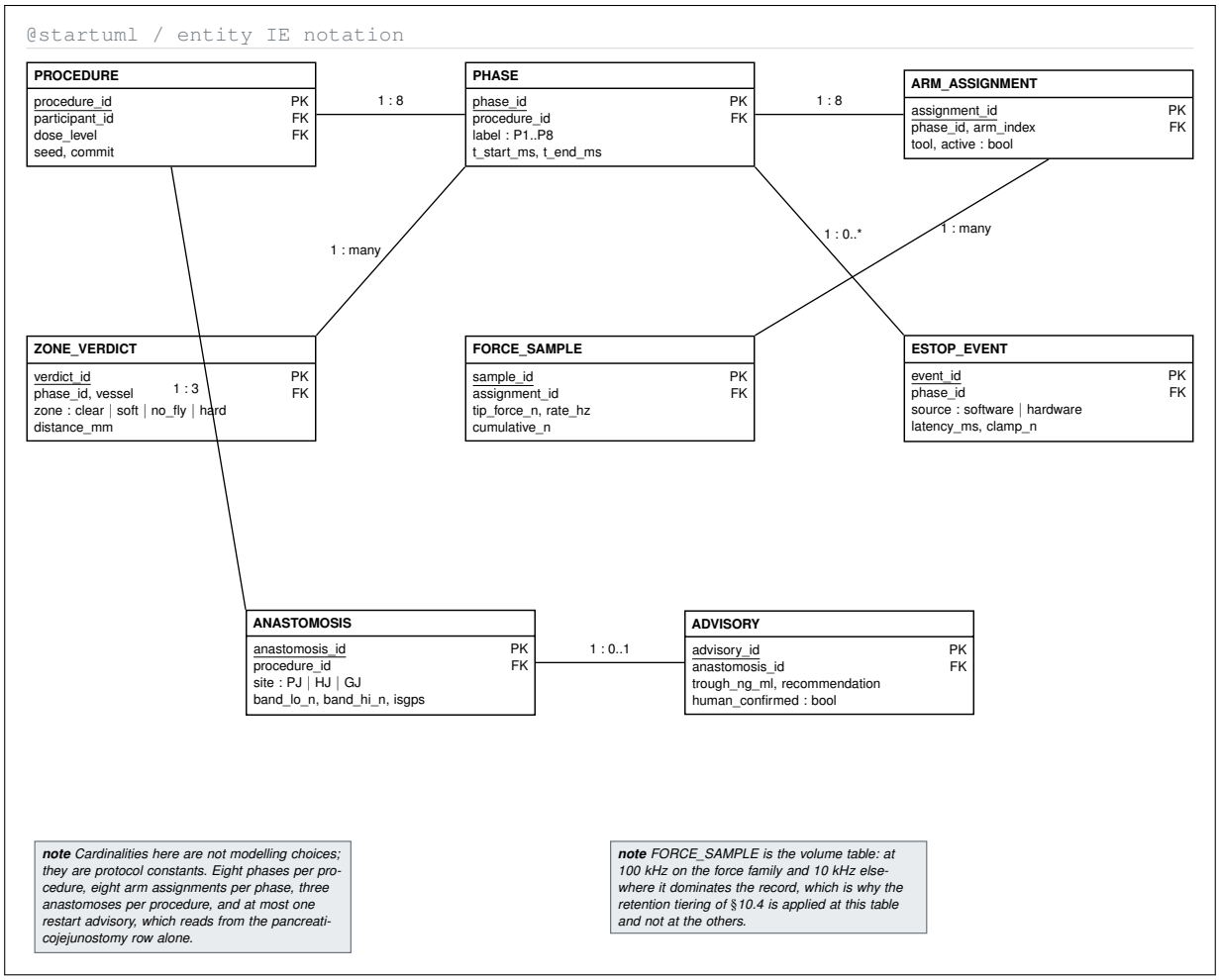


Figure 17. PlantUML-type entity-relationship diagram in information-engineering notation for the operative record. Every cardinality on it is a protocol constant rather than a modelling choice, and the one table that dominates the record by volume is identified, which is what motivates applying retention tiering at that table alone.

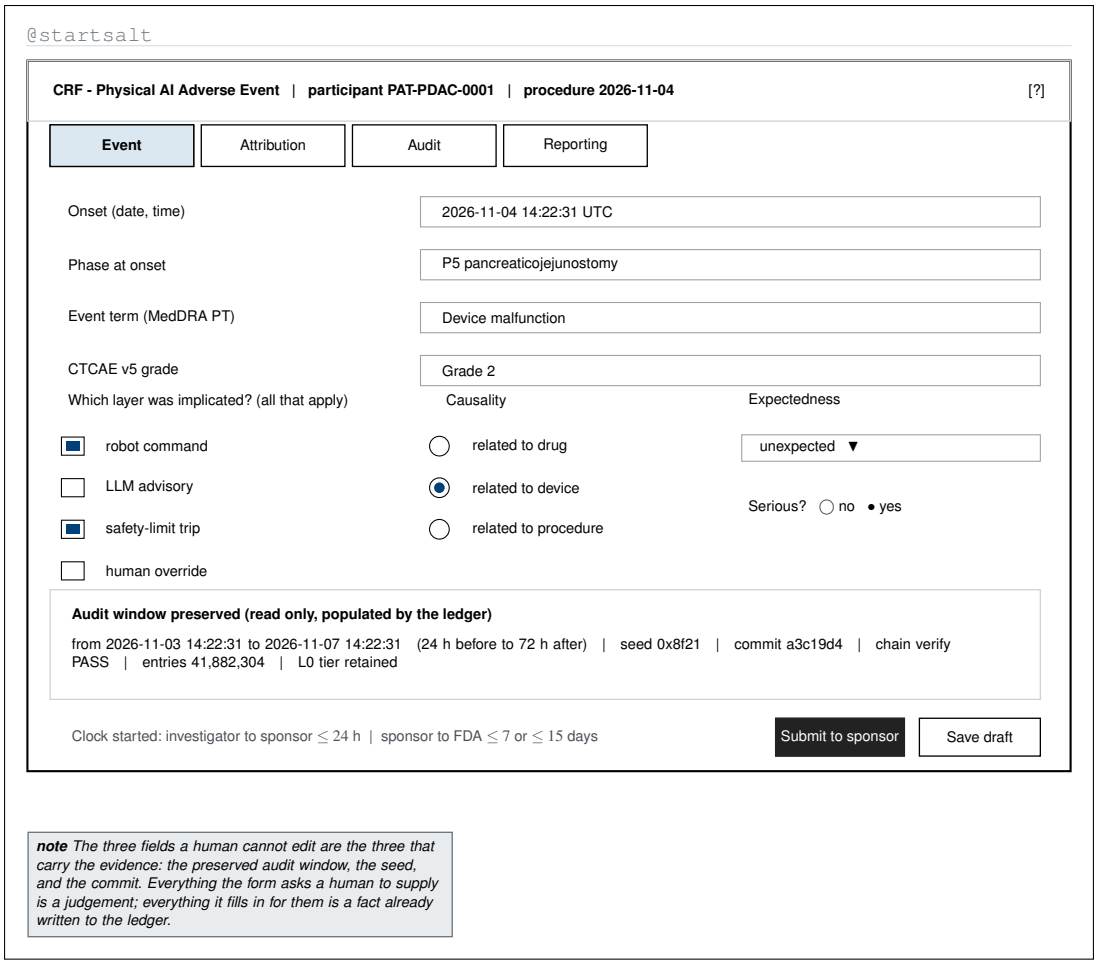


Figure 18. PlantUML-type salt wireframe of the case report form for a Physical AI adverse event, showing tabs, checkboxes, radio groups, and the read-only ledger panel. The wireframe separates the two kinds of field on the form: judgements a human supplies, and facts the hash-chained ledger has already written and the form merely displays.

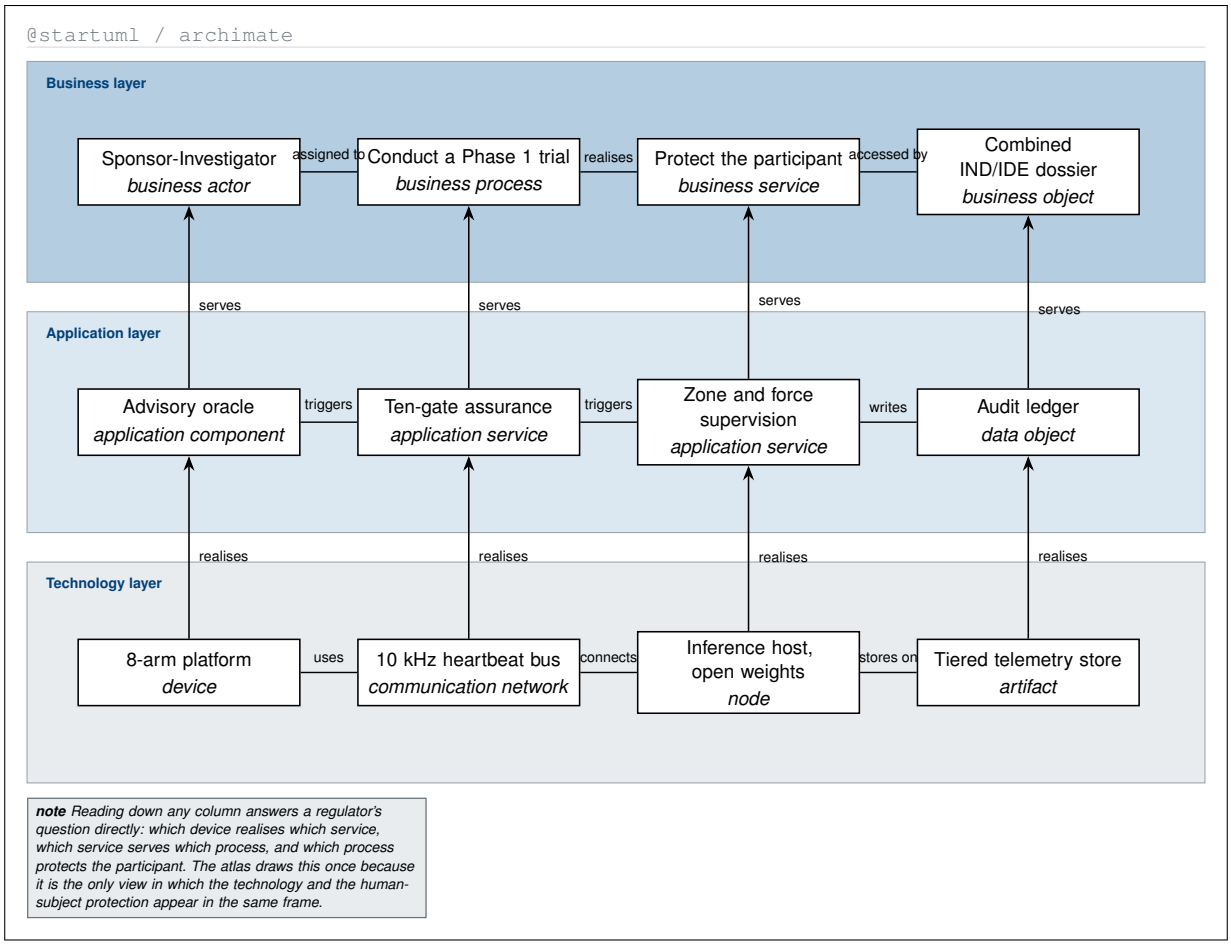


Figure 19. PlantUML-type ArchiMate layered view aligning business, application, and technology elements in three columns. Reading down a column answers the question a regulator actually asks, tracing a specific device through the service it realises to the process it serves and the participant protection that process delivers.

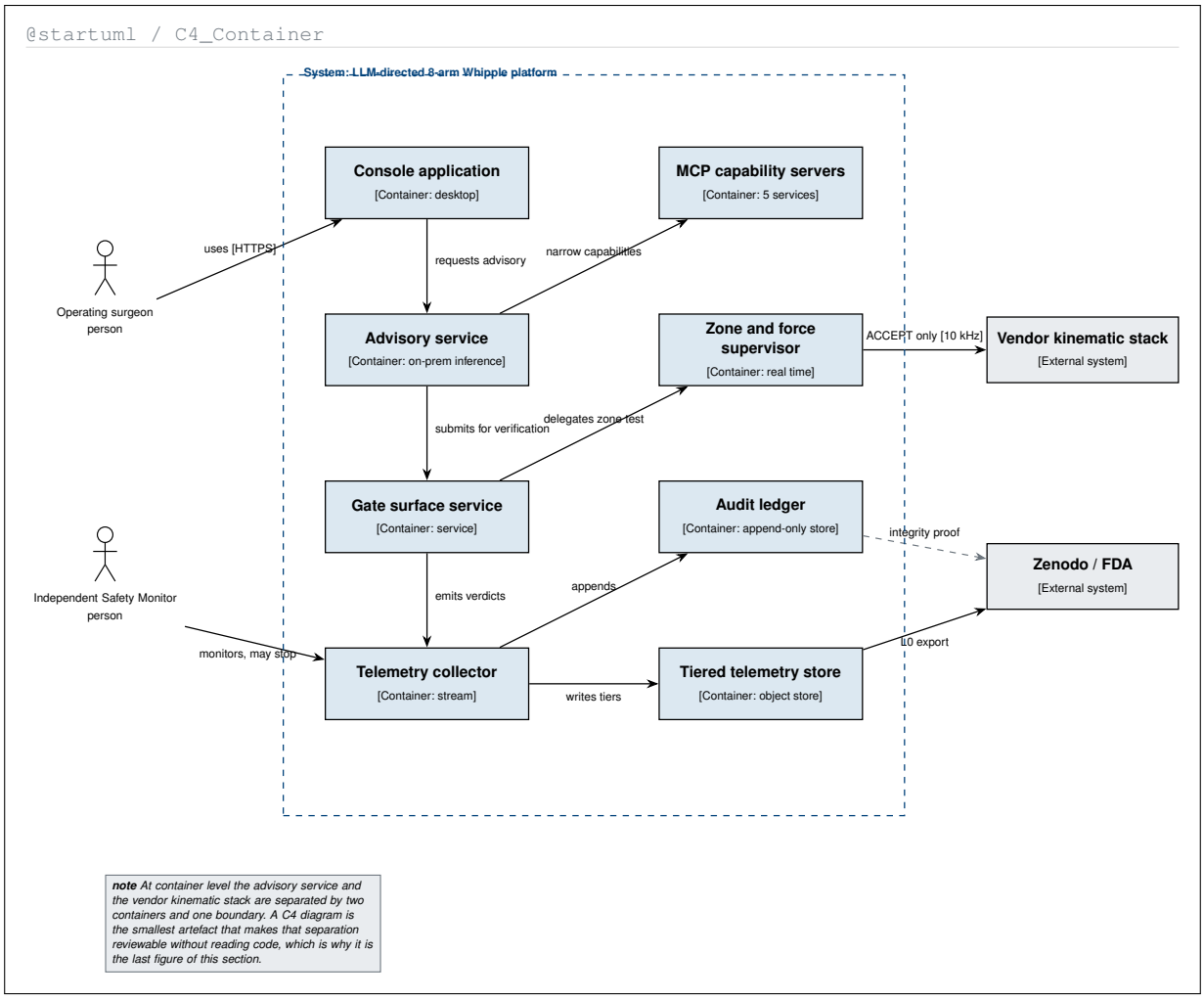


Figure 20. PlantUML-type C4 container diagram decomposing the system into the eight containers that run inside the boundary and the two external systems it reaches. At this level the advisory service and the vendor kinematic stack are separated by two containers and one boundary, which is the smallest artefact that makes the isolation reviewable without reading code.

4 D2-type Diagrams

D2 is a modern declarative diagram language whose distinctive strengths are nesting and tabulation: containers inside containers to arbitrary depth, a true `grid` layout with declared rows and columns, SQL-table and class shapes with typed rows, and `layers`, `scenarios`, and `steps` for telling one story across several panels [15]. A clinical protocol is full of structures that want exactly that treatment: schedules that are genuinely two-dimensional, record schemas with keys, organisational containment, and arguments that only make sense as a sequence of revealed states. The twenty figures below are referred to as **D2-type** figures.

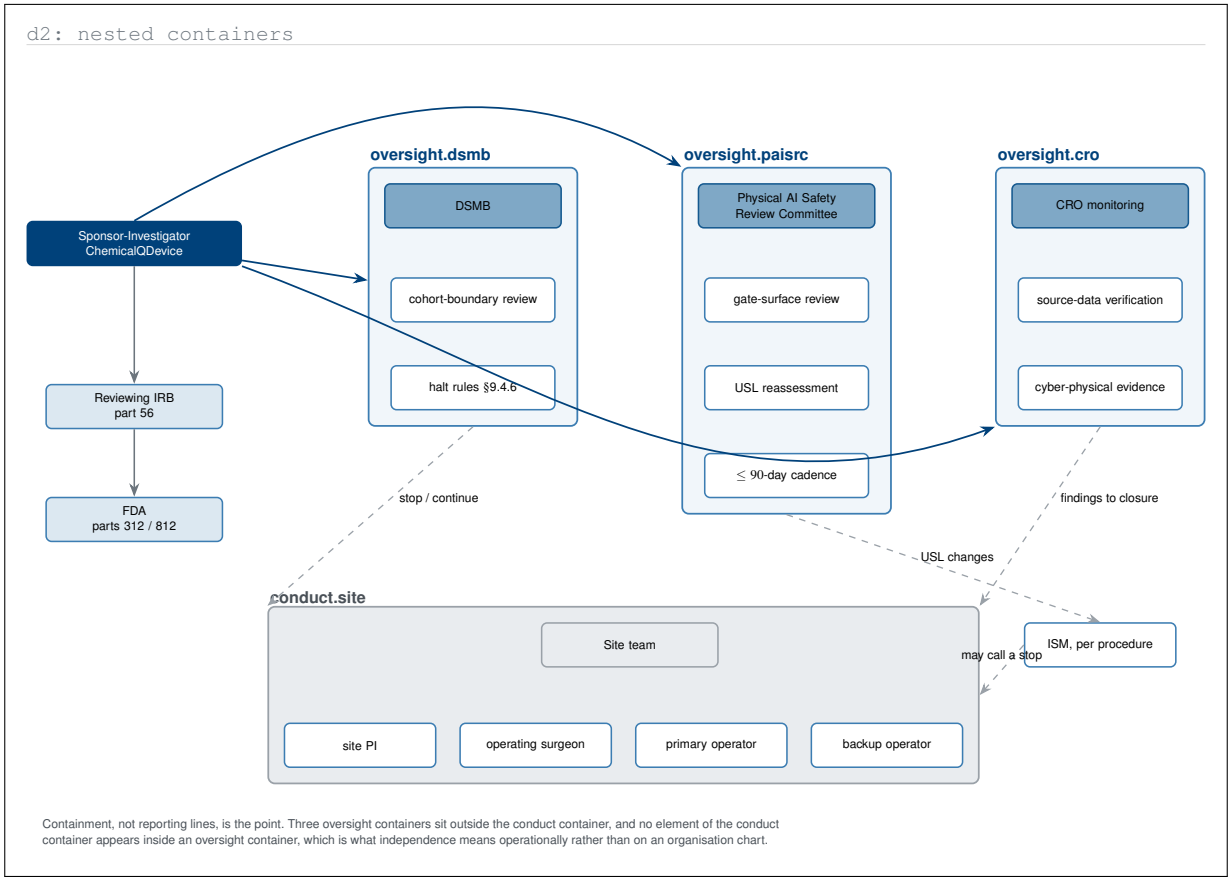


Figure 1. D2-type nested container map of the governance structure, drawn so that containment rather than reporting lines carries the meaning. No element of the conduct container appears inside an oversight container, which is what independence means operationally and is not visible on the organisation chart the parent protocol draws.

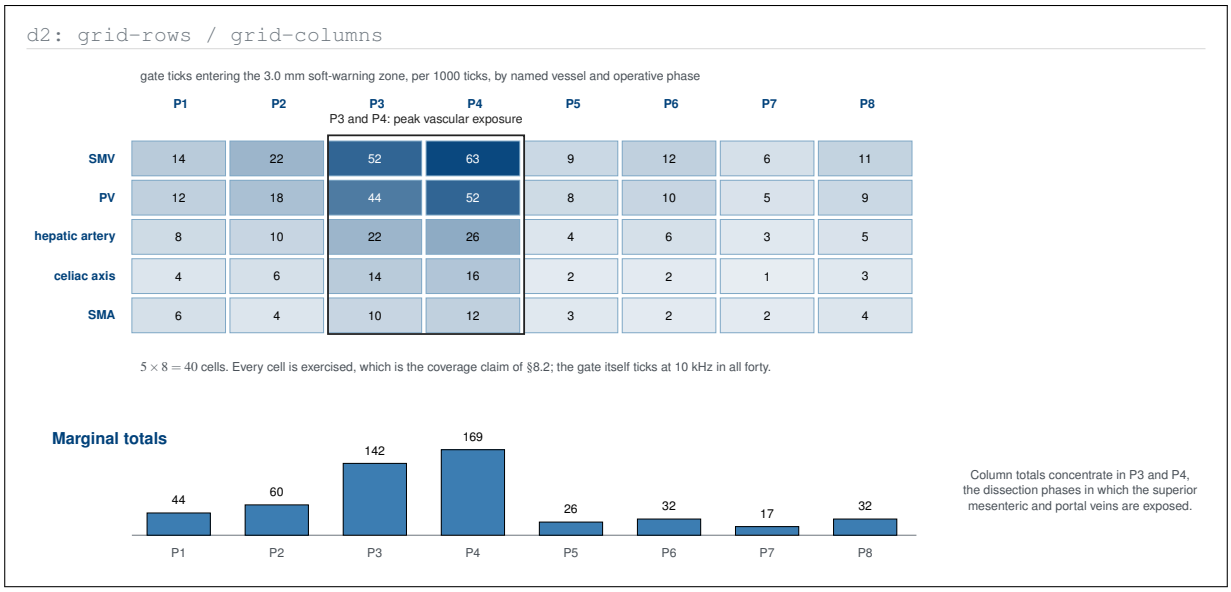


Figure 2. D2-type grid of the forty vessel-by-phase cells the vascular gate must cover, shaded by soft-warning rate and summed to column totals beneath. The parent protocol states that all five vessels are exercised across all eight phases; the grid shows where inside those forty cells the exposure actually concentrates.

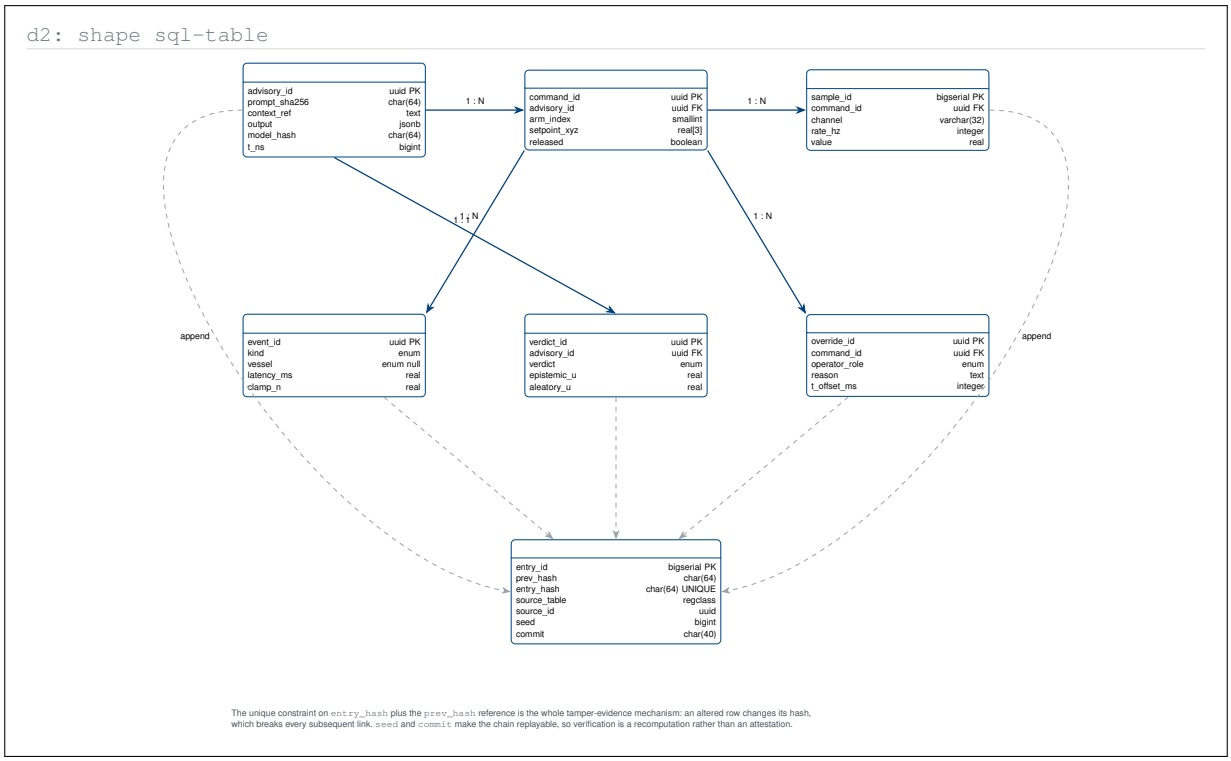


Figure 3. D2-type SQL-table view of the seven Physical AI record types with typed columns, primary keys, and foreign keys. Expressing the audit trail as a table with a unique hash column and a previous-hash reference shows why verification is a recomputation rather than an attestation, which is the property the protocol claims for it.

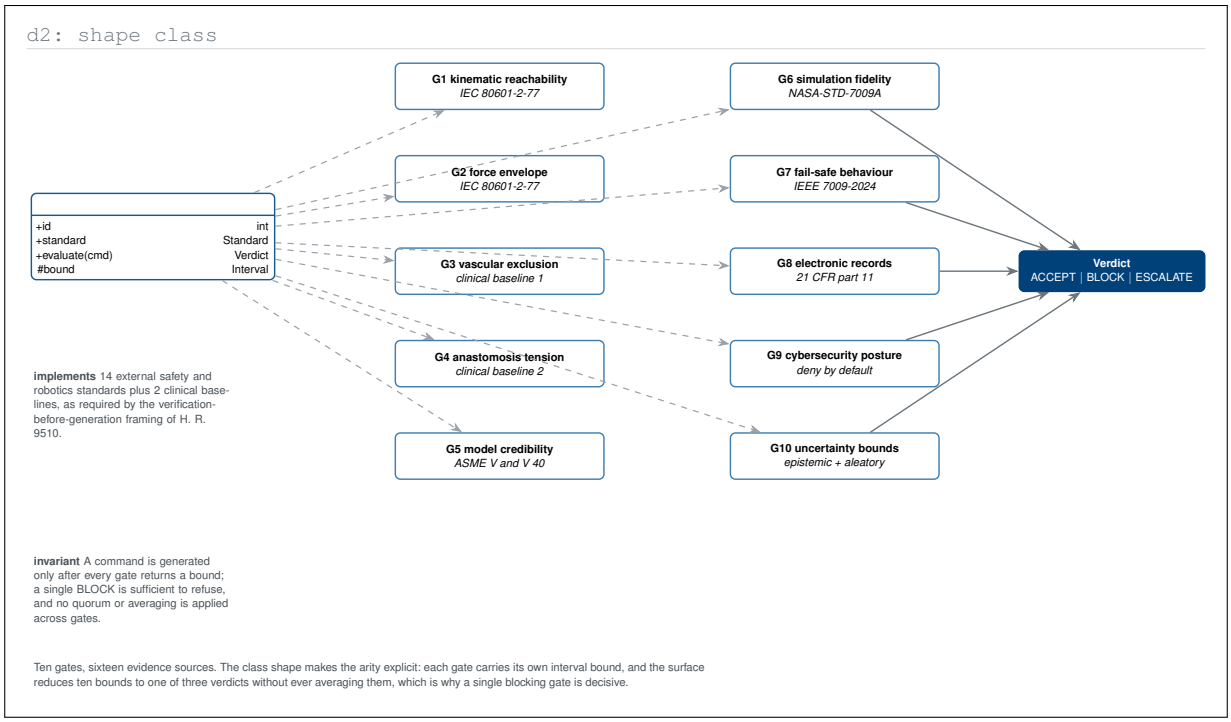


Figure 4. D2-type class-shape view of the ten-gate assurance suite, naming each gate and the external standard or clinical baseline it implements. The class invariant written beneath states the combination rule the parent protocol leaves implicit: ten bounds reduce to one verdict without averaging, so a single blocking gate is decisive.

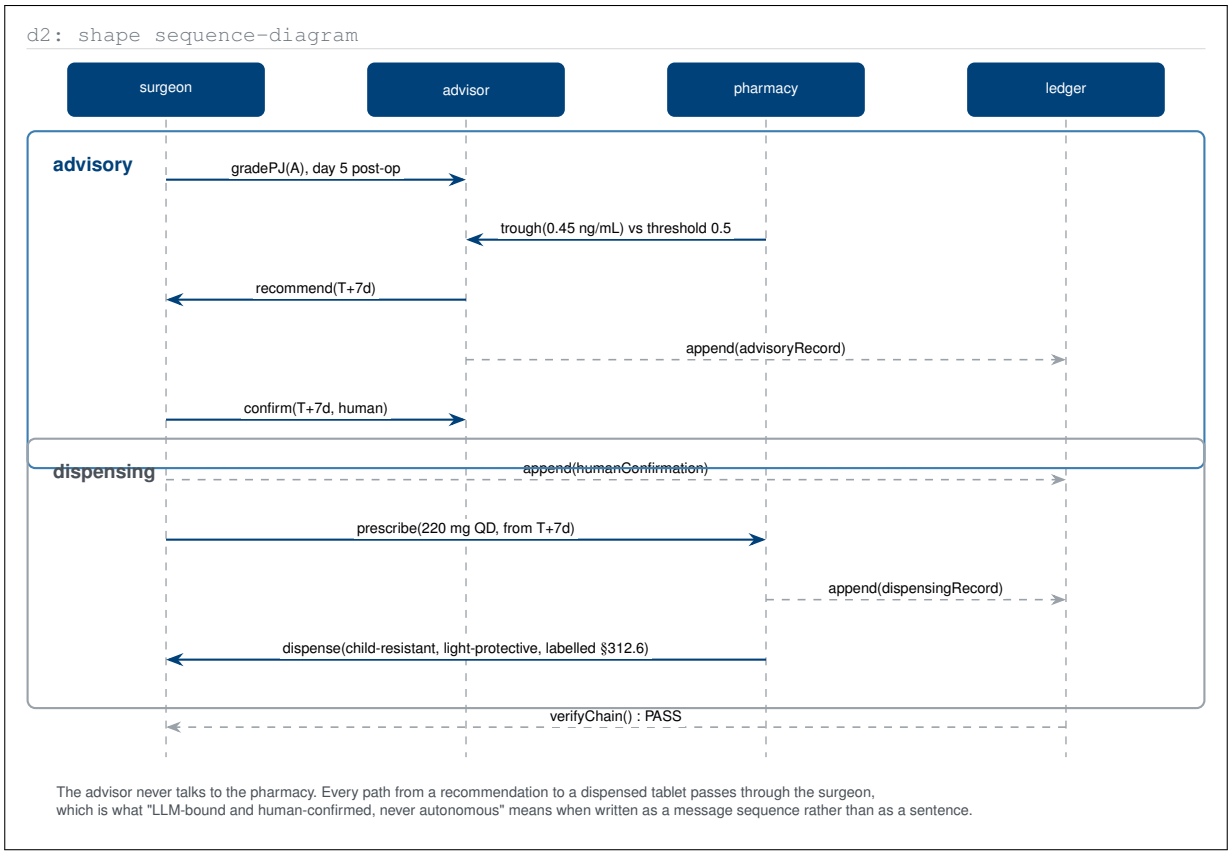


Figure 5. D2-type sequence diagram of the restart advisory as it is actually executed, with the advisory and dispensing blocks separated. Written as messages, the protocol's constraint becomes structural: the advisor has no edge to the pharmacy, so every path from a recommendation to a dispensed tablet passes through the surgeon.

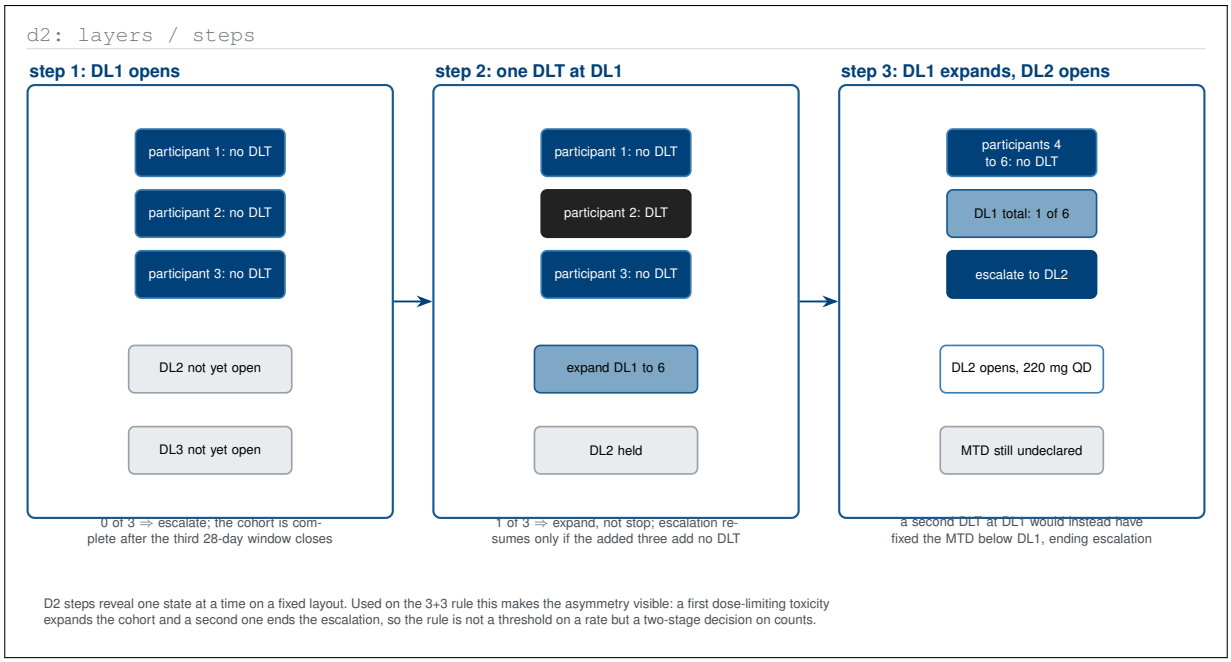


Figure 6. D2-type step sequence revealing the 3+3 escalation one cohort state at a time on a fixed layout. Stepping rather than summarising exposes the asymmetry in the rule: the first dose-limiting toxicity expands the cohort while the second ends the escalation, so it is a two-stage decision on counts and not a threshold on a rate.

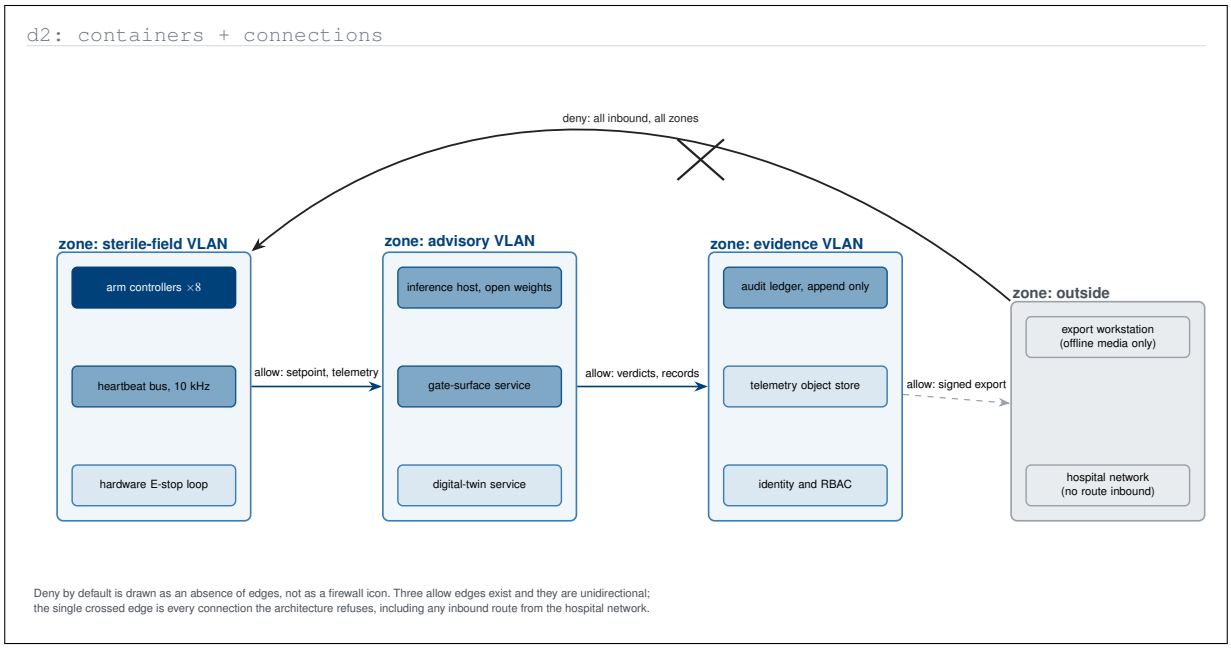


Figure 7. D2-type container view of the network zones with deny-by-default expressed as an absence of edges rather than as a firewall icon. Only three unidirectional allow edges exist, and the single crossed connection stands for every inbound route the architecture refuses, including any from the hospital network.

d2: grid, 14 x 7

	screen D −28 to −1	base D −1	surgery D 0	acute D 1 to 7	D 30	D 90	LT q12wk	visits
informed consent incl. Physical AI opt-out	X							1
staging and KRAS G12 confirmation	X							1
CA 19-9	X	X			X	X	X	5
ECOG performance status	X	X			X	X	X	5
safety laboratory panel	X	X	X	X	X	X	X	7
Phase 0 simulation sign-off		X						1
USL $\geq$ 7.0 + safety matrix		X						1
robotic Whipple surgery			X					1
intra-operative telemetry capture			X					1
ISGPS fistula grading				X	X			2
daraxonrasib restart advisory				X				1
AE / SAE review	X	X	X	X	X	X	X	7
RECIST imaging	X					X	X	3
PFS / OS assessment						X	X	2
assessments at this visit	7	6	4	5	6	7	6	

Row totals separate the schedule into a spine and a set of gates. Two rows occur at every visit (safety laboratory panel and adverse-event review); six rows occur exactly once and each of those is a gate: consent, staging, the Phase 0 sign-off, the USL check, the operation, and the restart advisory.

Figure 8. D2-type grid rendering of the Schedule of Activities with both marginal totals computed. Read by row the schedule splits into a two-item continuity spine and six single-occurrence gates; read by column it shows that the day 90 visit, not the day of surgery, is the densest assessment point in the study.

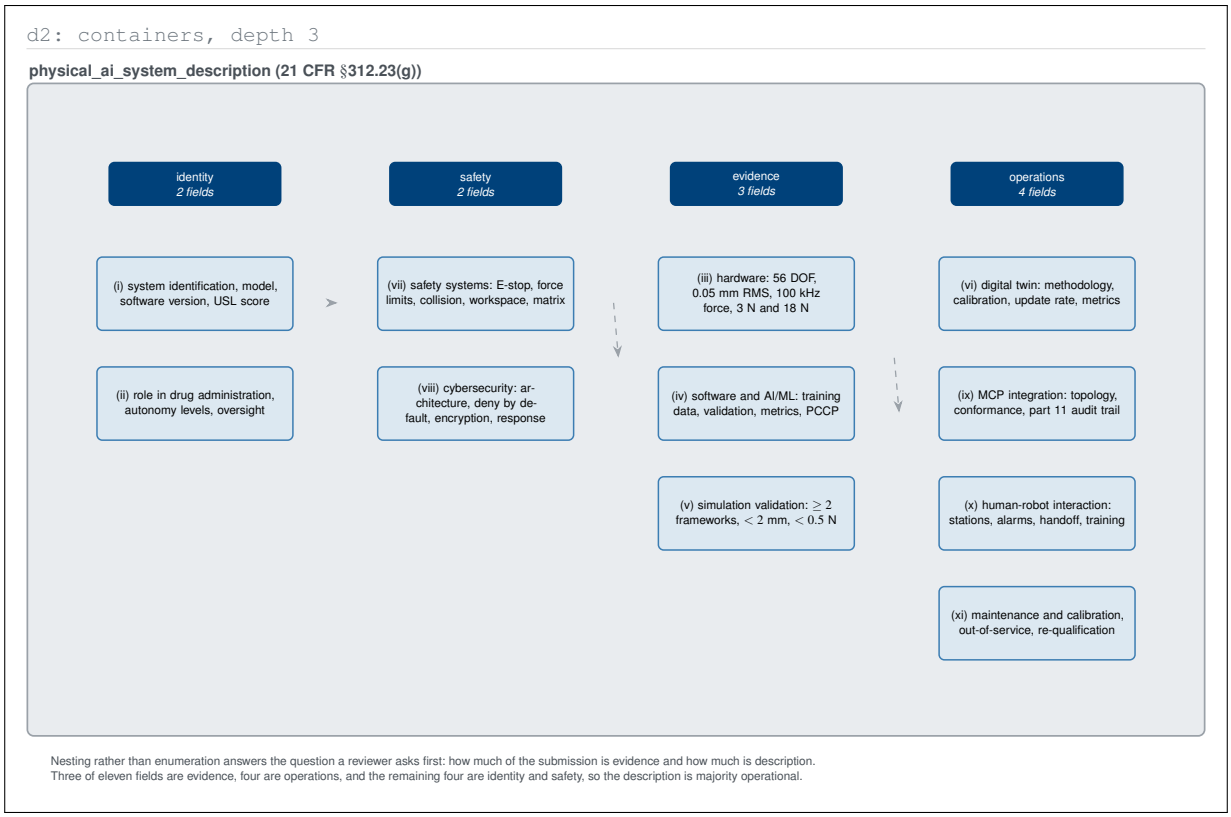


Figure 9. D2-type nested container view of the eleven System Description fields, grouped three levels deep instead of enumerated. The nesting answers a question the roman-numeral list cannot: how the submission divides between description, evidence, and continuing operational obligation, and it turns out to be majority operational.

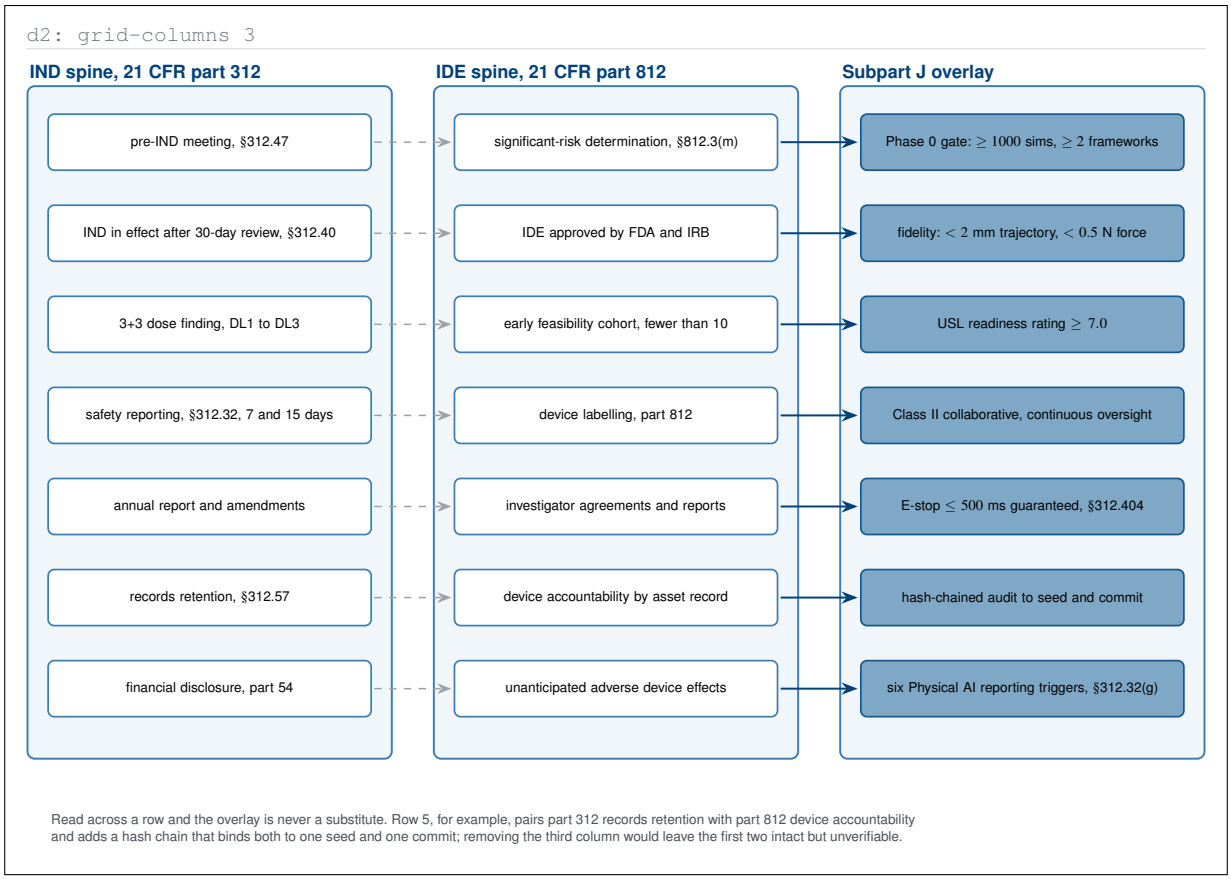


Figure 10. D2-type three-column grid aligning the drug spine, the device spine, and the Subpart J overlay row by row. Reading across shows what the compliance statement means in practice: the overlay adds a verification obligation to each row without replacing anything, so removing it would leave the first two columns intact but unverifiable.

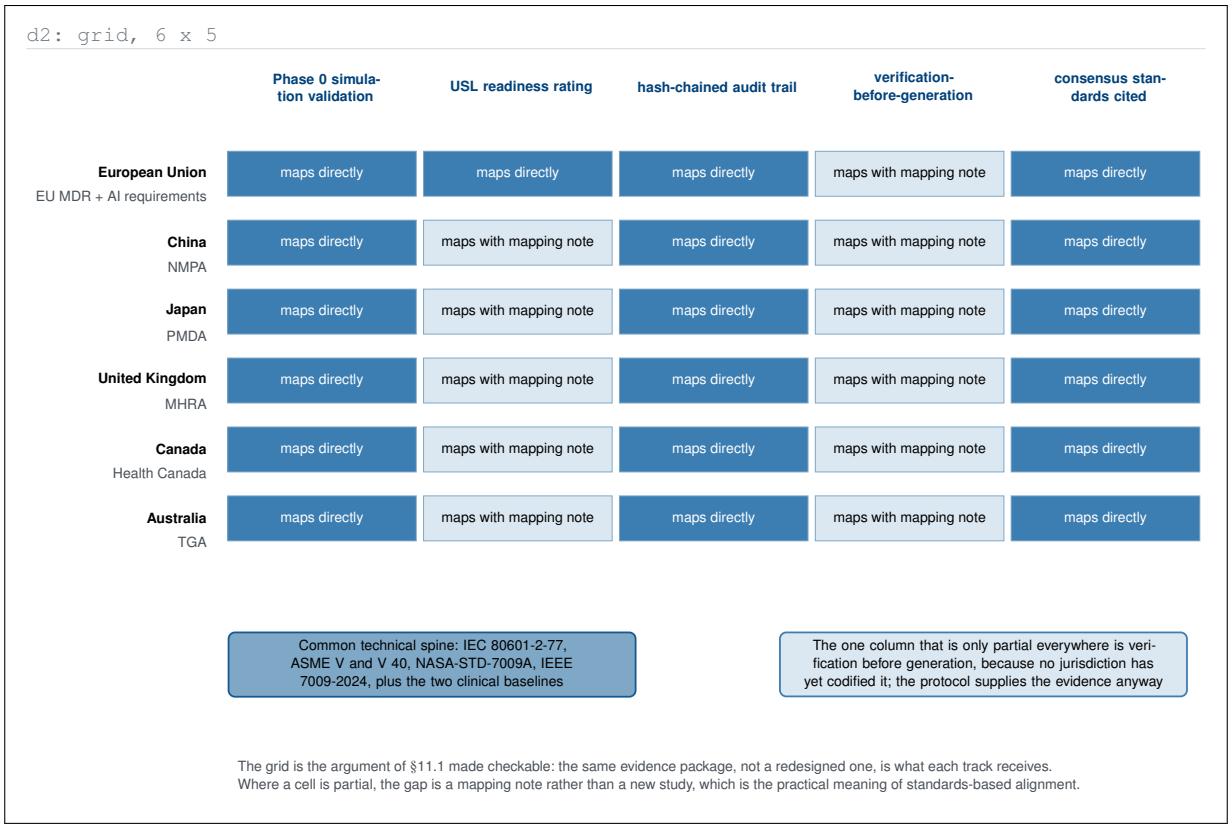


Figure 11. D2-type grid crossing the six named jurisdictions with the five evidence artefacts the protocol produces. Every cell is either a direct mapping or a mapping note, never a new study, which is the practical content of the cross-jurisdictional claim, and it isolates the single column no regulator has yet codified.

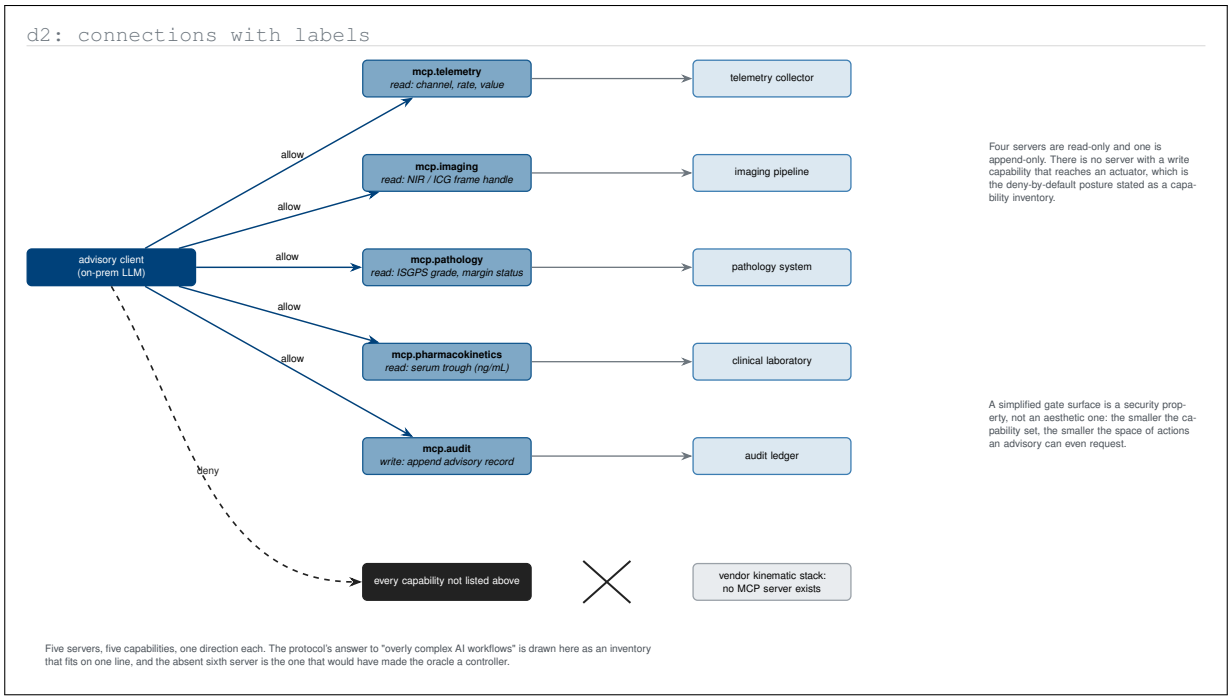


Figure 12. D2-type connection map of the five Model Context Protocol servers with the one capability each exposes. Four are read-only and one is append-only, and the absent sixth server, one with a write capability reaching an actuator, is what would have turned the advisory oracle into a controller.

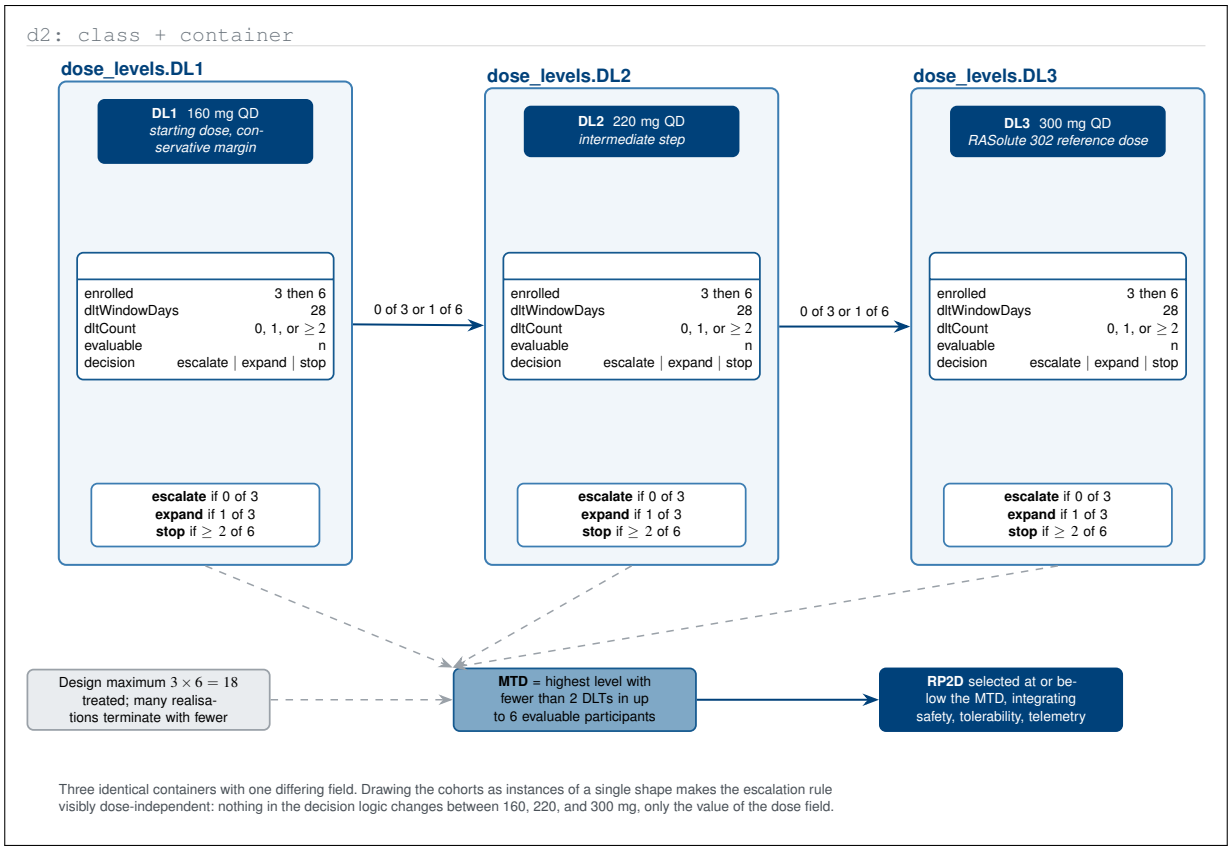


Figure 13. D2-type container-plus-class view of the three dose levels as three instances of one shape. Because only the dose field differs between the containers, the figure makes visible that the escalation logic is entirely dose-independent, and that the design ceiling of eighteen is an upper bound many realisations never reach.

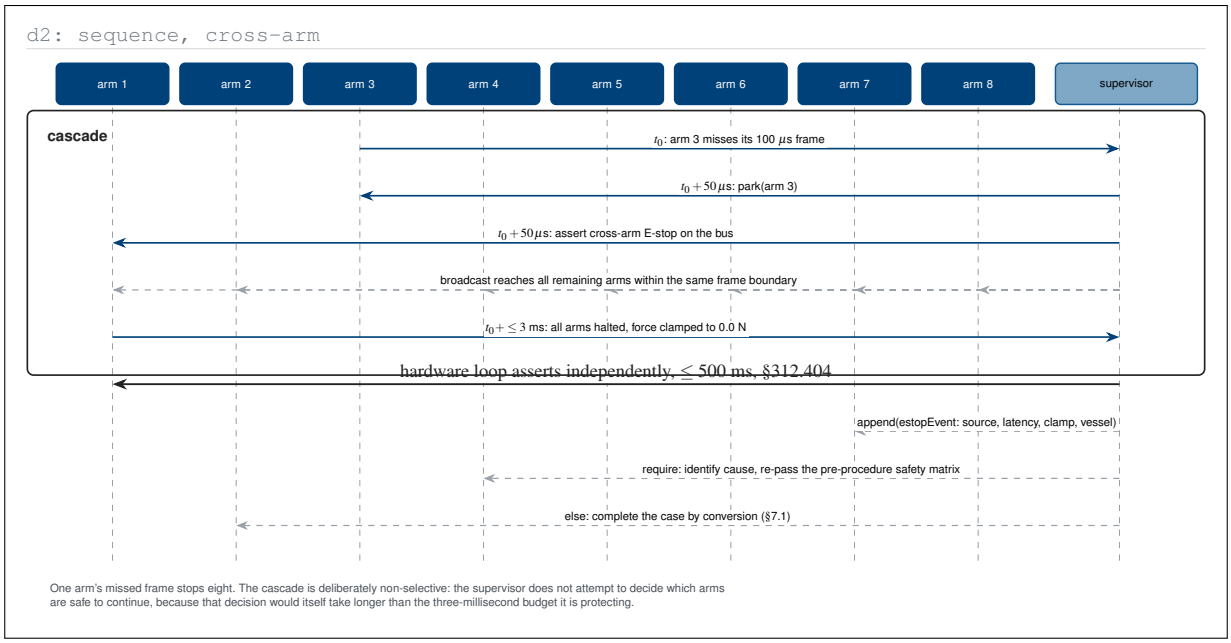


Figure 14. D2-type sequence diagram of the emergency-stop cascade across all eight arms after a single missed heartbeat frame. Drawing every lifeline shows the design choice the latency budget forces: the cascade is deliberately non-selective, because deciding which arms could safely continue would take longer than the three millisecond halt it protects.

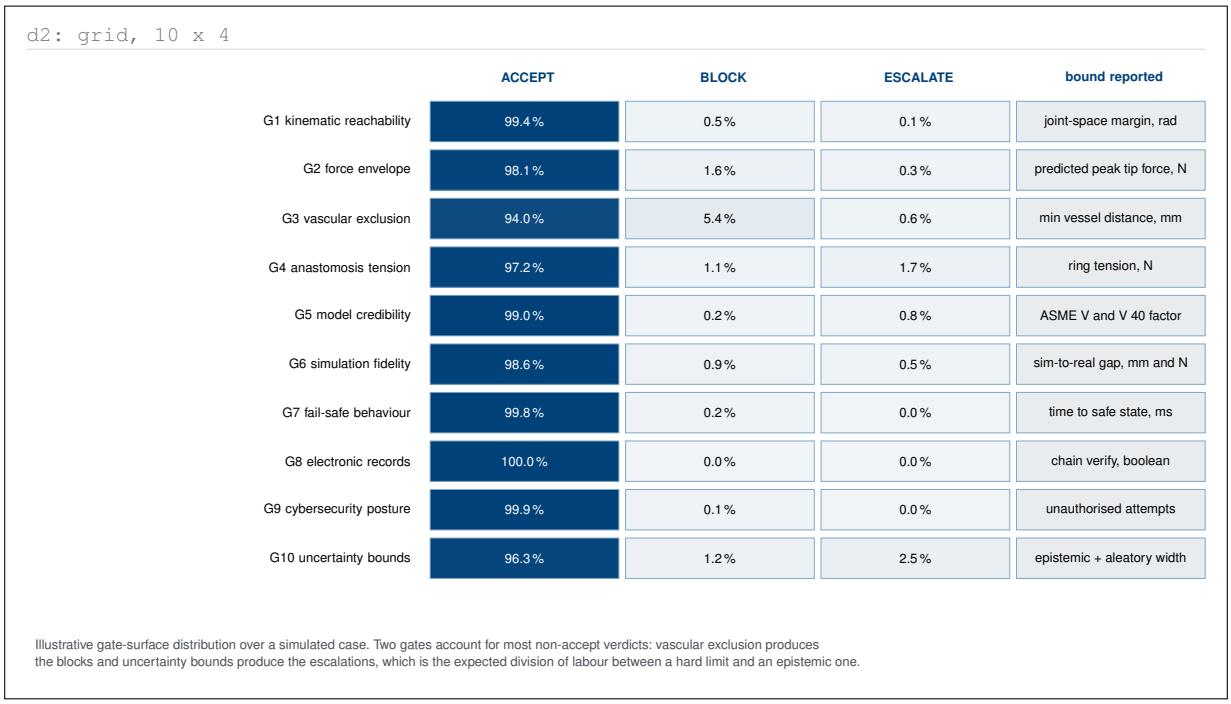


Figure 15. D2-type grid crossing the ten assurance gates with the three verdicts and the bound each gate reports. The distribution separates the two kinds of gate: vascular exclusion, a hard geometric limit, produces the blocks, while uncertainty bounds, an epistemic limit, produces the escalations that hand control back to the operator.

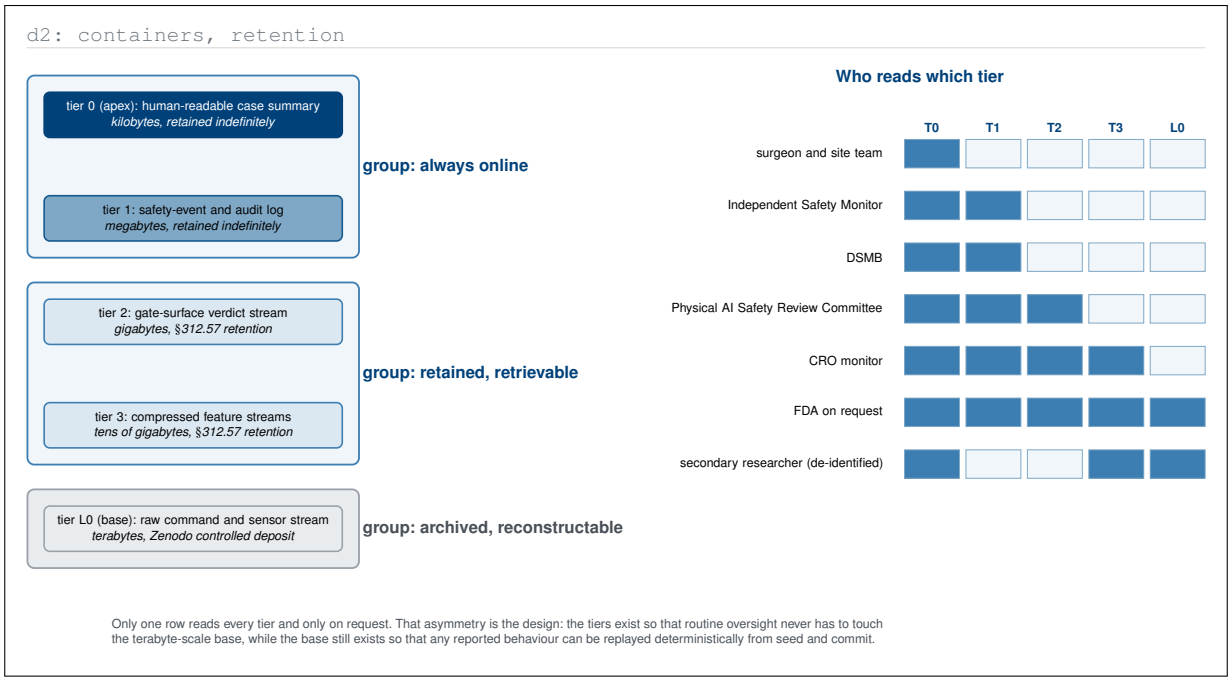


Figure 16. D2-type container view of the tiered telemetry record with an access matrix beside it. Only one reader touches every tier and only on request, which is the asymmetry the tiering exists to create: routine oversight never reaches the terabyte-scale base, yet the base remains available for deterministic replay.

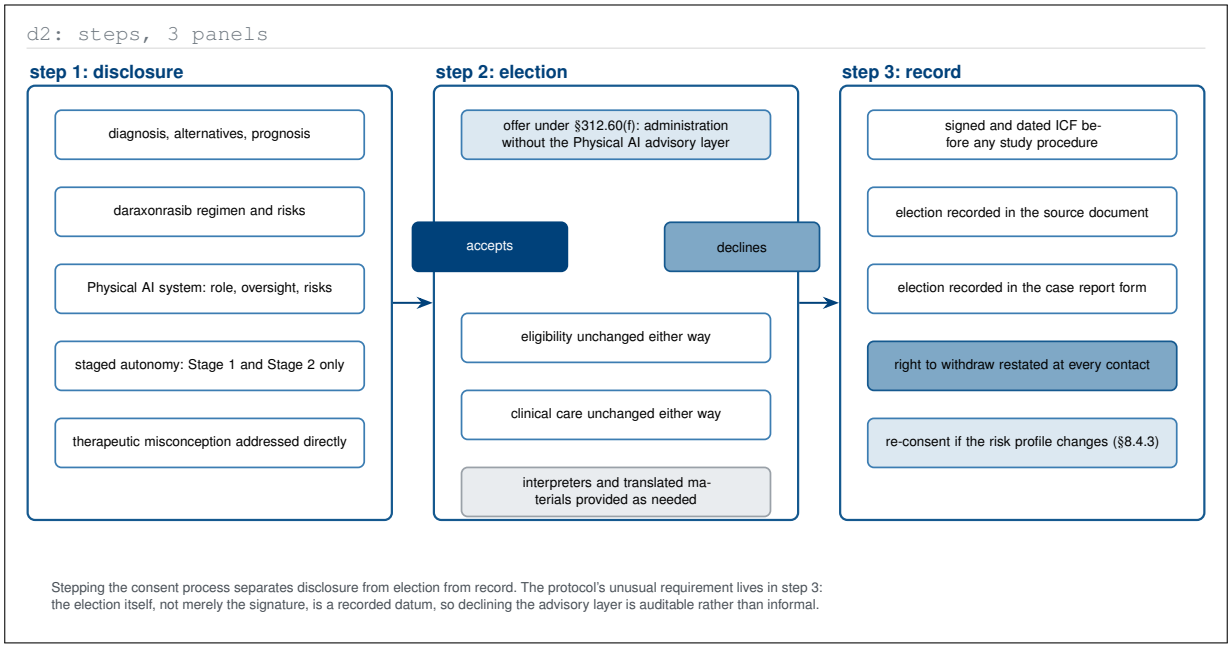


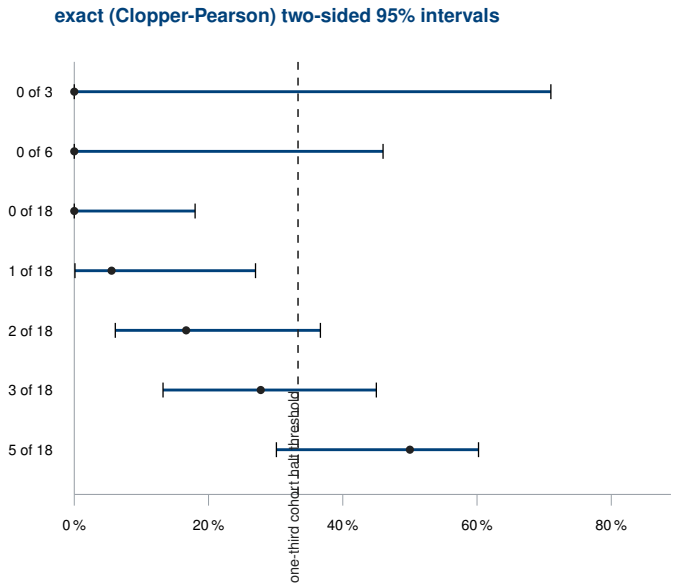
Figure 17. D2-type three-panel step sequence separating consent disclosure, election, and record. The protocol's distinctive requirement lives in the third panel: the participant's election of or declination from the Physical AI advisory layer is itself a recorded datum in both the source document and the case report form, so declining is auditable rather than informal.

d2: sql-table + plot column

quantity	planned
cohort size	3, expand to 6
per-dose maximum	6
dose levels	3 (DL1 to DL3)
total treated	up to 18
significance level	0.05 two-sided
multiplicity adjustment	none

Zero events in 18 still leaves an upper bound near 18%, so acceptability is judged by whether the interval excludes the prespecified unacceptable rate, not by a test.

The first row whose interval clears zero entirely is 1 of 18; the first row whose interval reaches the halt threshold is 5 of 18, which is why halting is decided per cohort rather than per study.



A D2 table and a plotted column in one frame. Reading the two together is what the parent Table on operating characteristics asks a reader to do mentally: hold the design constants on the left while judging, on the right, how much of the parameter space eighteen participants can actually exclude.

Figure 18. D2-type figure pairing the design constants as a typed table with the exact confidence intervals plotted beside them. Holding the two together is what the parent operating-characteristics table asks a reader to do mentally, and it locates the two thresholds that matter: the first count whose interval clears zero, and the first that reaches the cohort halt rule.

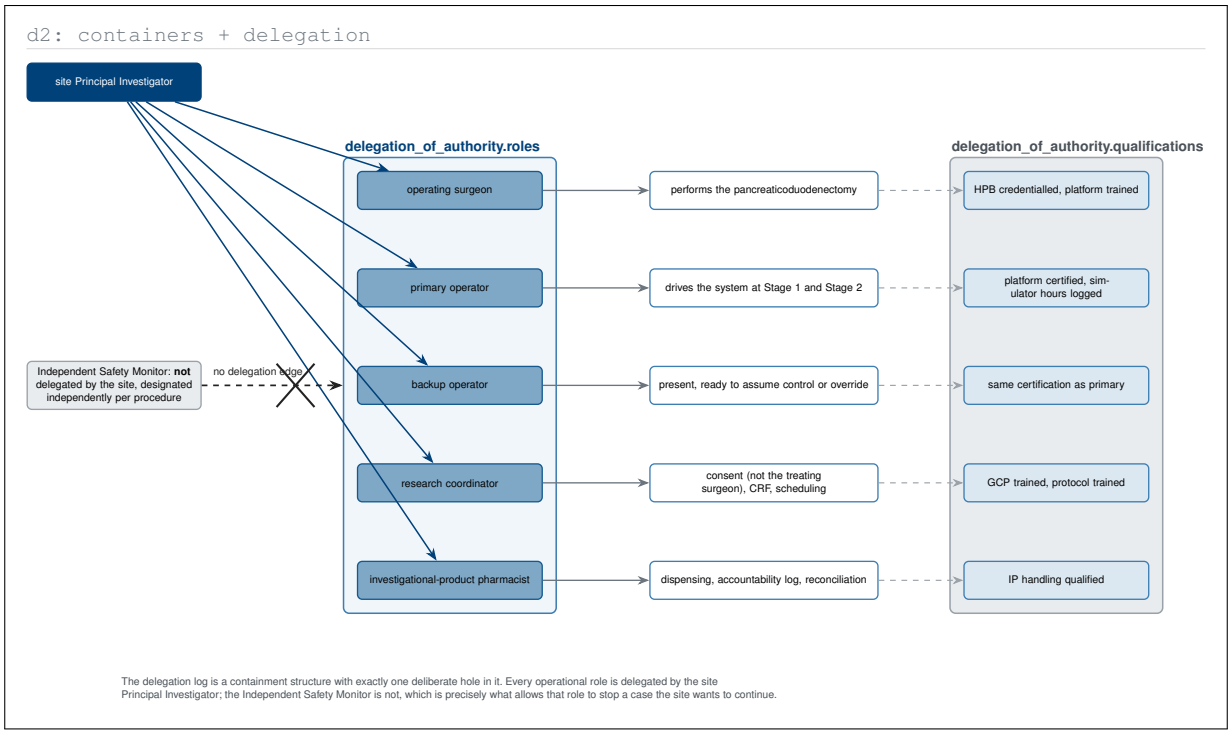


Figure 19. D2-type delegation-of-authority view with roles, delegated duties, and required qualifications in three containers. The figure is built around one deliberate hole: every operational role descends from the site Principal Investigator except the Independent Safety Monitor, which is what allows that role to stop a case the site wants to continue.

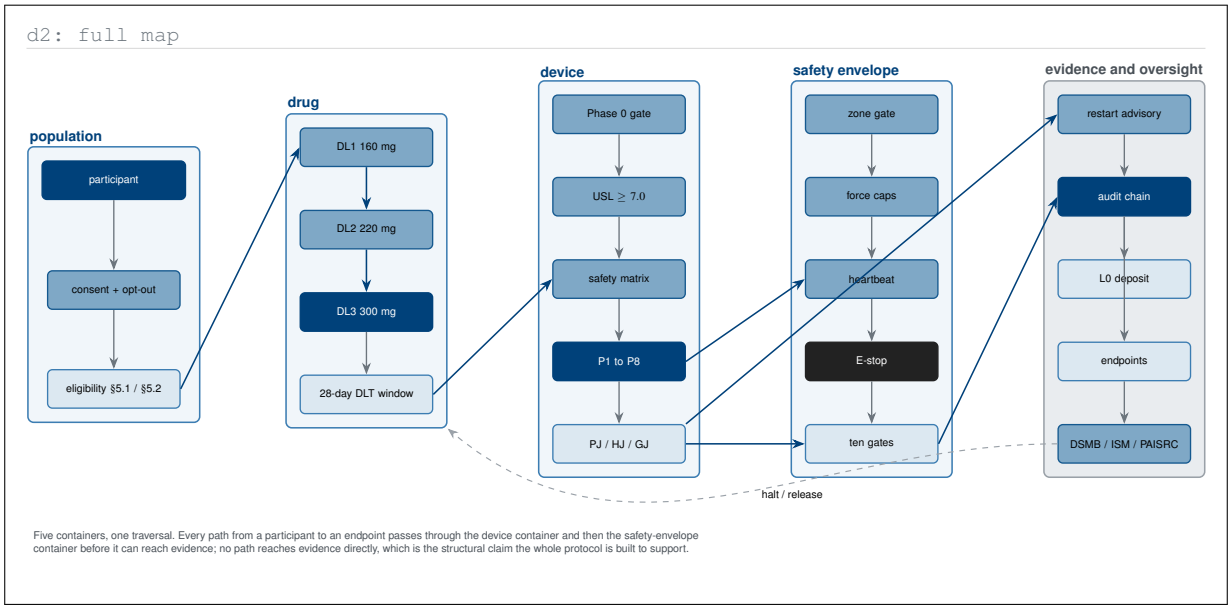


Figure 20. D2-type full map of the protocol as five containers traversed in one direction, from population through drug and device to safety envelope and finally to evidence and oversight. The single structural claim it makes is that no path reaches the evidence container without first passing through the safety envelope, with the only reverse edge being the halt authority.

5 Diagrams (Python)-type Diagrams

The `diagrams` library draws infrastructure as code: an icon per node, a dashed titled cluster per grouping, and an edge per real dependency [16]. Its strength is that it forces an architecture to be described in terms of things that are actually deployed, powered, and paid for, rather than in terms of conceptual boxes. A first-in-human Physical AI protocol has a substantial amount of that reality in it, and the parent publication carries almost none of it. The twenty figures below reconstruct the deployment, the pipelines, the segmentation, the failover, the lifecycle, and the publication path implied by the protocol text, and are referred to as **Diagrams (Python)-type** figures. Every icon is a vector pictogram drawn in TikZ, so nothing here is a raster asset.

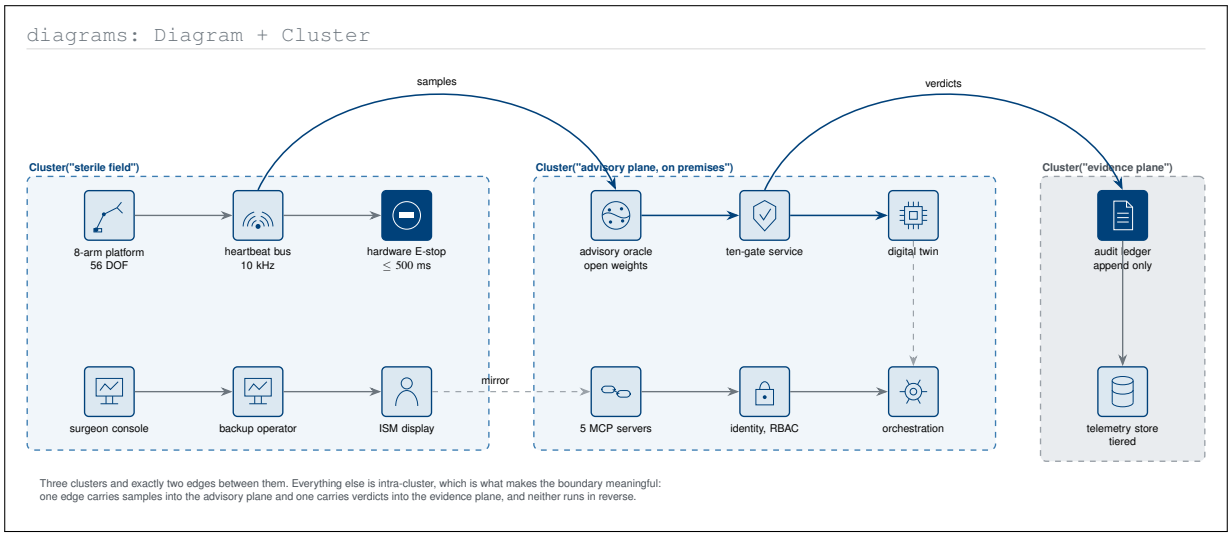


Figure 1. Diagrams (Python)-type reference deployment of the whole system, drawn as three clusters with icons for the things actually installed. Only two edges cross cluster boundaries, one carrying samples inward and one carrying verdicts outward, and neither runs in reverse, which is the deployment invariant the parent architecture section describes in prose.

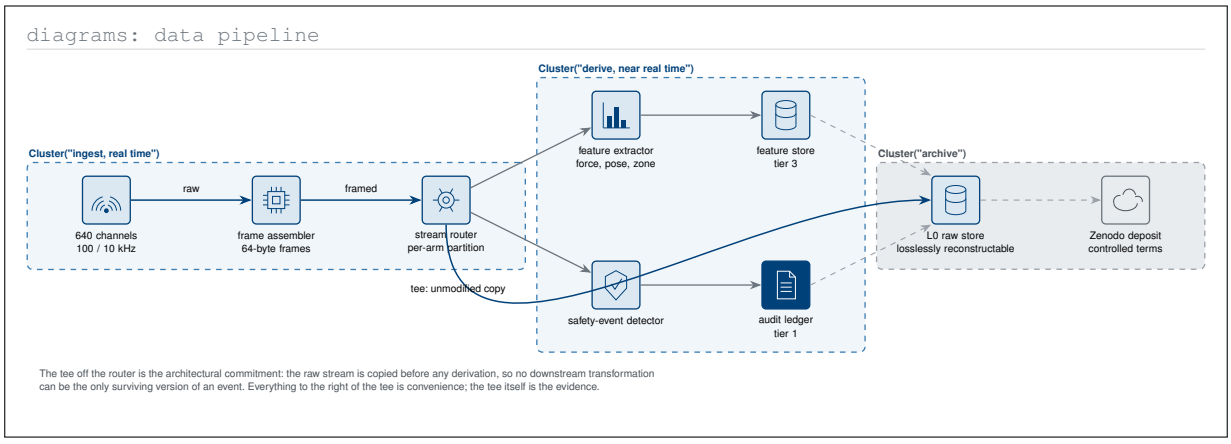


Figure 2. Diagrams (Python)-type data pipeline from the 640-channel sensor stack to the controlled Zenodo deposit. The architectural commitment is the tee immediately after the router: the raw stream is copied before any derivation, so no downstream transformation can ever be the only surviving version of an event.

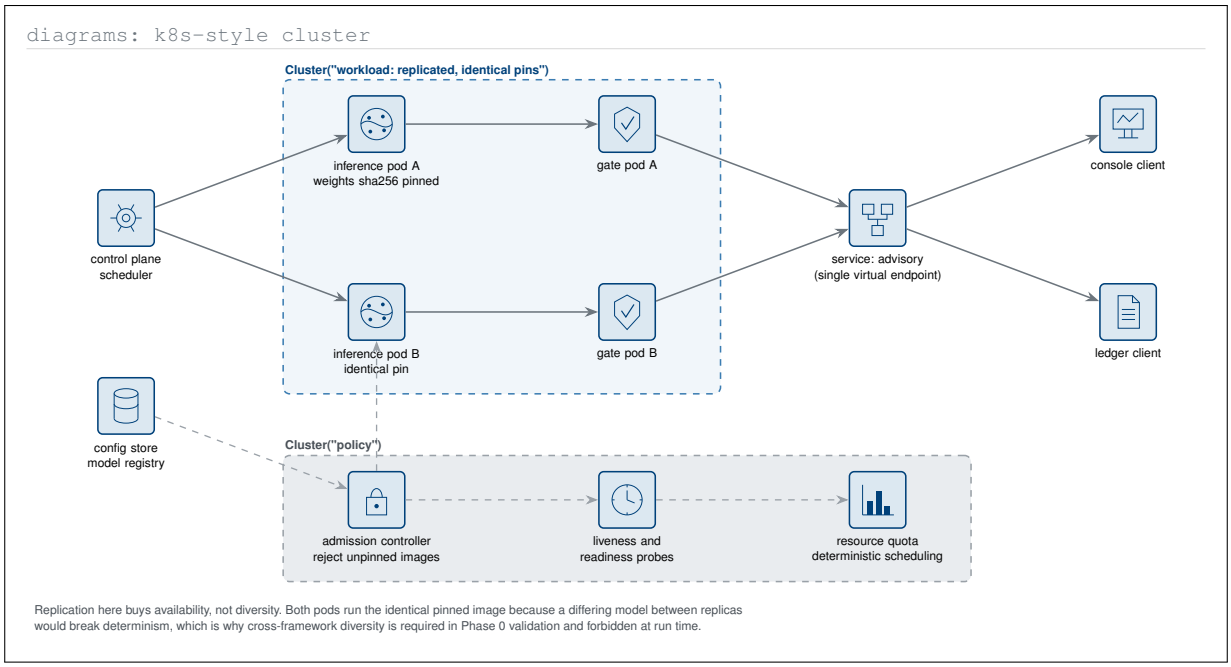


Figure 3. Diagrams (Python)-type view of the inference cluster with replicated pods behind a single advisory endpoint. It records an easily missed distinction: run time replication buys availability with identical pinned images, whereas the diversity requirement of the Phase 0 gate lives in validation, where differing frameworks are the point rather than a determinism hazard.

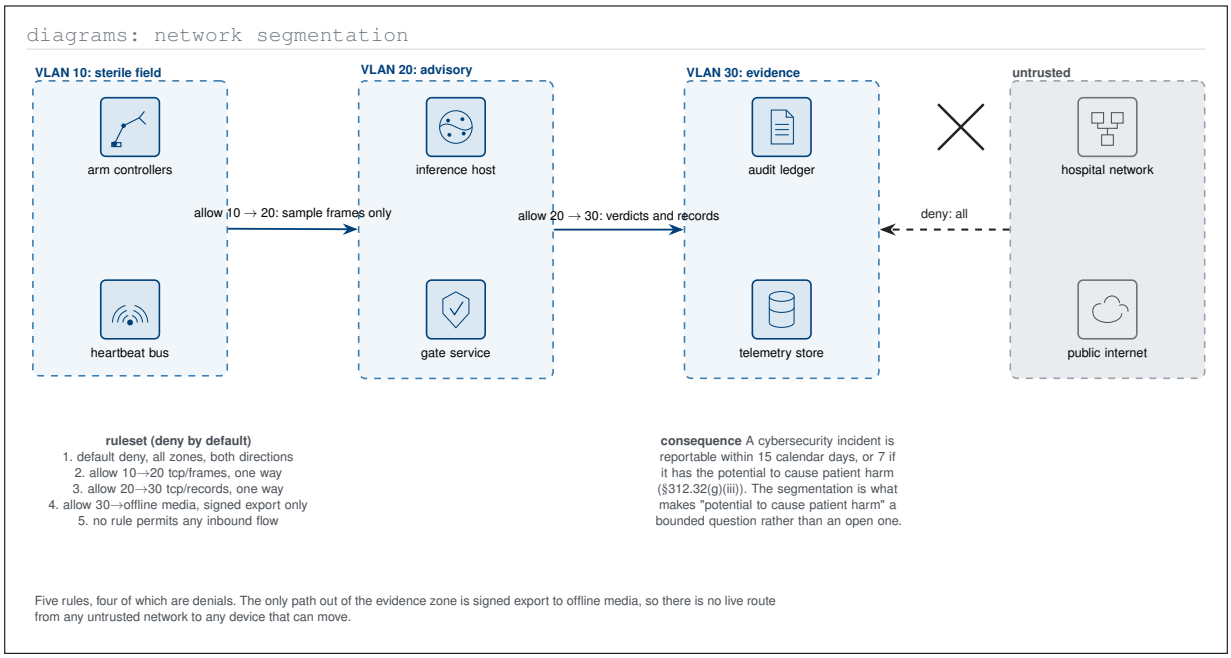


Figure 4. Diagrams (Python)-type network segmentation with the deny-by-default ruleset written out beside it. Four of the five rules are denials and the only outbound path is signed export to offline media, which bounds the question a cybersecurity report has to answer: no live route exists from an untrusted network to anything that can move.

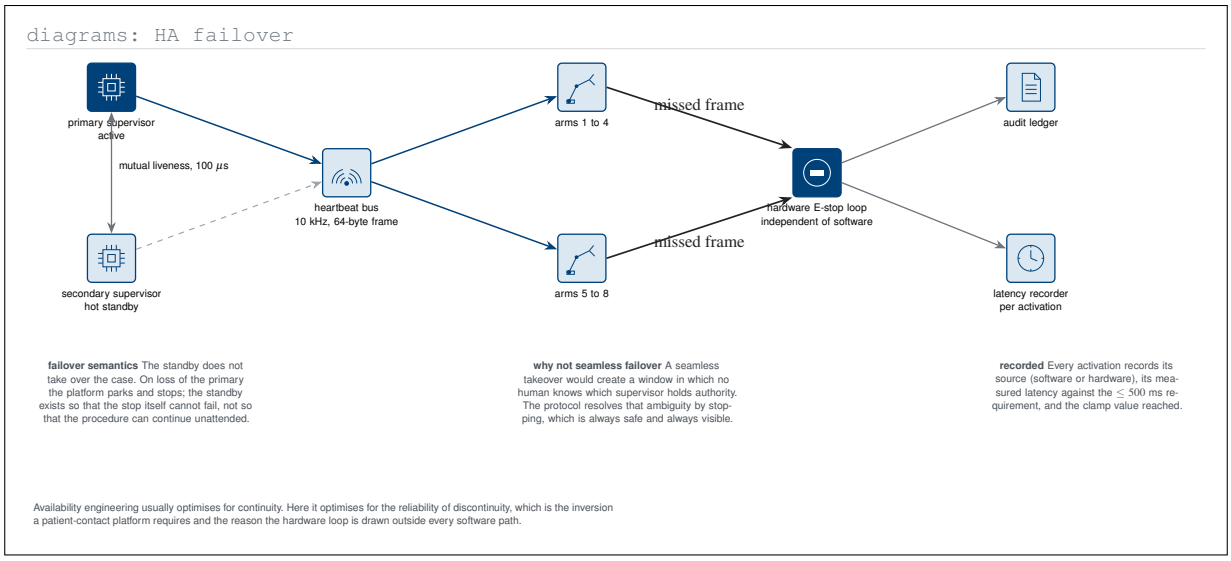


Figure 5. Diagrams (Python)-type high-availability view in which redundancy protects the stop rather than the procedure. On loss of the primary supervisor the platform parks and halts instead of failing over seamlessly, because a seamless takeover would create a window in which no human knows which supervisor holds authority.

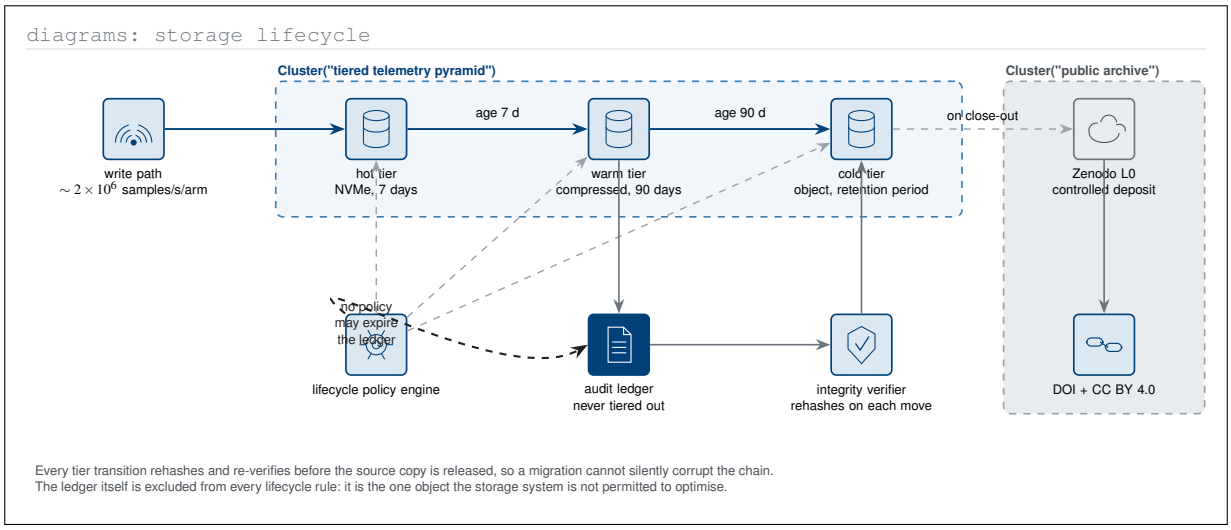


Figure 6. Diagrams (Python)-type storage lifecycle showing the tier transitions, the integrity verification that gates each one, and the single object excluded from every lifecycle rule. The audit ledger is what the storage system is not permitted to optimise, which is the operational form of the protocol's tamper-evidence claim.

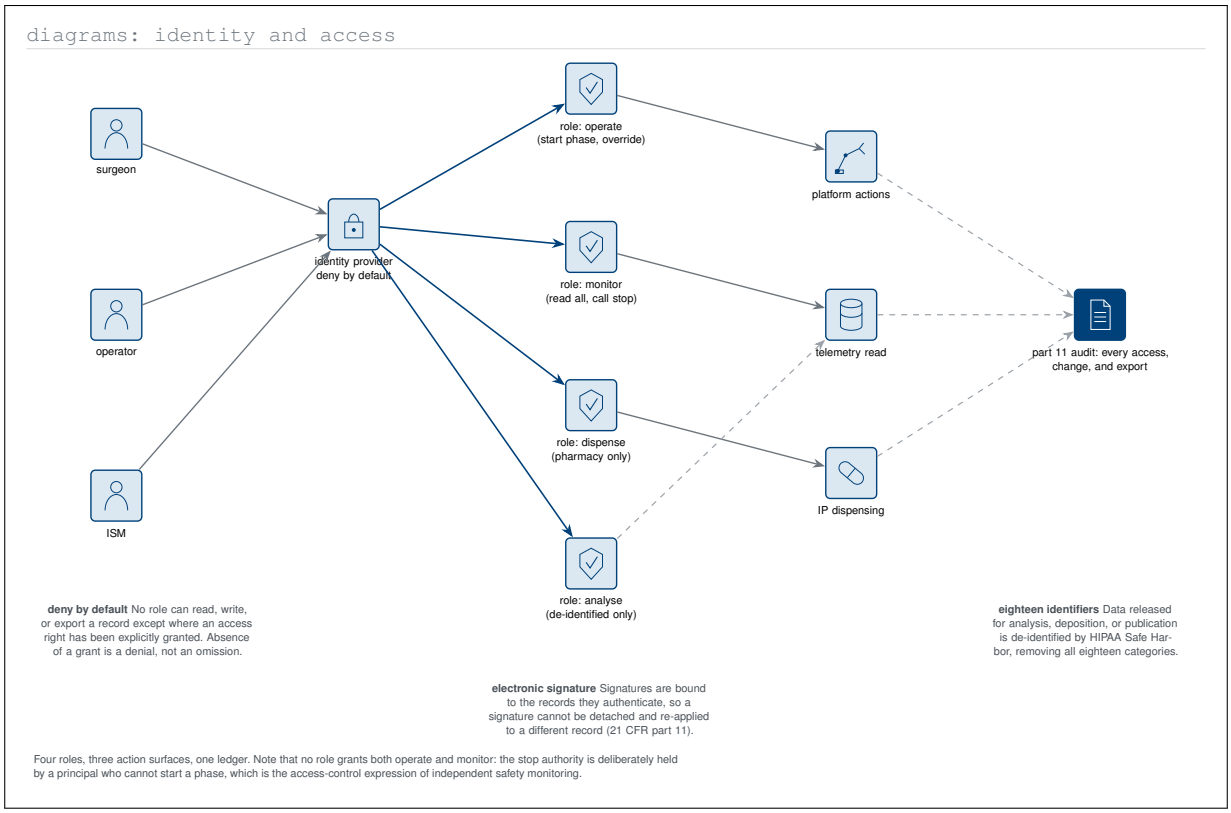


Figure 7. Diagrams (Python)-type identity and access architecture with four roles and three action surfaces under a deny-by-default provider. The access matrix encodes independent safety monitoring directly: no role grants both operate and monitor, so the principal who can stop a case is one who cannot start a phase.

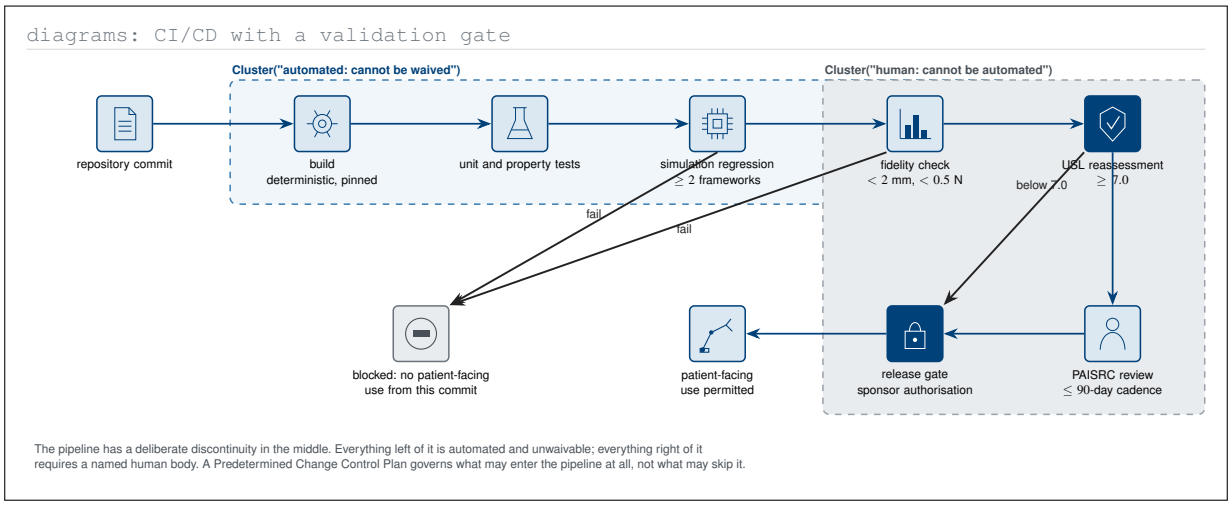


Figure 8. Diagrams (Python)-type continuous-integration pipeline for the locked model, with the Predetermined Change Control Plan expressed as a two-cluster discontinuity. The left cluster is automated and unwaivable; the right cluster requires named human bodies, and no commit reaches patient-facing use without traversing both.

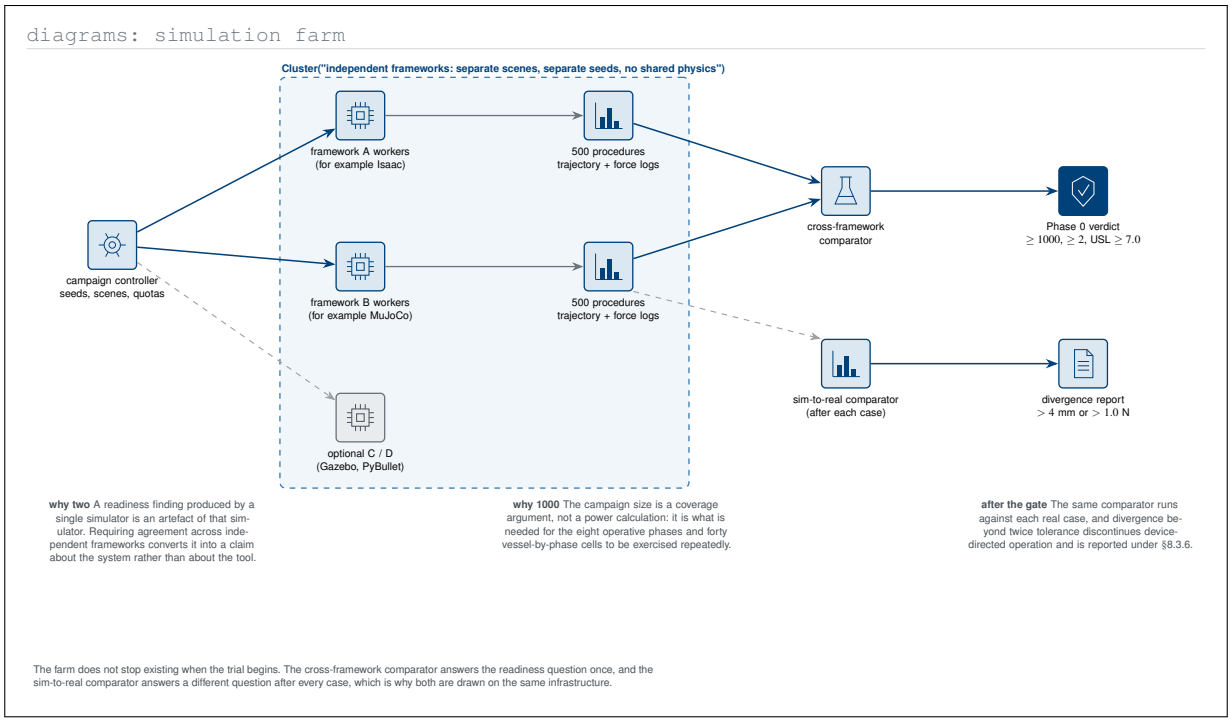


Figure 9. Diagrams (Python)-type simulation farm serving both the Phase 0 readiness question and the per-case sim-to-real question. Drawing both comparators on one infrastructure makes clear that the farm does not stop existing when the trial begins: one comparator answers a question once, the other answers a different question after every case.

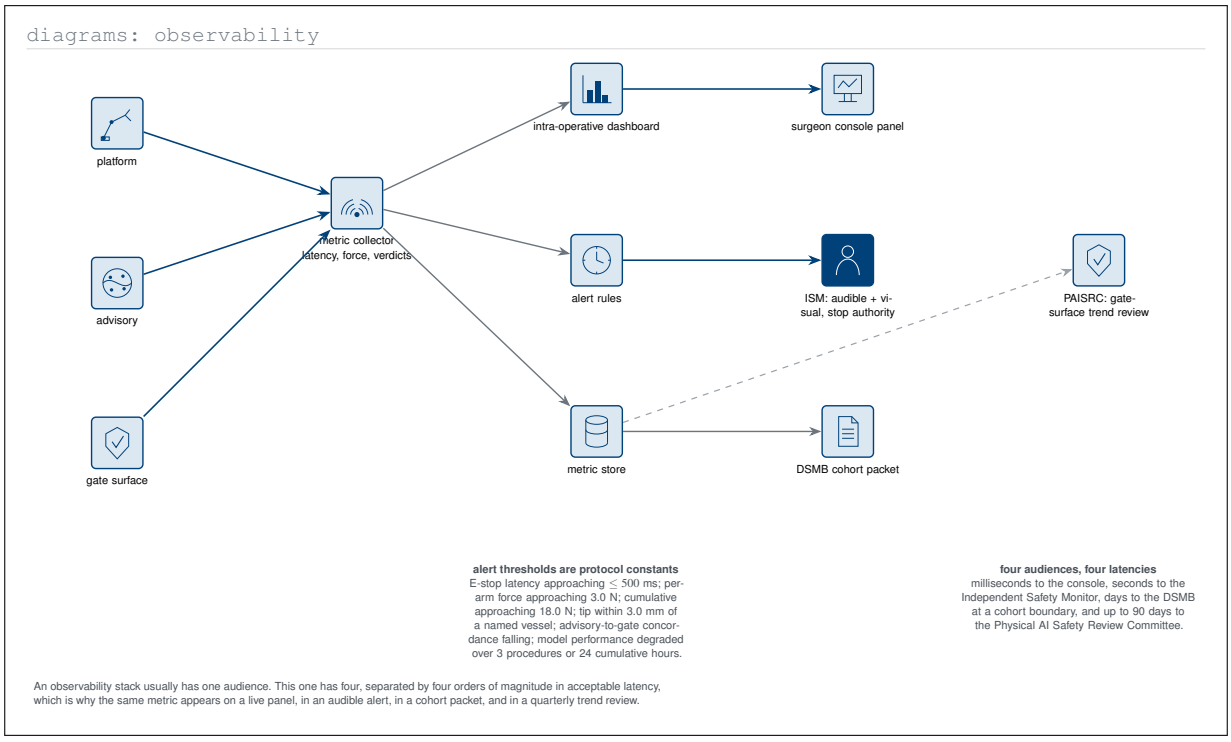


Figure 10. Diagrams (Python)-type observability architecture in which one metric stream serves four audiences at four different latencies. Separating the console, the Independent Safety Monitor, the data safety monitoring board, and the Physical AI Safety Review Committee by acceptable delay explains why the same measurement appears in four different artefacts.

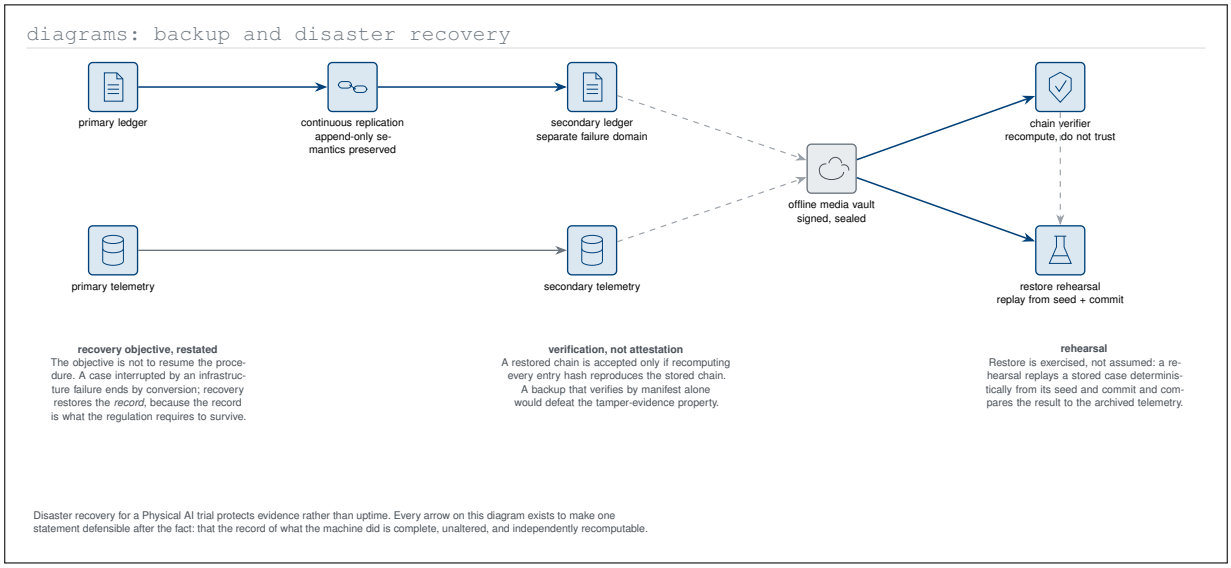


Figure 11. Diagrams (Python)-type backup and recovery architecture whose objective is evidence rather than uptime. A case interrupted by infrastructure failure ends by conversion, so recovery restores the record, and a restored chain is accepted only when recomputing every hash reproduces it.

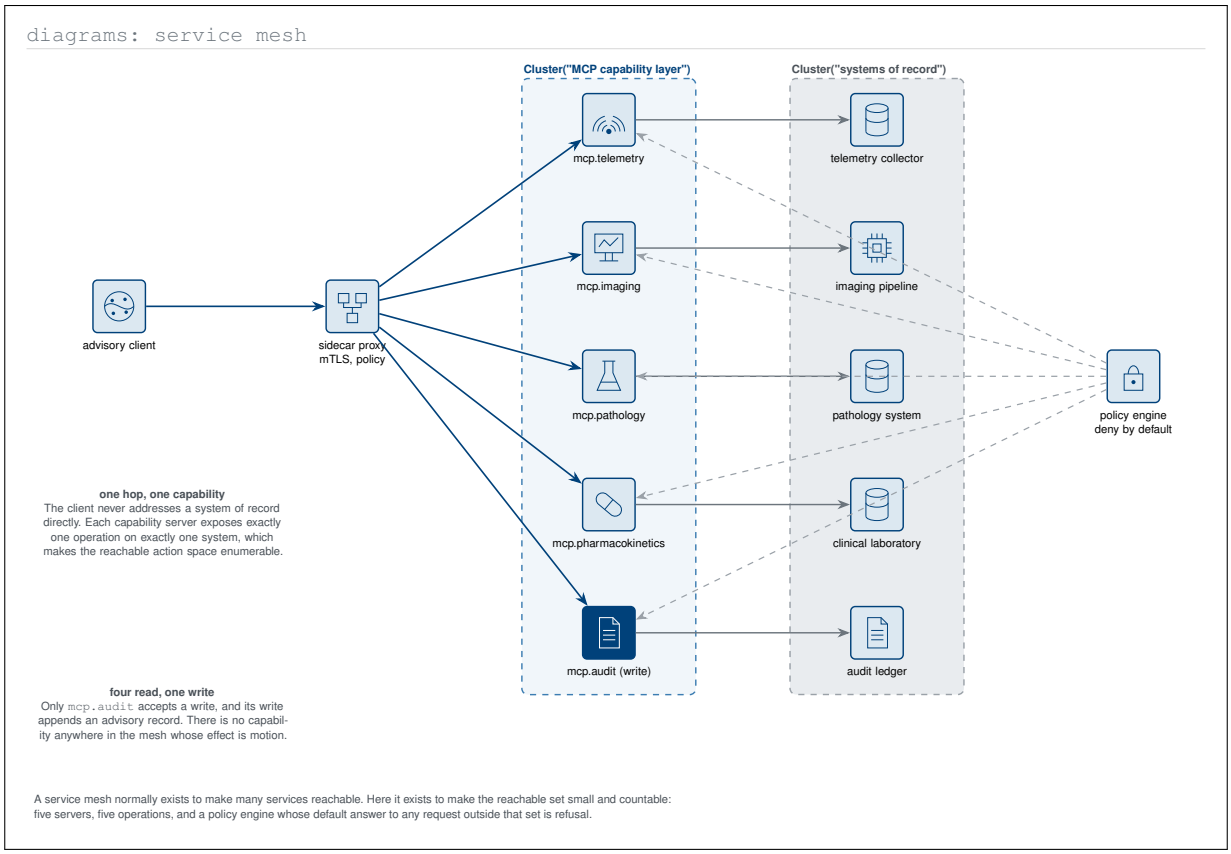


Figure 12. Diagrams (Python)-type service mesh of the five Model Context Protocol servers with a sidecar proxy and a deny-by-default policy engine. The mesh is used inversely to its usual purpose: not to make many services reachable, but to make the reachable action set small, countable, and free of any capability whose effect is motion.

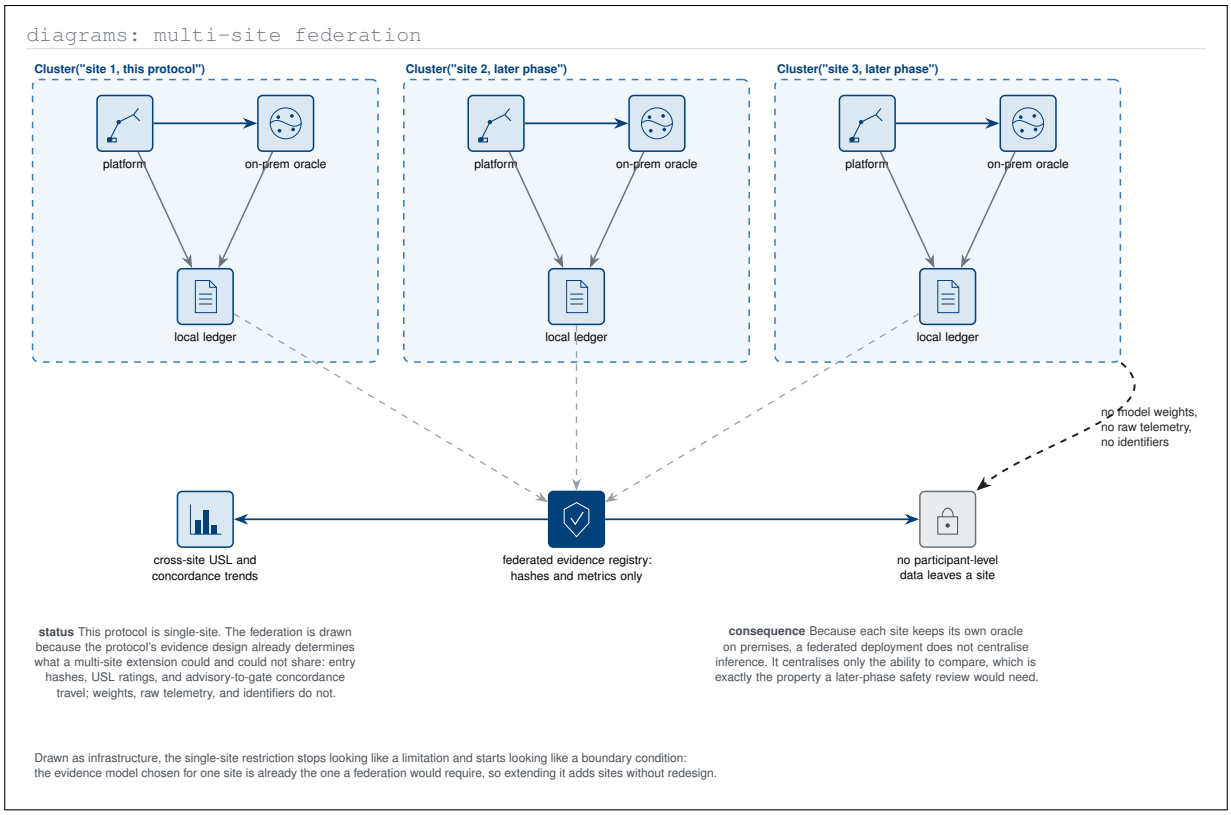


Figure 13. Diagrams (Python)-type federation view showing what a later-phase multi-site extension could and could not share. Entry hashes, readiness ratings, and concordance metrics travel while weights, raw telemetry, and identifiers do not, so the single-site evidence model is already the one a federation would require.

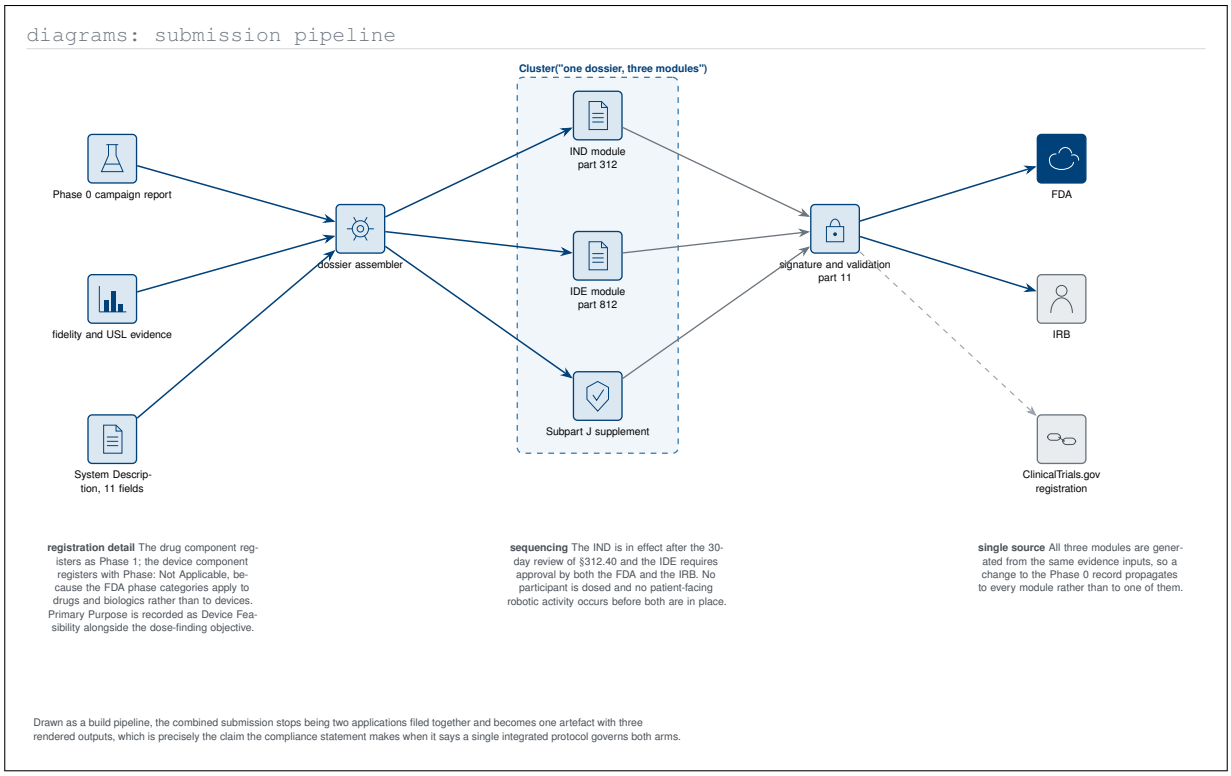


Figure 14. Diagrams (Python)-type submission pipeline treating the combined dossier as a build with one set of evidence inputs and three rendered modules. Drawn this way the filing stops being two applications submitted together and becomes one artefact, which is what a single integrated protocol means in practice.

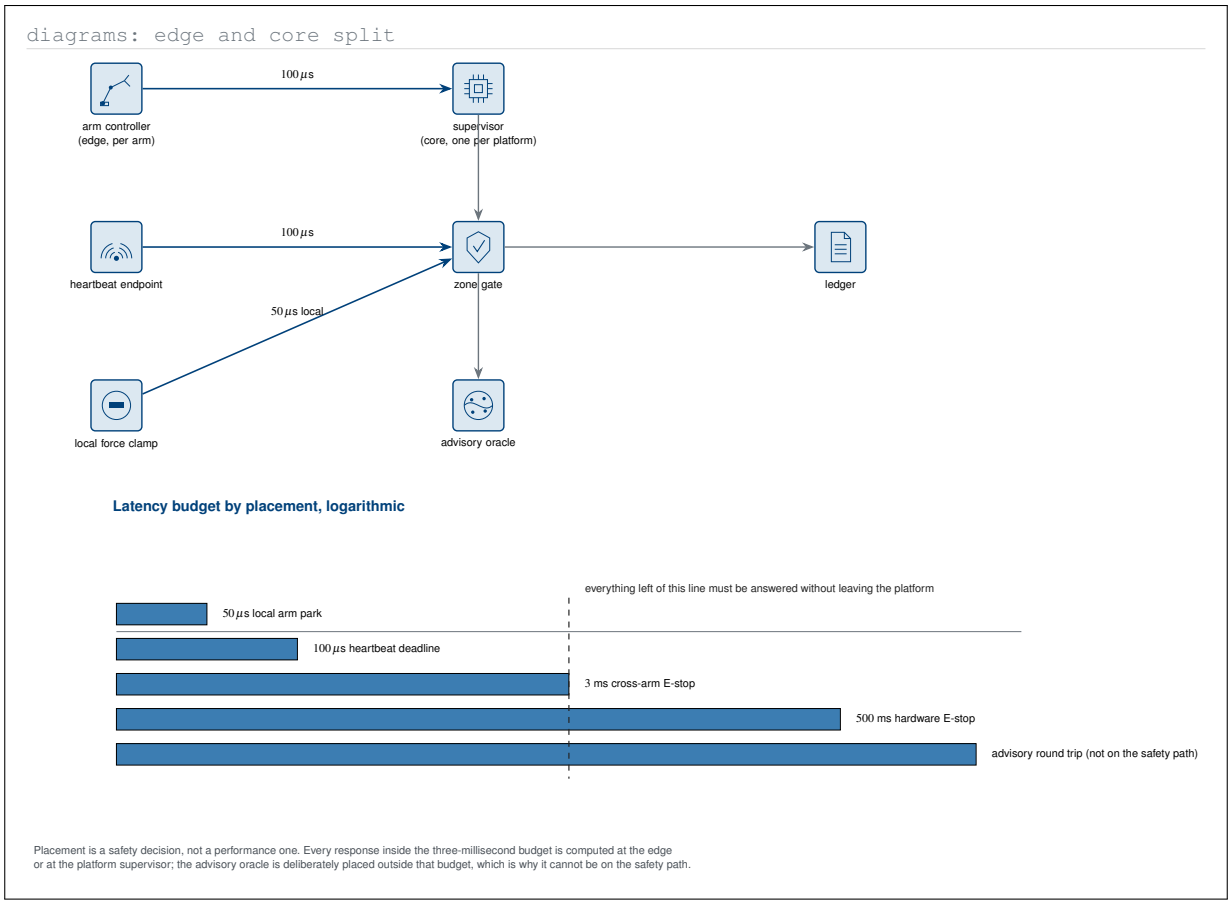


Figure 15. Diagrams (Python)-type edge-and-core split with the latency budget plotted beneath on a logarithmic scale. Placement is shown to be a safety decision rather than a performance one: everything inside the three millisecond budget is computed on the platform, and the advisory oracle is deliberately placed outside it.

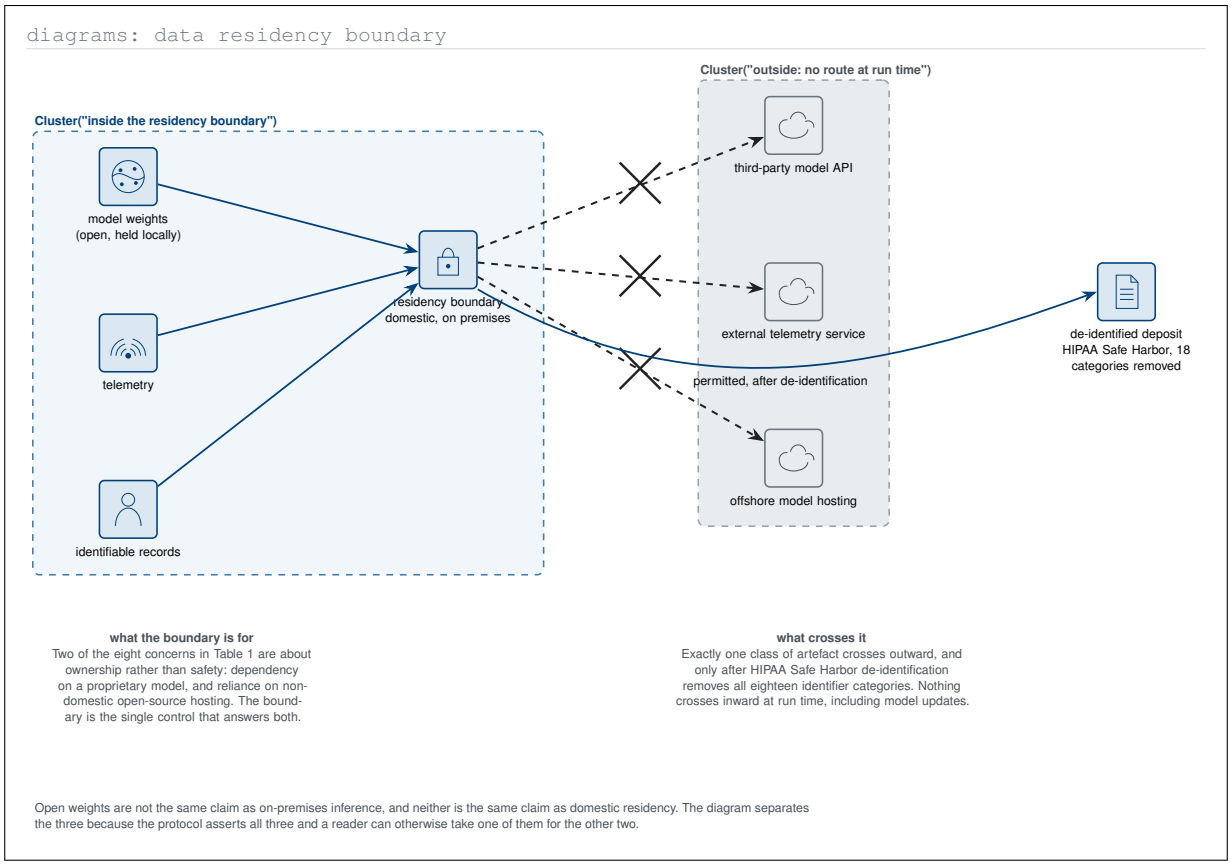


Figure 16. Diagrams (Python)-type residency boundary separating three claims the protocol asserts together and a reader can easily conflate: open weights, on-premises inference, and domestic residency. Exactly one class of artefact crosses outward, after Safe Harbor de-identification, and nothing crosses inward at run time.

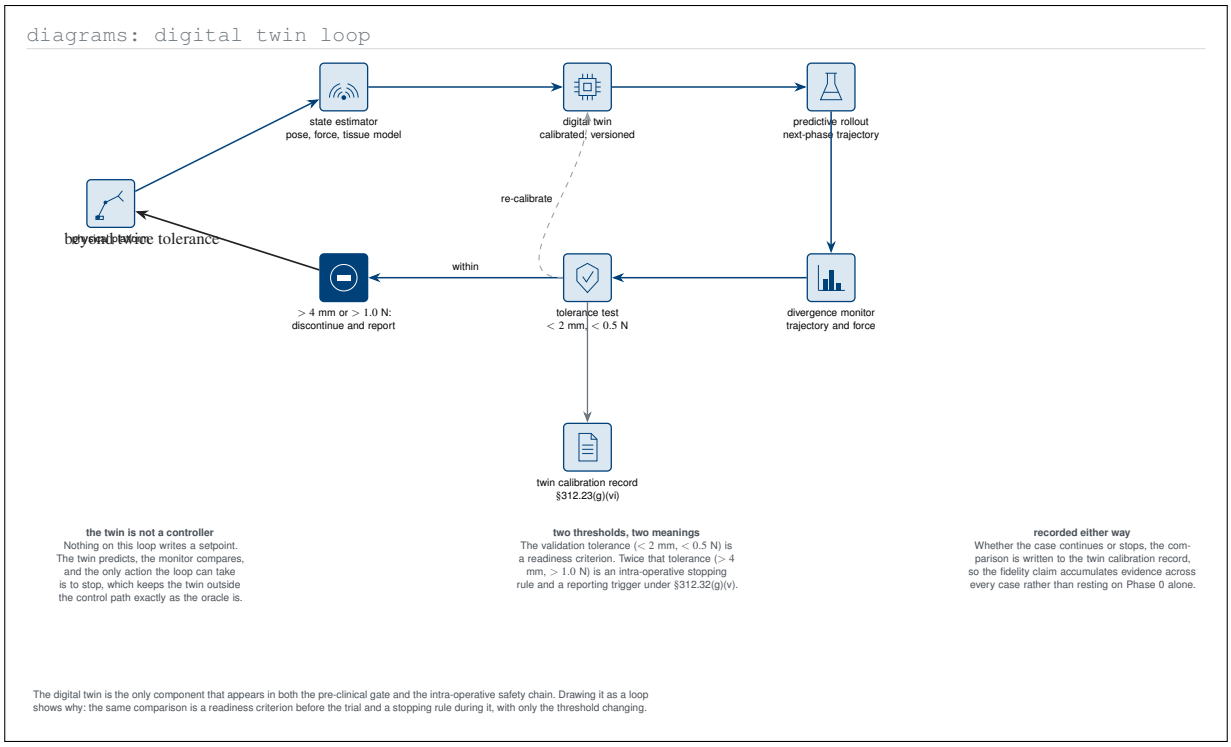


Figure 17. Diagrams (Python)-type digital-twin loop showing the same comparison serving as a readiness criterion before the trial and a stopping rule during it, with only the threshold changing. Nothing on the loop writes a setpoint, so the twin sits outside the control path exactly as the advisory oracle does.

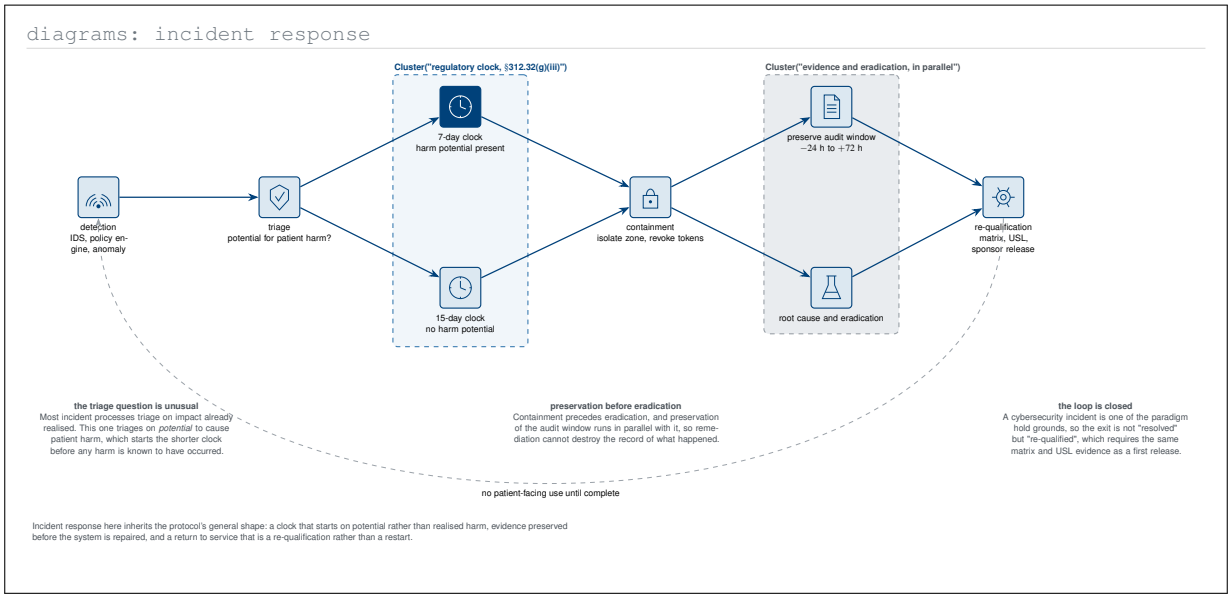


Figure 18. Diagrams (Python)-type incident-response architecture for the cybersecurity reporting trigger. Two features distinguish it from a conventional process: the clock starts on potential rather than realised patient harm, and preservation of the audit window runs in parallel with containment so that remediation cannot destroy the record.

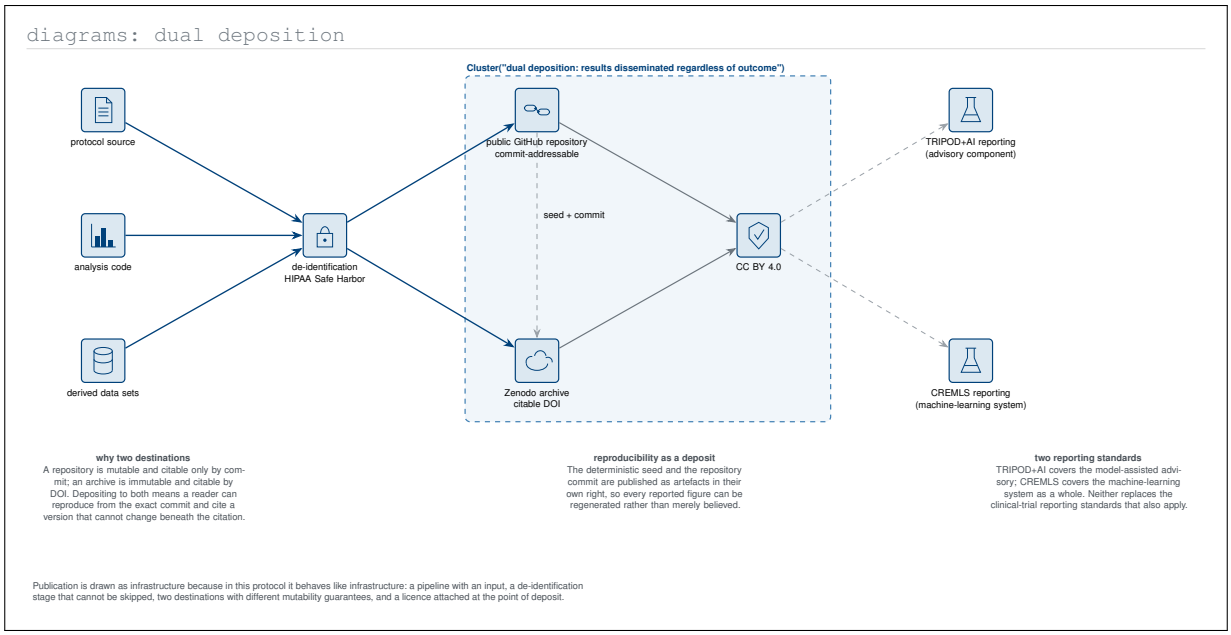


Figure 19. Diagrams (Python)-type dual-deposition pipeline treating publication as infrastructure with a mandatory de-identification stage and two destinations that differ in mutability. Depositing to both a commit-addressable repository and a DOI-addressable archive is what lets a reader reproduce from an exact commit while citing a version that cannot change.

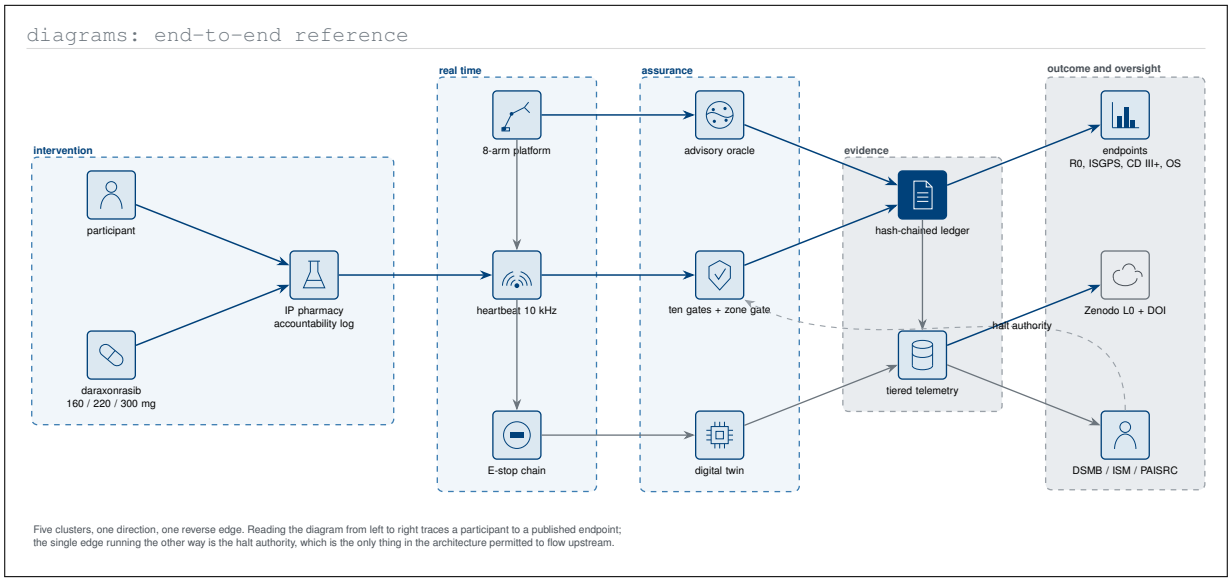


Figure 20. Diagrams (Python)-type end-to-end reference architecture of the whole investigation in five clusters. The figure closes the section on the same observation the deployment figure opened it with: the architecture flows in one direction from participant to published endpoint, and the only edge permitted to run upstream is the halt authority.

6 Graphviz-type Diagrams

Graphviz is the oldest of the six and the least concerned with appearance: it takes a graph and lays it out, and its engines differ only in what they optimise [17]. `dot` ranks a directed acyclic graph into layers, `neato` and `fdp` find a force-directed embedding, `circo` arranges a cyclic topology on a ring, and `twopi` grows a tree radially from a root. Record and HTML-like nodes let a table be a vertex. This makes Graphviz the right instrument for the questions a protocol raises that are purely structural: what depends on what, what reaches what, what is reachable from a failure, and what is left unconnected. The twenty figures below are referred to as **Graphviz-type** figures, each tagged with the engine and the graph attribute that would produce it.

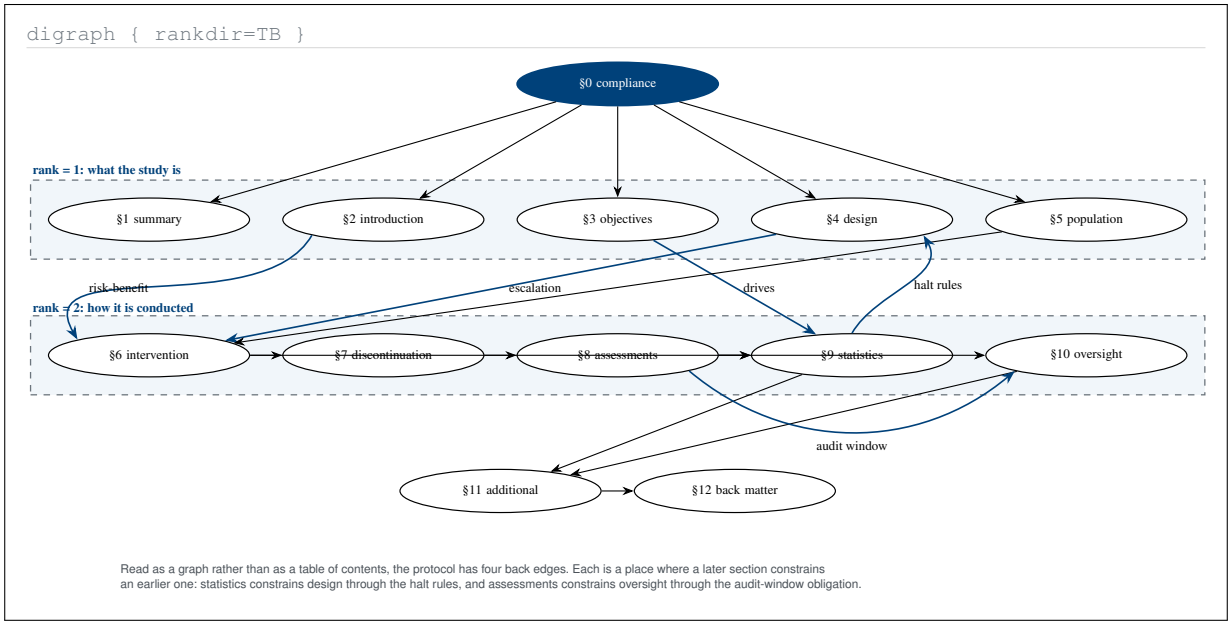


Figure 1. Graphviz-type directed graph of the protocol's own section dependencies, ranked by the `dot` engine rather than ordered by number. The four back edges are the interesting structure: each marks a place where a later section constrains an earlier one, which a linear table of contents cannot show.

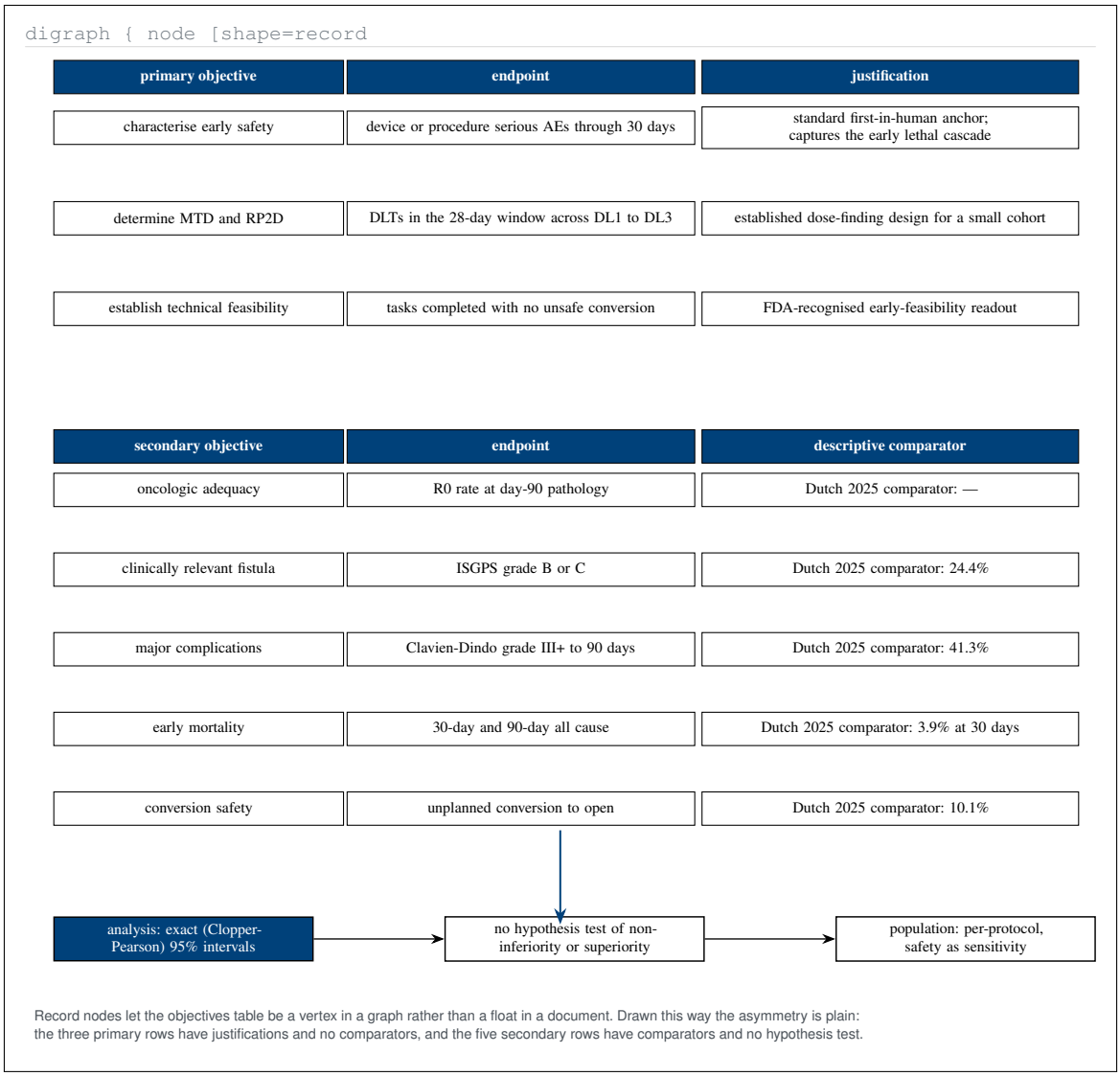


Figure 2. Graphviz-type record-node rendering of the objectives and endpoints table as a graph vertex. Laid out this way the asymmetry between the two blocks is immediate: primary rows carry justifications and no comparator, secondary rows carry a comparator and no test, and every row terminates at the same exact interval method.

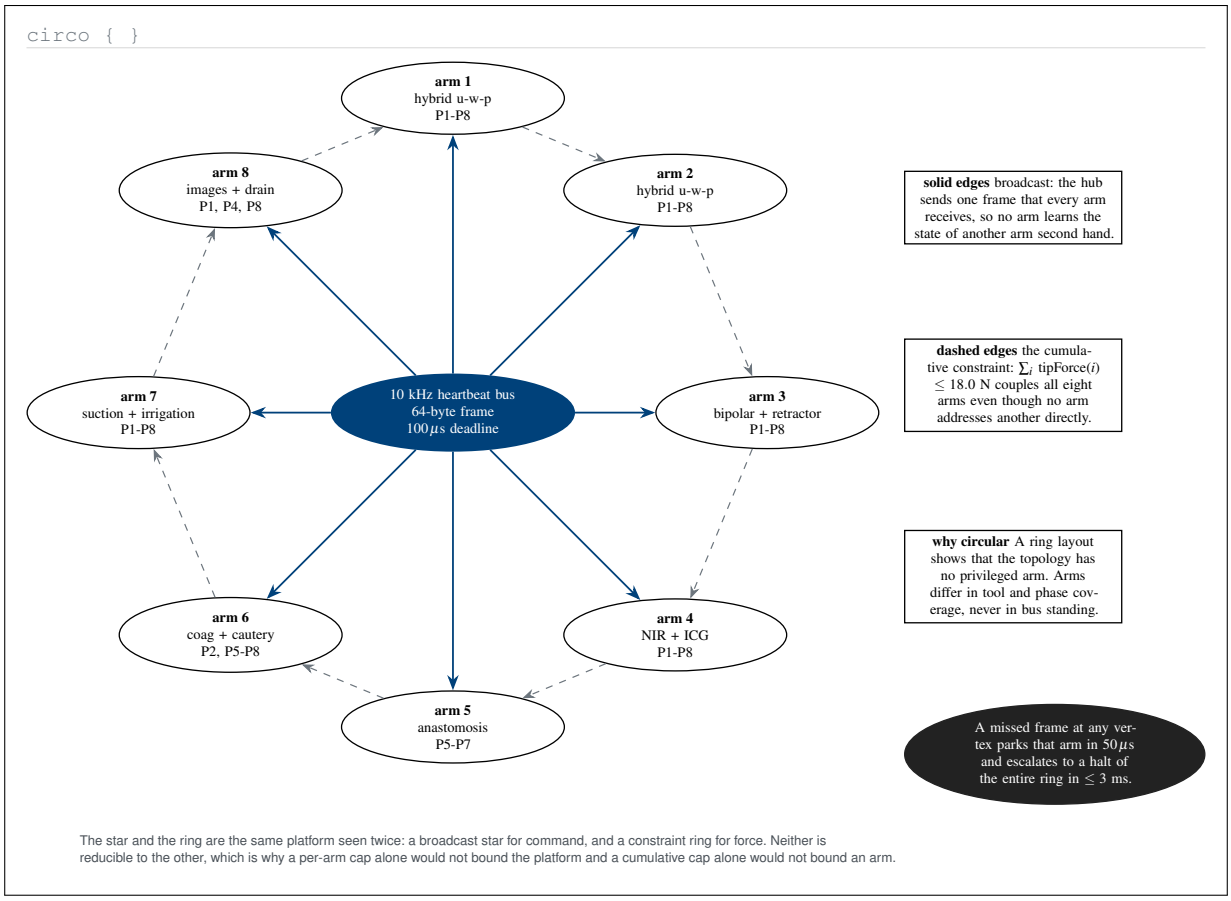


Figure 3. Graphviz-type `circo` layout of the eight arms on the heartbeat bus, showing the platform as a broadcast star and a constraint ring at once. Neither topology reduces to the other, which is why a per-arm cap alone would not bound the platform and a cumulative cap alone would not bound an arm.

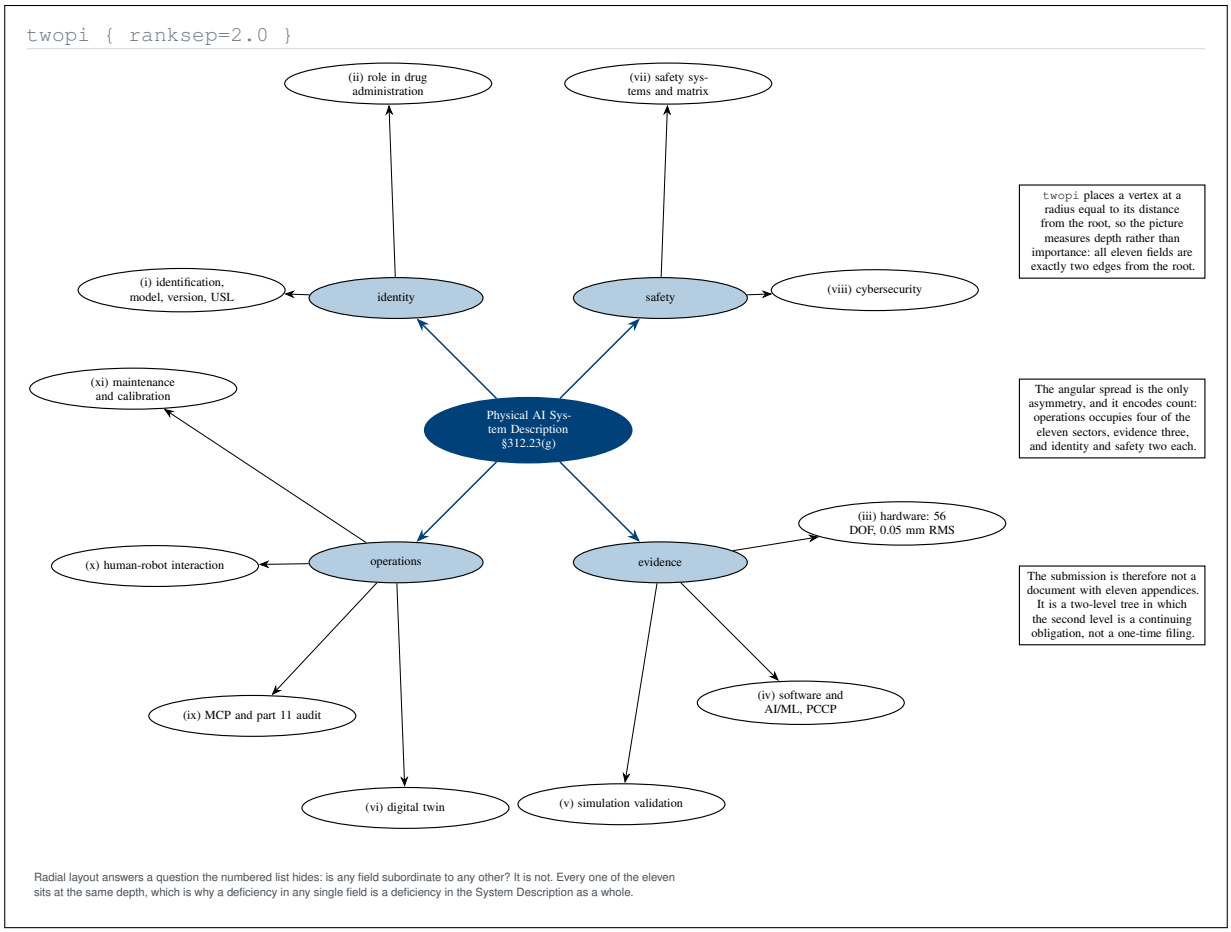


Figure 4. Graphviz-type `twopi` radial layout of the eleven System Description fields. Radial distance encodes depth rather than importance, and every field sits at the same depth, which is why a deficiency in any one of them is a deficiency in the System Description as a whole rather than in an appendix.

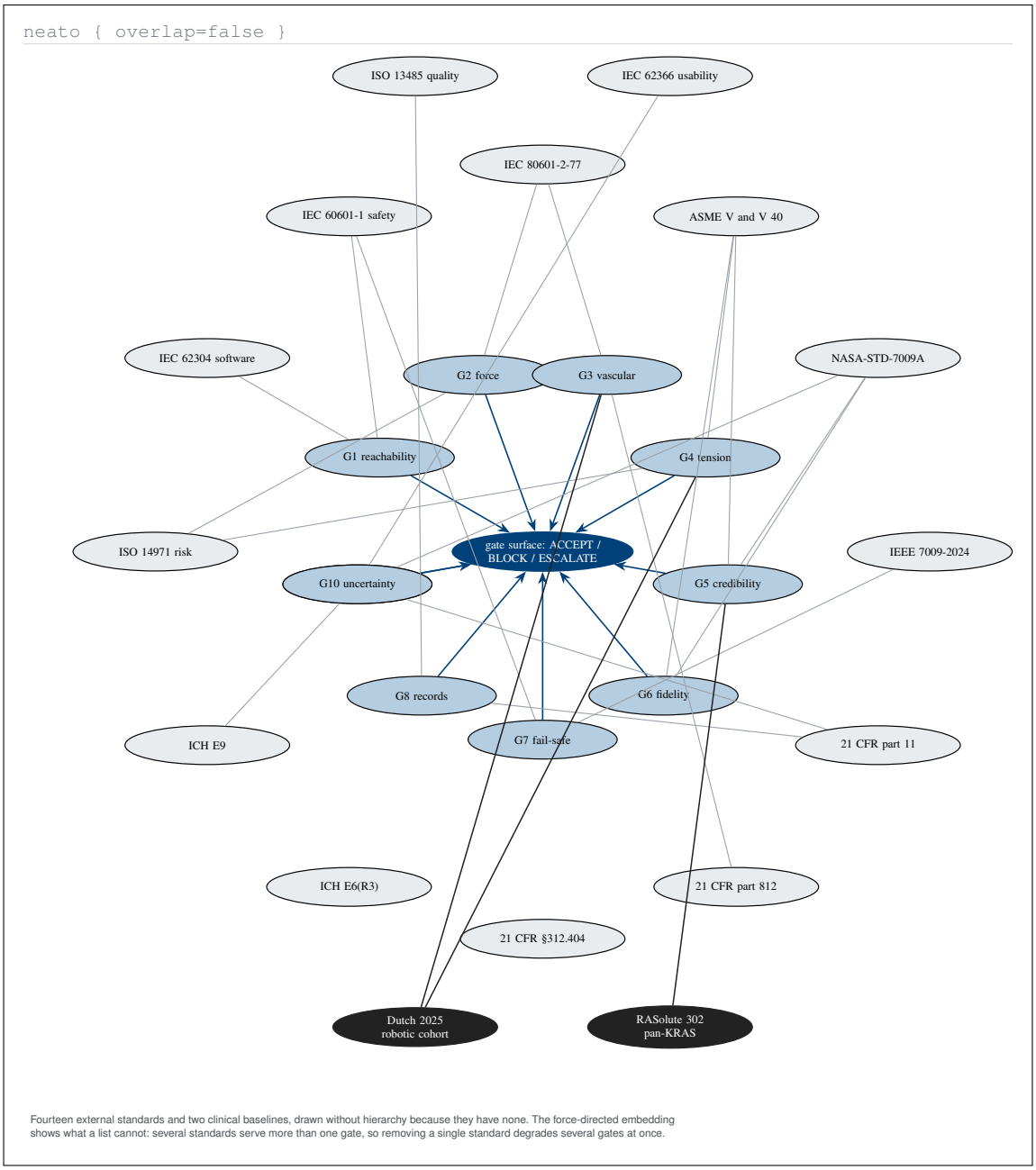


Figure 5. Graphviz-type `neato` force-directed embedding of the ten assurance gates against the fourteen external standards and two clinical baselines they implement. The embedding shows what an enumerated list cannot, namely that several standards serve more than one gate, so removing one degrades several gates simultaneously.

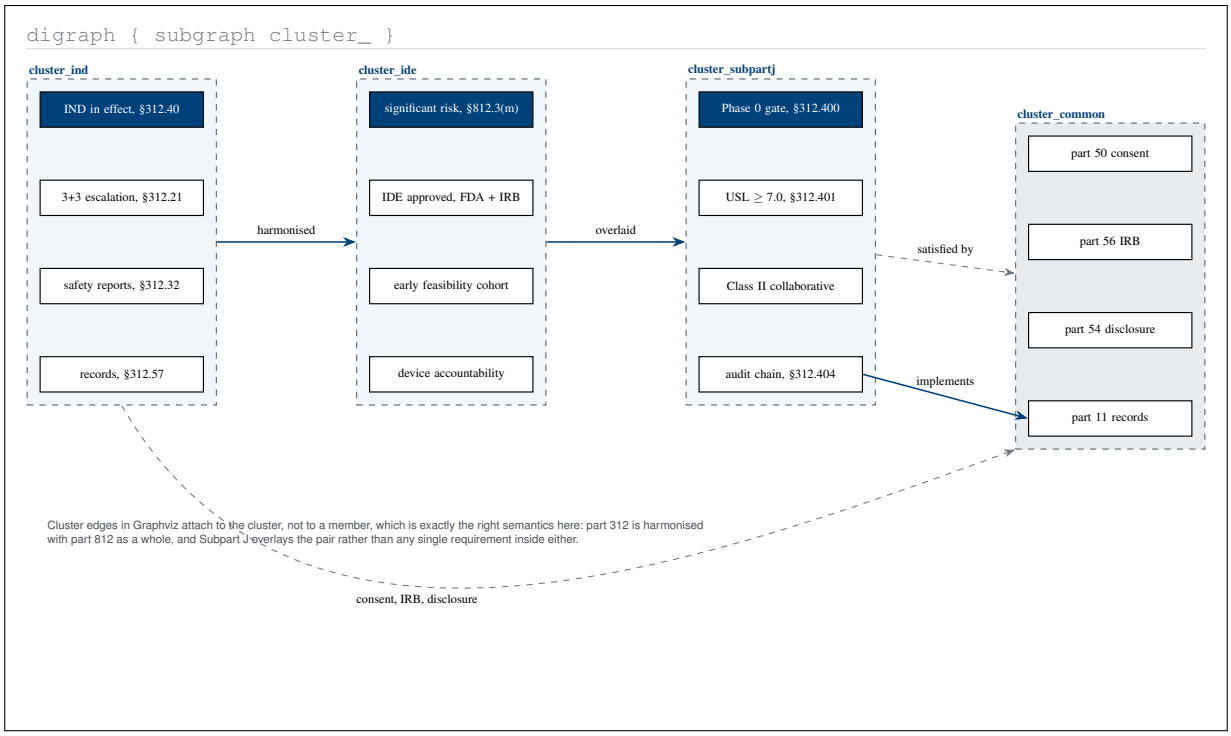


Figure 6. Graphviz-type clustered subgraph of the regulatory spine, using the property that cluster edges attach to the cluster rather than to a member. That is the correct semantics for this protocol: part 312 is harmonised with part 812 as a whole, and Subpart J overlays the pair rather than any requirement inside either.

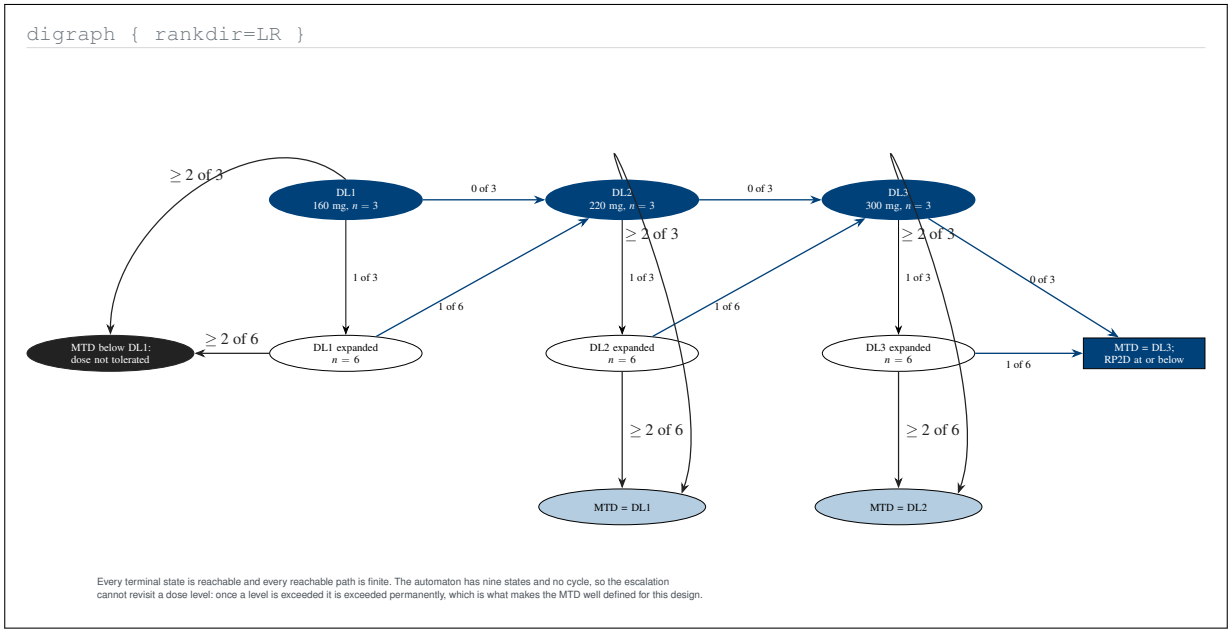


Figure 7. Graphviz-type left-to-right automaton of the complete 3+3 escalation, with all nine states and every labelled transition. Because the graph is acyclic no dose level can be revisited, which is the structural property that makes the maximum tolerated dose well defined for this design rather than merely conventional.

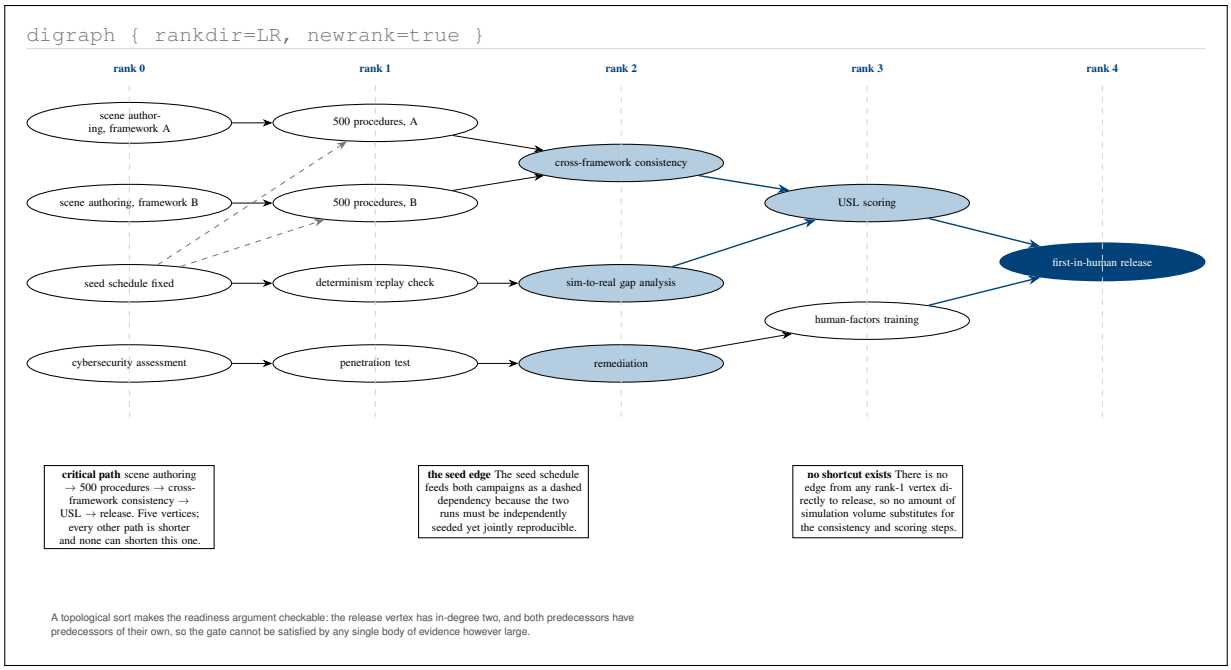


Figure 8. Graphviz-type topological ordering of the Phase 0 prerequisites, ranked into five layers with the critical path marked. The release vertex has in-degree two and no rank-one vertex reaches it directly, so no volume of simulation evidence alone can satisfy the gate without the consistency and scoring steps.

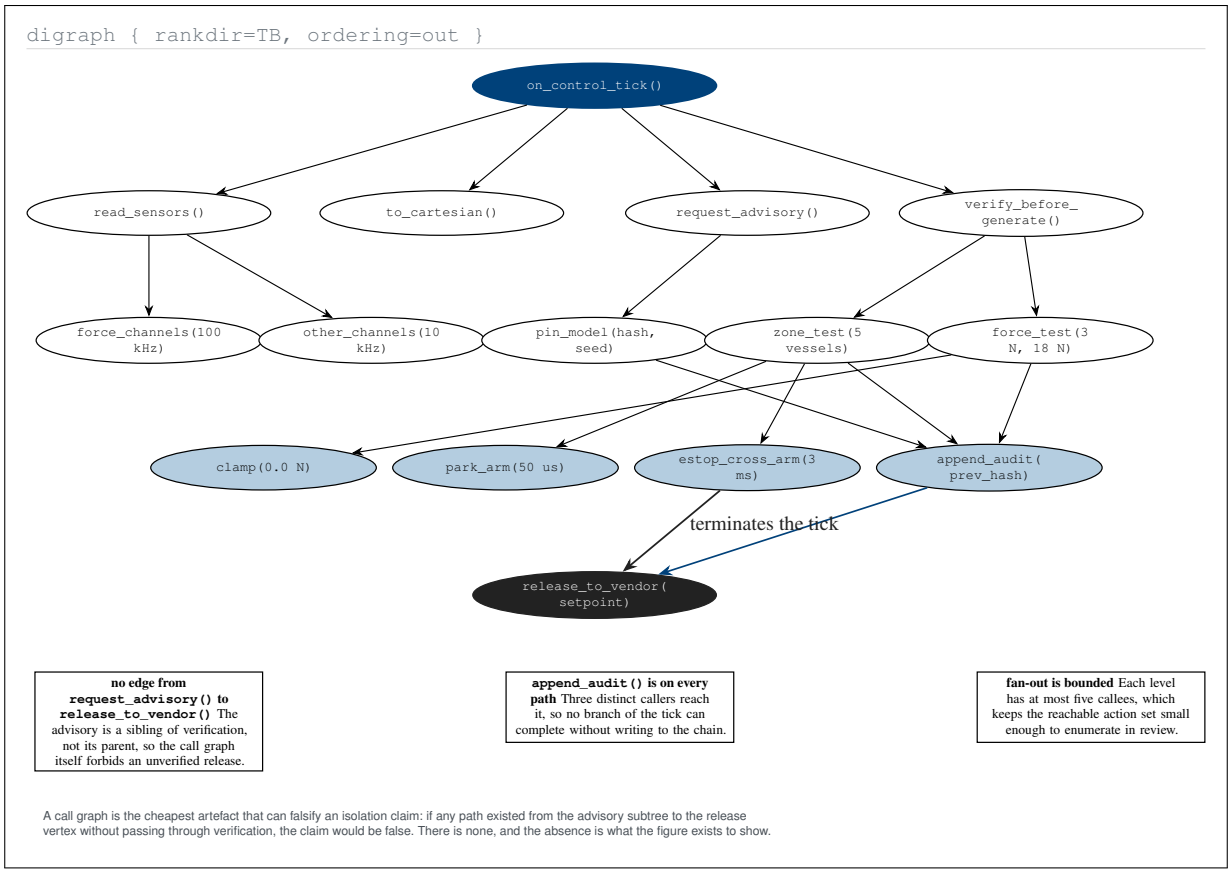
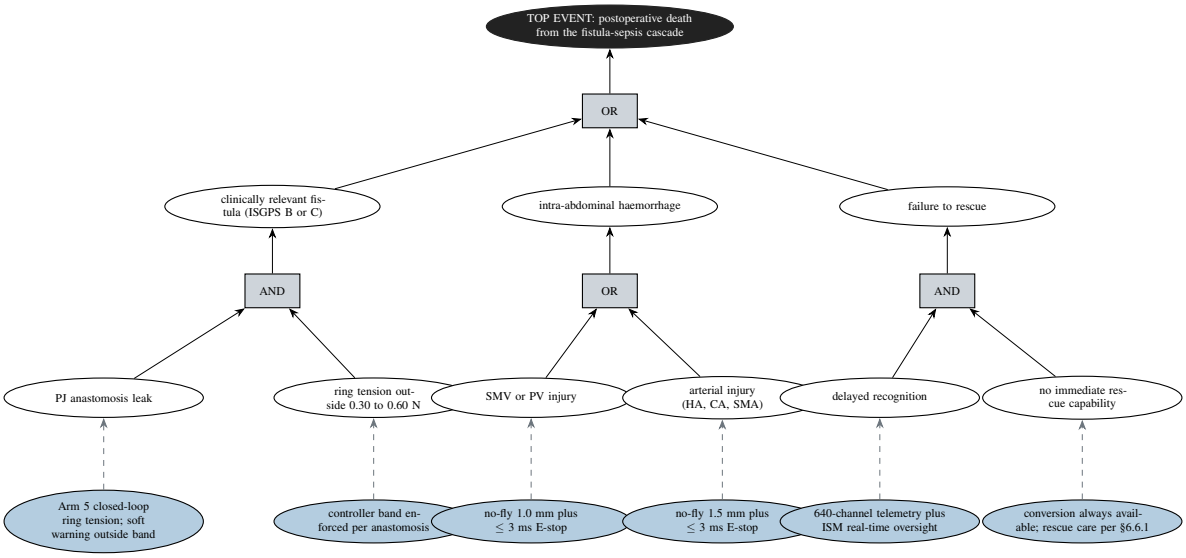


Figure 9. Graphviz-type call graph of one control tick, drawn because it is the cheapest artefact that could falsify the isolation claim. No path runs from the advisory subtree to the release vertex without passing through verification, and the audit append is reached on every branch, so neither property depends on discipline at run time.

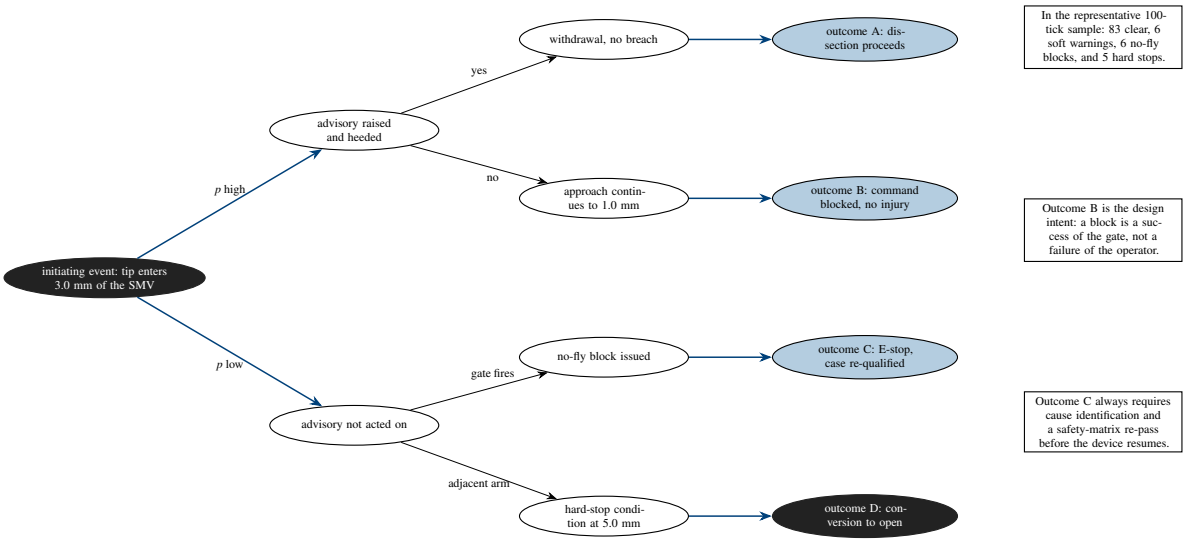
digraph { fault tree }



Every basic event carries a mitigation, and the two AND gates are where the protocol concentrates its effort: an AND gate fails only if both inputs occur, so breaking either input is sufficient, which is why tension control and real-time monitoring are worth their cost.

Figure 10. Graphviz-type fault tree for the postoperative fistula-sepsis cascade, with the protocol's mitigation attached to every basic event. The two AND gates are where effort concentrates, because an AND gate fails only if both inputs occur, which is what makes ring-tension control and real-time monitoring worth their cost.

digraph { event tree, rankdir=LR }



An event tree makes the counting rule explicit. Only one of the four outcomes is an adverse result for the study, and two of the four are the safety system working exactly as designed, which is why block and stop counts are reported separately from conversion counts.

Figure 11. Graphviz-type event tree from a single vessel approach to its four terminal outcomes. The tree makes the counting rule explicit: two of the four outcomes are the safety system working as designed, which is why blocks and stops are reported separately from conversions in the feasibility endpoint.

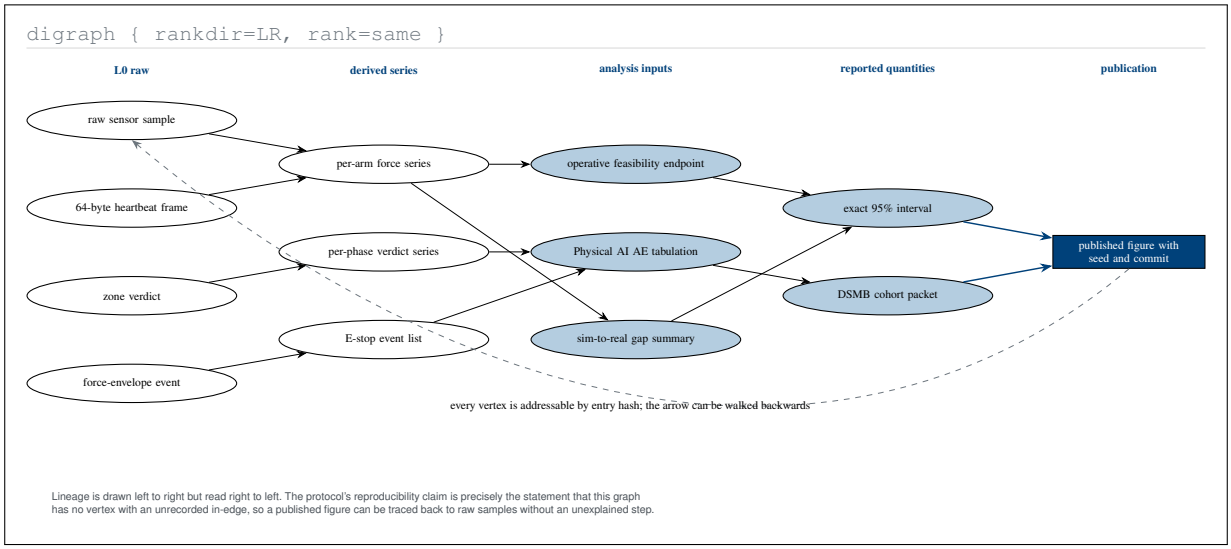


Figure 12. Graphviz-type data-lineage graph from raw samples to a published figure, drawn left to right and intended to be read right to left. The reproducibility claim is exactly the statement that no vertex in this graph has an unrecorded in-edge, so any published quantity traces back to raw samples without an unexplained step.

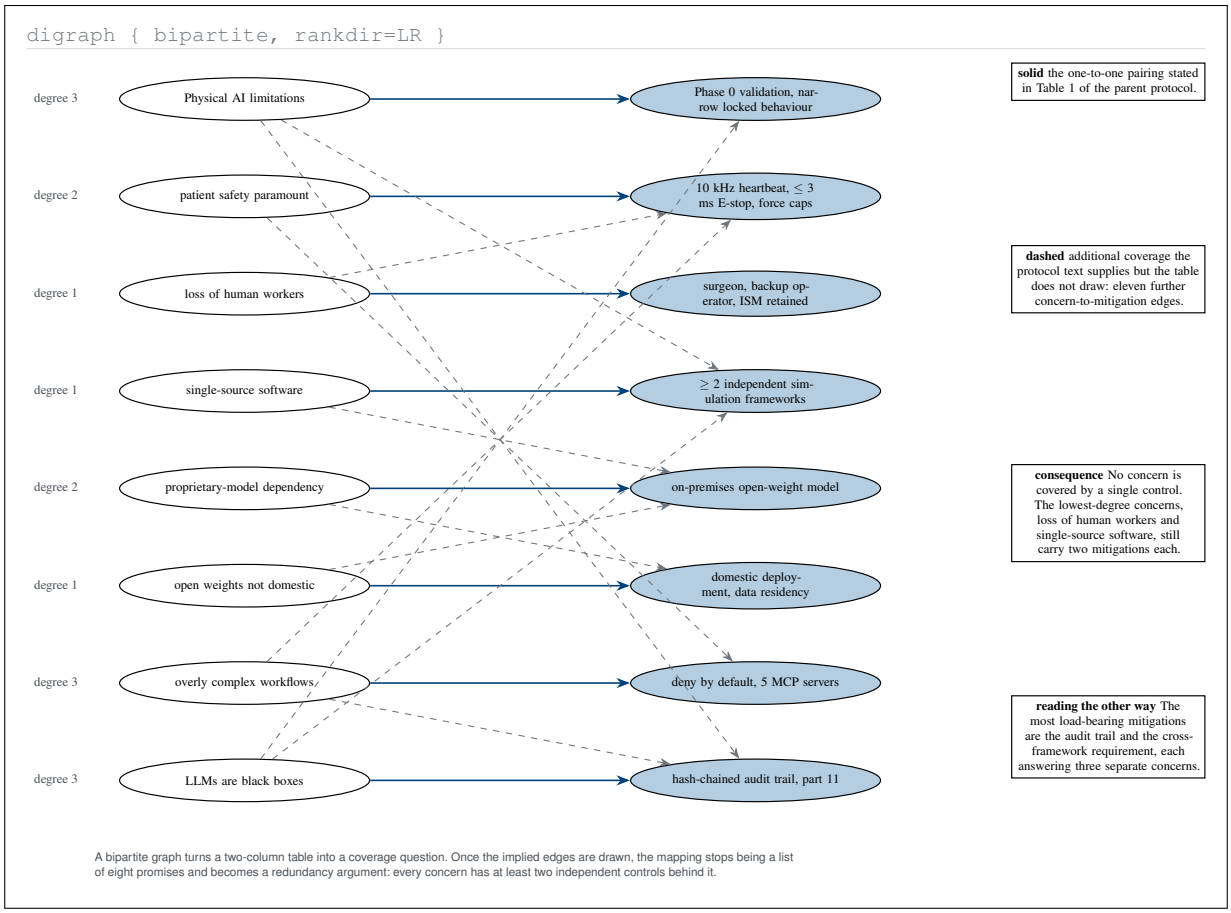


Figure 13. Graphviz-type bipartite graph of the eight Physical AI concerns against their mitigations, with the one-to-one pairing of Table 1 drawn solid and the additional coverage implied by the protocol text drawn dashed. Once the implied edges appear, the mapping becomes a redundancy argument rather than a list of eight promises.

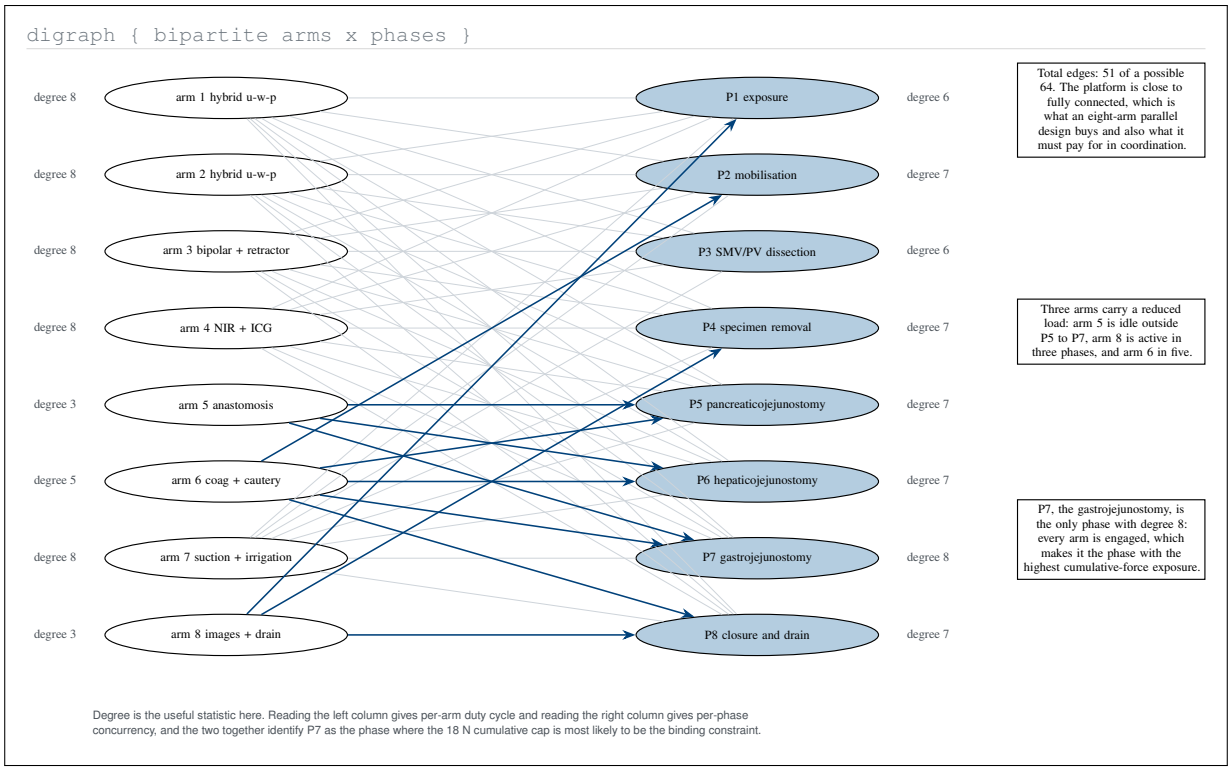


Figure 14. Graphviz-type bipartite coverage graph of the eight arms against the eight operative phases, annotated with vertex degrees. Reading degrees on the left gives per-arm duty cycle and on the right per-phase concurrency, together identifying the gastrojejunostomy as the phase where the cumulative force cap is most likely to bind.

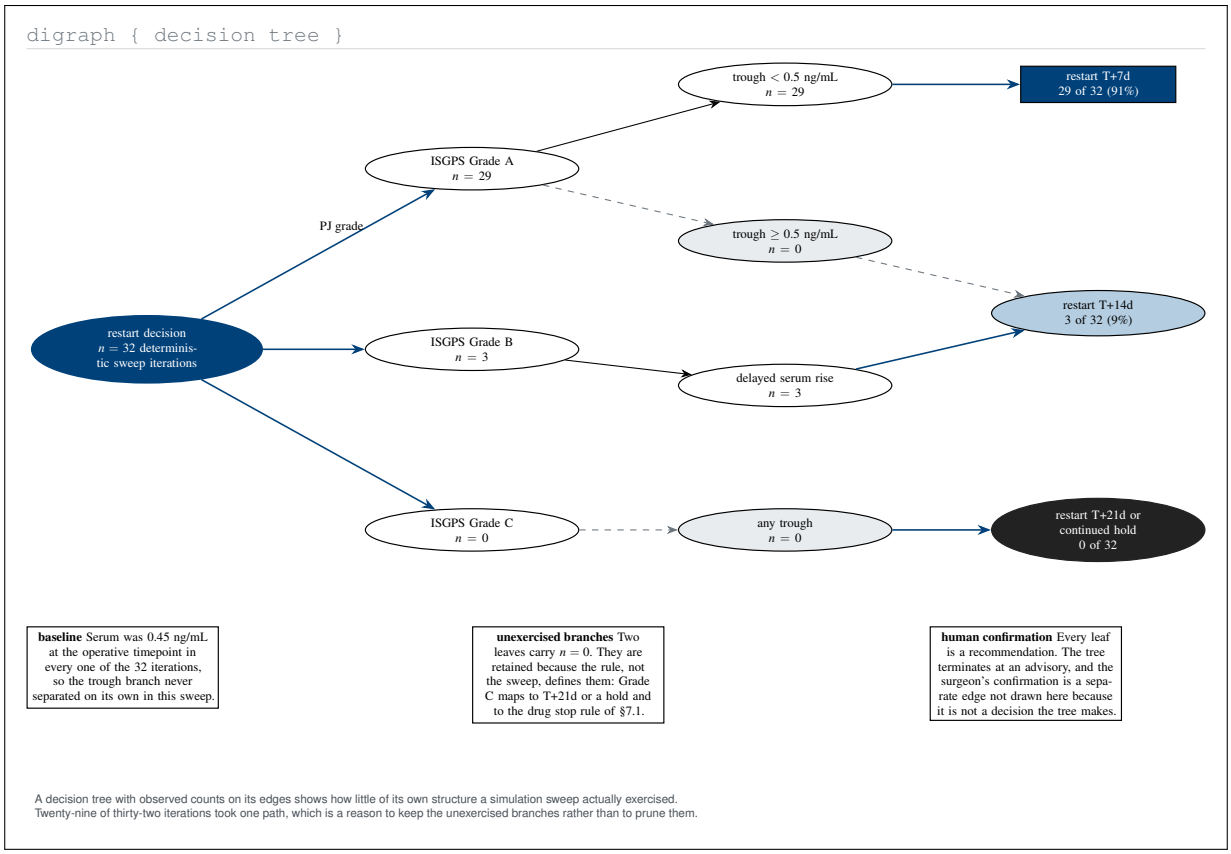


Figure 15. Graphviz-type decision tree of the restart advisory with the observed counts from the 32-iteration deterministic sweep on its edges. Twenty-nine of thirty-two iterations took a single path, which shows how little of its own structure the sweep exercised and argues for retaining rather than pruning the unexercised branches.

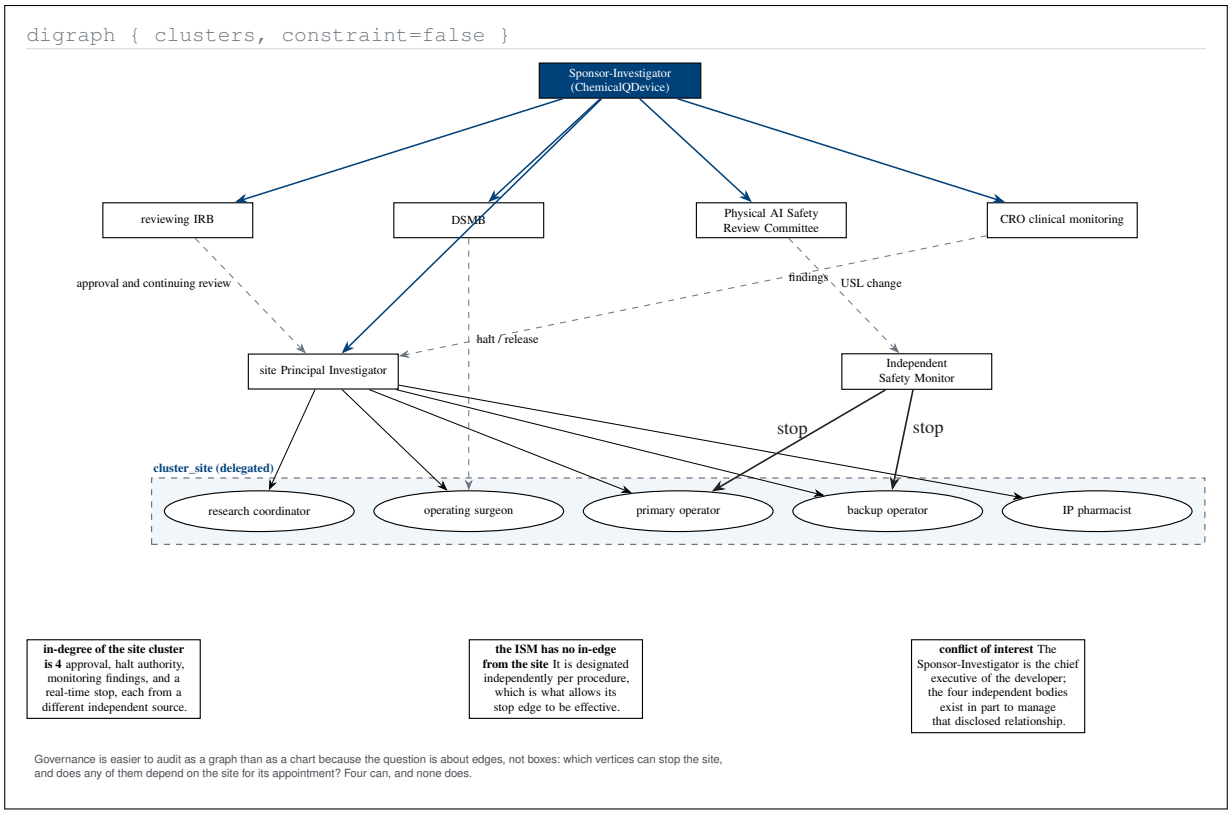


Figure 16. Graphviz-type governance graph in which the audit question is about edges rather than boxes. Four independent vertices can stop the site cluster, and none of them depends on the site for its appointment, which is the property the conflict-of-interest disclosure requires the structure to have.

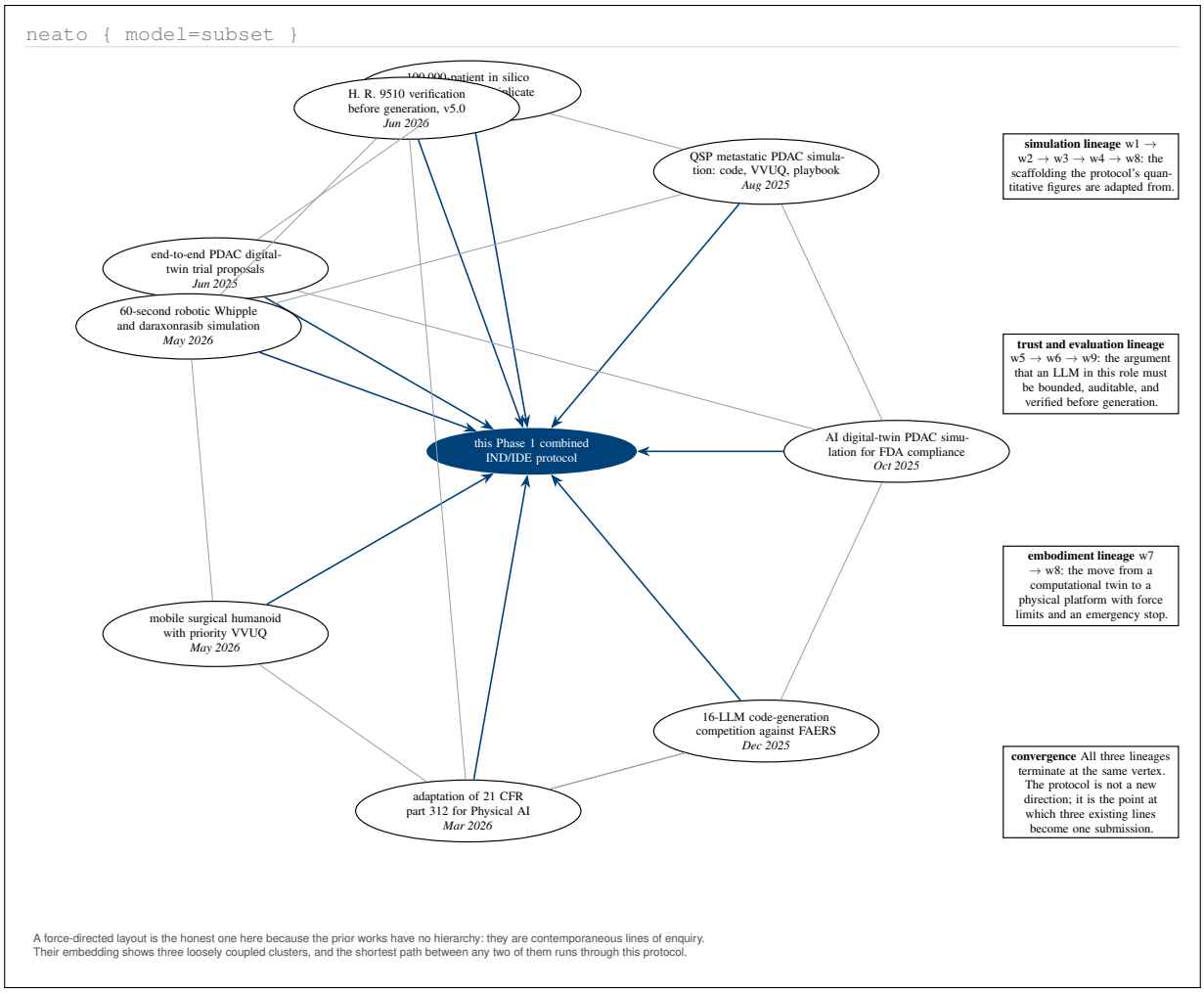


Figure 17. Graphviz-type `neato` embedding of the nine prior computational works cited in the parent introduction, with the protocol at the centre. The embedding separates three loosely coupled lineages, simulation, trust and evaluation, and embodiment, whose shortest connecting path runs through this protocol.

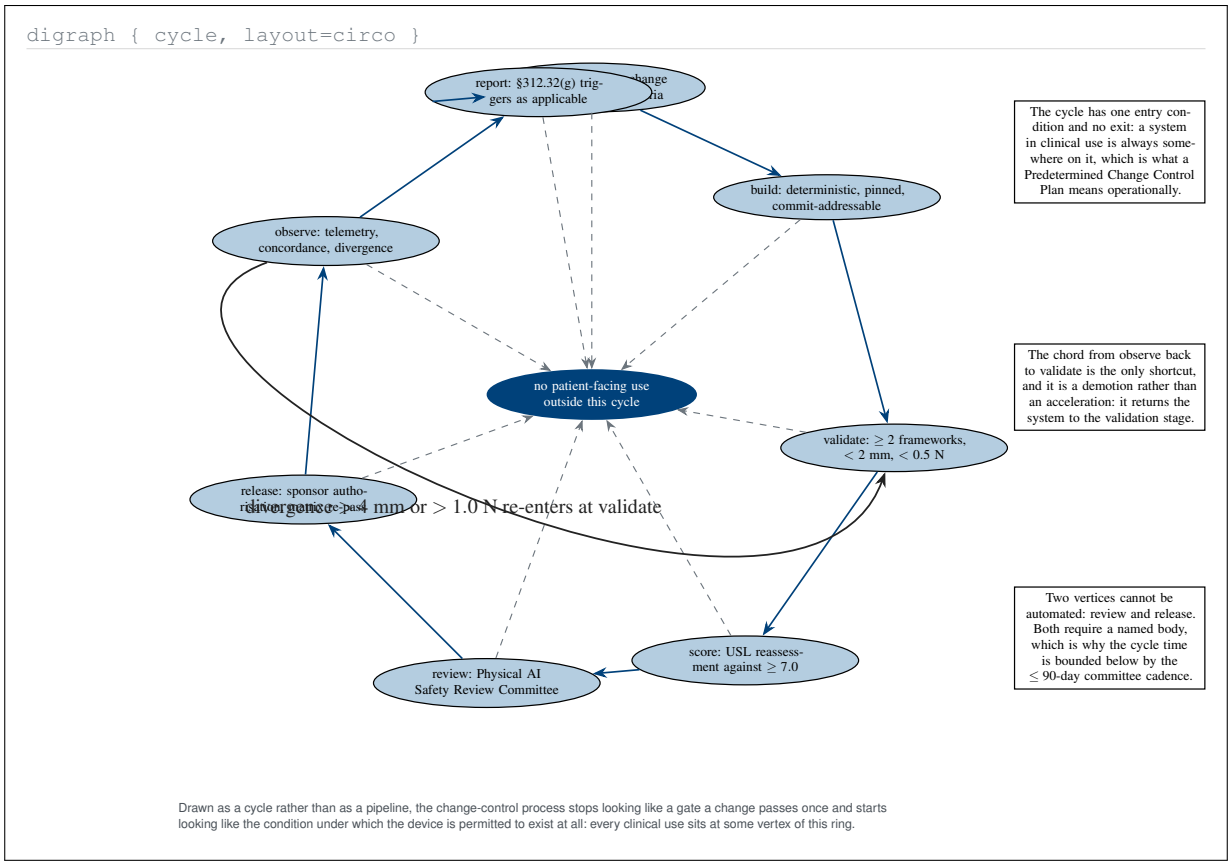


Figure 18. Graphviz-type circular layout of the change-control cycle. Drawn as a ring rather than a pipeline, the Predetermined Change Control Plan stops being a gate a change passes once and becomes the condition under which the device is permitted to exist, with the only chord being a demotion back to validation.

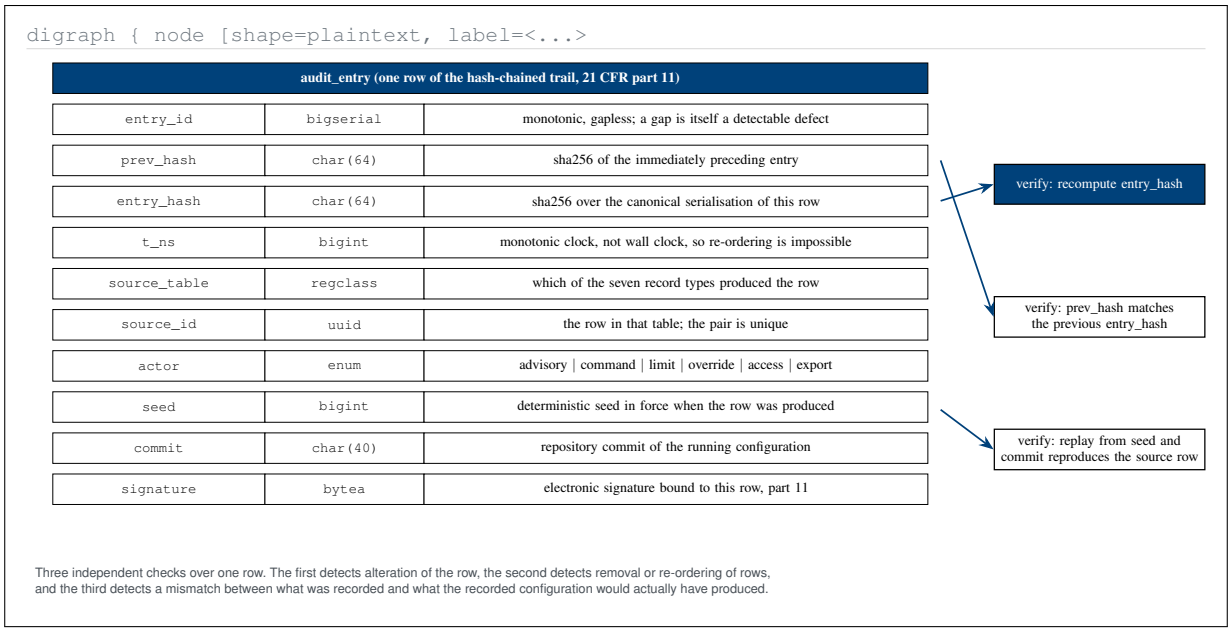


Figure 19. Graphviz-type HTML-like table node giving the audit-trail entry its full column schema. The three verification edges are independent: one detects alteration of a row, one detects removal or re-ordering of rows, and one detects a mismatch between the record and what the recorded configuration would have produced.

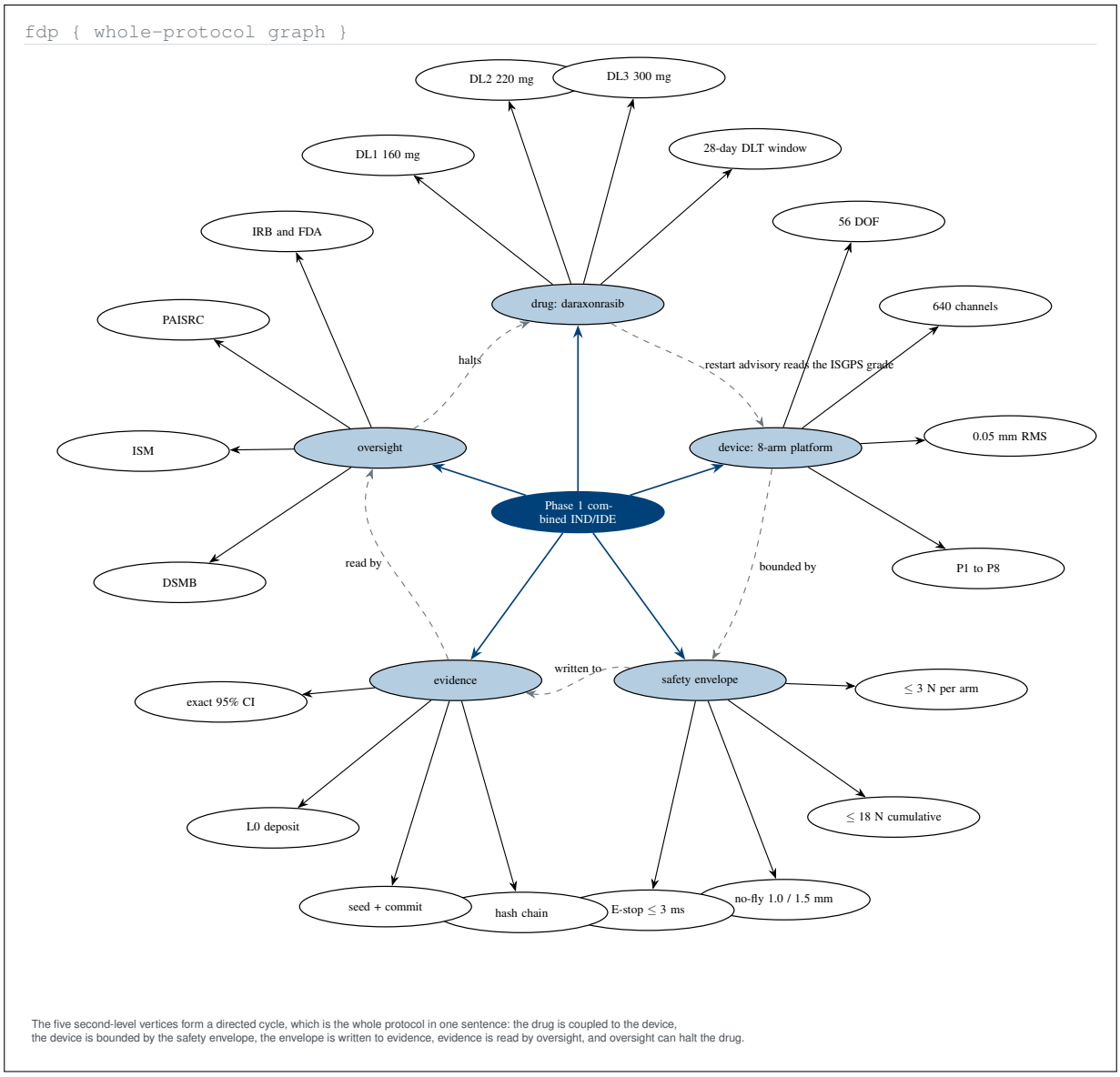


Figure 20. Graphviz-type `fdp` embedding of the whole protocol as a knowledge graph with twenty leaf constants. The five second-level vertices form a directed cycle, which states the protocol in one traversal: drug coupled to device, device bounded by envelope, envelope written to evidence, evidence read by oversight, and oversight able to halt the drug.

References

References

[1] T. Tantau and PGF/TikZ Team, *The TikZ and PGF Packages: Manual for Version 3.1.10*. Comprehensive TeX Archive Network, 2023. GPL / LPPL / GNU FDL, https://ctan.org/pkg/pgf.

[2] R. L. Siegel, A. N. Giaquinto, and A. Jemal, "Cancer statistics, 2025," *CA: A Cancer Journal for Clinicians*, vol. 75, no. 1, pp. 10–45, 2025. 10.3322/caac.21871, https://doi.org/10.3322/caac.21871.

[3] Dutch Pancreatic Cancer Group, "Nationwide outcomes of 1000 robotic pancreatoduodenectomies across the four phases of the learning curve," *British Journal of Surgery*, 2025. 10.1093/bjs/znaf210, https://doi.org/10.1093/bjs/znaf210.

[4] C. Bassi, G. Marchegiani, C. Dervenis, *et al.*, "The 2016 update of the isgps definition and grading of postoperative pancreatic fistula," *Surgery*, vol. 161, no. 3, pp. 584–591, 2017. 10.1016/j.surg.2016.11.014, https://doi.org/10.1016/j.surg.2016.11.014.

[5] Revolution Medicines, "Phase 3 study of daraxonrasib (rmc-6236) in previously treated metastatic pdac (rasolute 302)," 2025. NCT06625320, https://clinicaltrials.gov/study/NCT06625320.

[6] U.S. Food and Drug Administration, "Investigational device exemptions (ides) for early feasibility medical device clinical studies," tech. rep., Center for Devices and Radiological Health, Oct. 2013. https://www.fda.gov/regulatory-information/search-fda-guidance-documents/investigational-device-exemptions-ides-early-feasibility-medical-device-clinical-studies-including.

[7] Electronic Code of Federal Regulations, "21 cfr part 11 - electronic records; electronic signatures," 2026. https://www.ecfr.gov/current/title-21/chapter-I/subchapter-A/part-11.

[8] K. Kawchak, "Adaption: 21 cfr part 312, end-to-end physical ai oncology clinical trial unification," Mar. 2026. 10.5281/zenodo.19057628, https://doi.org/10.5281/zenodo.19057628.

[9] K. Kawchak, "Verification before generation in physical ai oncology trials act of 2026 (h. r. 9510), bill v5.0: A financial data amendment," June 2026. 10.5281/zenodo.20619762, https://doi.org/10.5281/zenodo.20619762.

[10] G. S. Collins *et al.*, "Tripod+ai statement: updated guidance for reporting clinical prediction models that use regression or machine learning methods," *BMJ*, vol. 385, p. e078378, 2024. 10.1136/bmj-2023-078378, https://doi.org/10.1136/bmj-2023-078378.

[11] K. Kawchak, "2030: 60 second pancreatic ductal adenocarcinoma robotic whipple procedure and daraxonrasib simulation," May 2026. 10.5281/zenodo.20196639, https://doi.org/10.5281/zenodo.20196639.

[12] Mermaid Contributors, "Mermaid: Generation of diagrams from text in a similar manner as markdown." MIT License, 2026. MIT License, https://mermaid.js.org.

[13] Excalidraw Contributors, “Excalidraw: Virtual hand-drawn style whiteboard.” MIT License, 2026. MIT License, https://excalidraw.com.

[14] A. Roques and PlantUML Contributors, “Plantuml: Open-source tool using simple textual descriptions to draw uml and non-uml diagrams.” GPL / LGPL / Apache dual licensing, 2026. https://plantuml.com.

[15] Terrastruct and D2 Contributors, “D2: A modern declarative diagram scripting language that turns text to diagrams.” MPL-2.0 License, 2026. MPL-2.0 License, https://d2lang.com.

[16] Diagrams Contributors, “Diagrams: Diagram as code for prototyping cloud system architectures.” MIT License, 2026. MIT License, https://diagrams.mingrammer.com.

[17] E. R. Gansner and S. C. North, “An open graph visualization system and its applications to software engineering,” *Software: Practice and Experience*, vol. 30, no. 11, pp. 1203–1233, 2000. Graphviz, Eclipse Public License, https://graphviz.org.